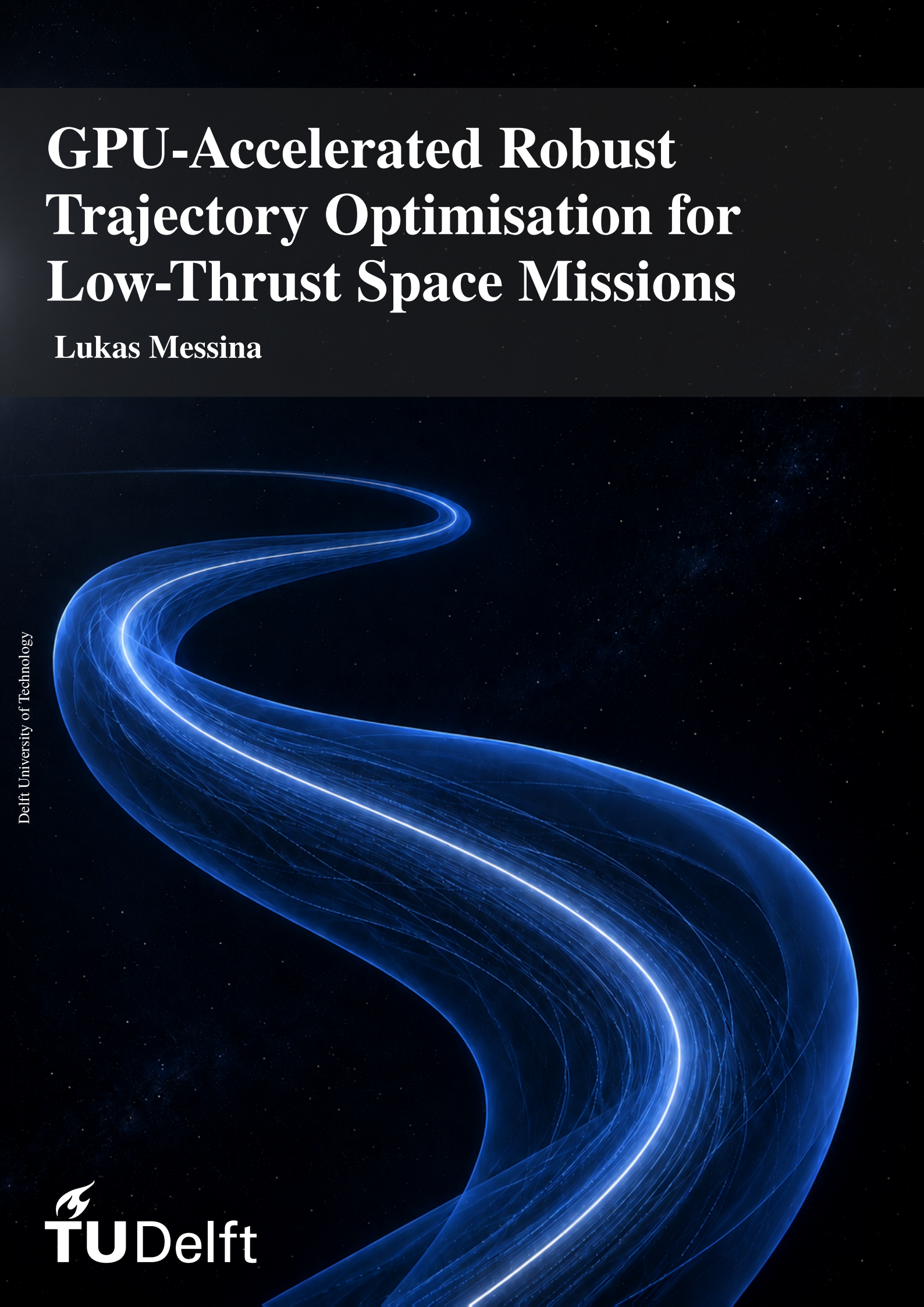
GPU-Accelerated Robust Trajectory Optimisation for Low-Thrust Space Missions

Lukas Messina



Delft University of Technology

TU Delft

GPU-ACCELERATED ROBUST TRAJECTORY OPTIMISATION FOR LOW-THRUST SPACE MISSIONS

by

LUKAS MESSINA

to obtain the degree of Master of Science in Aerospace Engineering at Delft University of Technology, to be defended publicly on –, –, 2026, at –.

Student number:	6084281	
Project duration:	May 11, 2026 – February 2026	
Thesis supervisor:	Dr. ir. E. Mooij	TU Delft, Aerospace Engineering
External supervisor:	Dr. Z. Manchester	MIT, Aeronautics and Astronautics

Cover: An artistic impression of a space trajectory and the uncertainty ensemble.

An electronic version of this thesis is available at `http://repository.tudelft.nl/`.

TU Delft

PREFACE

A preface...

Lukas Messina
Delft, October 2026

SUMMARY

CONTENTS

Preface iii

Summary v

Contents vii

Nomenclature xi

1 Introduction 1
- 1.1 Uncertainties in Mission Design . . . 1
- 1.2 Parallel Computing for Robust Trajectory Optimisation . . . 2
- 1.3 Problem Statement . . . 2
- 1.4 Report Outline . . . 3

2 Mission Heritage 5
- 2.1 Sources of Uncertainty . . . 5
- 2.2 Stochastic Solvers for Mission Analysis . . . 5
- 2.3 State of the Art . . . 6
  - 2.3.1 Robust Trajectory Planning . . . 7
    - 2.3.1.1 Control-Policy Formulations . . . 7
    - 2.3.1.2 Research Directions . . . 7
    - 2.3.1.3 Selected Methodology . . . 8
  - 2.3.2 GPU Acceleration in Mission Analysis . . . 9
  - 2.3.3 Delft University of Technology Research Heritage . . . 10
- 2.4 Low-Thrust Interplanetary Missions . . . 10
- 2.5 Reference Mission . . . 11
  - 2.5.1 History and Characteristics of (16) Psyche . . . 12
  - 2.5.2 Spacecraft . . . 13
  - 2.5.3 Requirements . . . 15

3 Flight Dynamics 19
- 3.1 Reference Frames . . . 19
- 3.2 State Representation . . . 21
- 3.3 Coordinate and Frame Transformations . . . 22
  - 3.3.1 State Conversions . . . 22
    - 3.3.1.1 Frame Rotations . . . 23
- 3.4 Method of Patched Conics . . . 25
  - 3.4.1 Planetary Departure . . . 26
  - 3.4.2 The B-Plane . . . 27
- 3.5 Interplanetary Environment . . . 28
  - 3.5.1 Gravity Field . . . 28
  - 3.5.2 Solar Radiation Pressure . . . 31
- 3.6 Equations of Motion . . . 32

4 Parallel Computing 35
- 4.1 History and Motivation . . . 35
- 4.2 Forms of Parallelism . . . 35
- 4.3 Flynn's Taxonomy . . . 36
- 4.4 Memory Organisation Models . . . 36
  - 4.4.1 Shared-Memory Systems . . . 36
  - 4.4.2 Distributed-Memory Systems . . . 37
  - 4.4.3 Hybrid-Memory Systems . . . 37

4.5 Volatile Memory Technologies . . . 37
4.5.1 Static Random-Access Memory . . . 37
4.5.2 Dynamic Random-Access Memory . . . 37
4.6 Performance Metrics . . . 38
4.7 Multi-core CPU Architecture . . . 38
4.8 Graphics Processing Unit . . . 39
4.8.1 Advantages of GPU Computing . . . 39
4.9 CUDA Programming Model . . . 39
4.9.1 Heterogeneous Systems . . . 39
4.9.2 GPU Hardware Model . . . 40
4.9.2.1 Compute Capability . . . 40
4.9.2.2 Thread Blocks and Grids . . . 41
4.9.2.3 Thread-Block Clusters . . . 42
4.9.2.4 Warps and SIMT Execution . . . 42
4.9.2.5 Tile Programming . . . 44
4.9.3 Asynchronous Execution . . . 45
4.9.3.1 CUDA Streams . . . 46
4.9.3.2 CUDA Events . . . 46
4.9.4 Latency Hiding . . . 46
4.9.5 Dynamic Parallelism . . . 47
4.9.6 GPU Memory . . . 48
4.9.6.1 DRAM in Heterogeneous Systems . . . 48
4.9.6.2 On-Chip Memory in GPUs . . . 48
4.9.6.3 Unified Memory . . . 49

5 Robust Trajectory Optimisation 51
5.1 The Optimal Control Problem . . . 51
5.1.1 Deterministic Representation . . . 51
5.1.1.1 System Constraints . . . 51
5.1.1.2 Objective Function . . . 52
5.1.2 Stochastic Representation . . . 52
5.1.2.1 Probabilistic Constraints . . . 52
5.1.2.2 Probabilistic Objective Function . . . 53
5.1.2.3 Expected-Value Definition . . . 53
5.1.2.4 Quantile-Based Definition . . . 54
5.2 Uncertainty Propagation . . . 54
5.2.1 Summary of Methods . . . 55
5.2.2 Uncertainty Propagation in Spaceflight Applications . . . 58
5.2.3 Selection of Methods . . . 60
5.3 Transcription Methods . . . 61
5.3.1 Norm Chance Constraints . . . 61
5.3.2 Scalar Linear Chance Constraints . . . 61
5.3.3 Multidimensional Linear Chance Constraints . . . 62
5.3.4 Risk Estimation Methods . . . 63
5.3.5 Measure of Conservatism . . . 65
5.3.6 Failure Risk Allocation Methods . . . 66
5.4 Deterministic Trajectory Optimisation Solvers . . . 66
5.4.1 Indirect, Direct, and Dynamic Programming Approaches . . . 66
5.4.2 Gradient-Based, Heuristic, and Hybrid Solution Methods . . . 67
5.4.3 Solver Selection . . . 68
5.5 Problem Formulation . . . 70
5.5.1 Multiple-Leg Trajectories . . . 73
5.5.2 Unscented Transform . . . 74
5.5.3 Gaussian Mixture Model . . . 75

References 77

Appendix **89**
A1 Research Plan 89
A1.1 Work Breakdown Structure 89
A1.2 Reflection 93
A2 Dynamical disturbance model 95

NOMENCLATURE

ACRONYMS

Acronym	Definition
API	Application Programming Interface
BC	Behavioural Cloning
CADET	Covariance Analysis Describing Function Technique
CDP	CUDA Dynamic Parallelism
CI	Chebyshev Interpolation
CL	Closed-Loop
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DA	Differential Algebra
DDP	Differential Dynamic Programming
DOF	Degrees of Freedom
DQMOM	Direct Quadrature Method of Moments
DRAM	Dynamic Random-Access Memory
DSP	Digital Signal Processor
ECI	Earth-Centred Inertial
EHE	Ellipsoidal Harmonic Expansion
EME2000	Earth Mean Equator and Mean Equinox of J2000
FPE	Fokker–Planck Equation
FPGA	Field-Programmable Gate Array
GME	Gaussian Mixture Element
GMM	Gaussian Mixture Model
GNC	Guidance, Navigation, and Control
GPC	Graphics Processing Cluster
GPGPU	General-Purpose Graphics Processing Unit
GPS	Global Positioning System
GPU	Graphics Processing Unit
GRACE	Gravity Recovery and Climate Experiment
HET	Hall-Effect Thruster
HJB	Hamilton–Jacobi–Bellman
IA	Interval Analysis
IAU	International Astronomical Union
ICRF	International Celestial Reference Frame
ICRS	International Celestial Reference System
IERS	International Earth Rotation and Reference Systems Service
ITRF	International Terrestrial Reference Frame
KKT	Karush–Kuhn–Tucker
LAM	Likelihood Agreement Measure
LinCov	Linear Covariance (analysis)
LOADS	Low-Order Automatic Domain Splitting

Acronym	Definition
MC	Monte Carlo
MEE	Modified Equinoctial Elements
MGA	Mars Gravity Assist
MIMD	Multiple Instruction, Multiple Data
MISD	Multiple Instruction, Single Data
MPBVP	Multipoint Boundary-Value Problem
MPI	Message Passing Interface
MPPI	Model Predictive Path Integral
MTE	Missed-Thrust Event
NAIF	Navigation and Ancillary Information Facility
NLP	Nonlinear Programming
OCP	Optimal Control Problem
OL	Open-Loop
OpenCL	Open Computing Language
PCE	Polynomial Chaos Expansion
PDF	Probability Density Function
PMP	Pontryagin's Minimum Principle
PSD	Positive Semi-Definite
RAAN	Right Ascension of the Ascending Node
RAM	Random-Access Memory
RL	Reinforcement Learning
ROCP	Robust Optimal Control Problem
RSQ	Research Sub-Question
SDDP	Stochastic Differential Dynamic Programming
SDE	Stochastic Differential Equation
SEP	Solar Electric Propulsion
SHE	Spherical Harmonic Expansion
SIMD	Single Instruction, Multiple Data
SIMT	Single Instruction, Multiple Threads
SISD	Single Instruction, Single Data
SM	Streaming Multiprocessor
SoC	System-on-Chip
SOCP	Stochastic Optimal Control Problem
SP	Sigma Point
SPK	Spacecraft and Planet Kernel
SRAM	Static Random-Access Memory
SRP	Solar Radiation Pressure
SSB	Solar System Barycentre
STT	State Transition Tensor
TAG	Touch-And-Go
TBM	Trajectory Bundle Method
TDB	Barycentric Dynamical Time
TIP	Targeting Interface Point
ToF	Time of Flight
TSI	Taylor Series Integration
UT	Unscented Transform
VLBI	Very Long Baseline Interferometry

Acronym	Definition
WBS	Work Breakdown Structure
WP	Work Package
ZOH	Zero-Order Hold

LATIN SYMBOLS

Symbol	Definition	Unit
a	Semi-major axis	$[\mathrm{m}]$
$\boldsymbol{a}$	Acceleration vector	$[\mathrm{m\,s^{-2}}]$
A	Spacecraft cross-sectional area	$[\mathrm{m^2}]$
$\boldsymbol{b}$	B-plane miss parameter	$[\mathrm{m}]$
c	Speed of light in vacuum	$[\mathrm{m\,s^{-1}}]$
c_j	Weight of the j-th sigma point	[-]
$\mathcal{C}$	Tractable sufficient condition of a transcription method	[-]
C_3	Characteristic energy	$[\mathrm{km^2\,s^{-2}}]$
C_{nm}, S_{nm}	Spherical-harmonic coefficients of degree n and order m	[-]
C_r	Radiation-pressure coefficient	[-]
$\boldsymbol{e}$	Eccentricity vector	[-]
$\boldsymbol{f}$	Drift (deterministic dynamics) function	[various]
$\mathcal{F}$	Uncertainty-propagation operator	[-]
g_0	Standard gravitational acceleration at the Earth's surface	$[\mathrm{m\,s^{-2}}]$
$\boldsymbol{g}$	Diffusion matrix	[various]
G	Universal gravitational constant	$[\mathrm{m^3\,kg^{-1}\,s^{-2}}]$
$\boldsymbol{h}$	Specific angular momentum	$[\mathrm{m^2\,s^{-1}}]$
$\mathbf{H}$	Measurement matrix	[-]
i	Inclination	$[\mathrm{rad}]$
$\mathbb{I}_n$	Identity matrix of size n	[-]
I_{sp}	Specific impulse	$[\mathrm{s}]$
J	Cost functional	[various]
$\boldsymbol{K}_k$	Feedback control gain matrix of the k-th arc	[various]
L	Running cost (Lagrange term) / Number of trajectory legs	[various]
$\mathcal{L}$	Likelihood agreement measure	[-]
m	Spacecraft mass	$[\mathrm{kg}]$
M	Mass of the primary body / Number of GMM mixands	$[\mathrm{kg}]$ / [-]
n	Euclidean dimension	[-]
N	Number of trajectory arcs / number of samples	[-]
p	Probability level	[-]
P_{nm}	Associated Legendre functions of degree n and order m	[-]
$\boldsymbol{Q}_k$	Accumulated process-noise covariance matrix of the k-th arc	[various]
$Q_\Phi(p)$	p-th percentile of the random variable Φ	[various]
$\boldsymbol{r}$	Position vector	$[\mathrm{m}]$
r_{SOI}	Radius of the sphere of influence	$[\mathrm{m}]$
R_e	Reference radius of the celestial body	$[\mathrm{m}]$
$\tilde{\mathbf{R}}$	Covariance matrix of the measurement (estimation) error	[various]
$\boldsymbol{S}_k$	Feedforward (open-loop) control of the k-th arc	$[\mathrm{N}]$

Symbol	Definition	Unit
$\boldsymbol{S}_j$	Covariance matrix of the j-th GMM mixand	[various]
t	Time	$[\mathrm{s}]$
$\boldsymbol{T}$	Thrust vector	$[\mathrm{N}]$
$\mathbf{u}$	Control vector	[various]
$\boldsymbol{v}$	Velocity vector	$[\mathrm{m\,s^{-1}}]$
$\boldsymbol{v}_\infty$	Hyperbolic excess velocity	$[\mathrm{m\,s^{-1}}]$
$\boldsymbol{w}$	Wiener-process (process-noise) vector	[various]
W	Prime-meridian angle	$[\mathrm{rad}]$
$\boldsymbol{x}$	Spacecraft state vector	[various]
$\mathcal{X}_t$	Path-constraint function	[various]
$\mathcal{X}_i$, $\mathcal{X}_f$	Admissible sets of initial and final states	[-]
$\boldsymbol{y}$	Measurement vector / generic constraint random vector	[various]
$\mathbf{0}_n$	Null vector of size n	[-]
$\mathbf{0}_{n\times m}$	Null matrix of size $n \times m$	[-]

GREEK SYMBOLS

Symbol	Definition	Unit
α	Right ascension / Violation parameter	$[\mathrm{rad}]$ / [-]
α_i	Weight of the i-th GMM mixand	[-]
α_0	Right ascension of the body's north pole	$[\mathrm{rad}]$
δ_0	Declination of the body's north pole	$[\mathrm{rad}]$
β	Risk leve l / Angle between the apsidal line and the asymptote of a hyperbolic trajectory	[-] / $[\mathrm{rad}]$
β_T	Transcribed (conservative) failure risk	[-]
β_R	Real failure risk	[-]
γ	Conservatism of a transcription method	[-]
δ	Declination	$[\mathrm{rad}]$
δ_{0m}	Kronecker delta	[-]
η	Threading efficiency	[-]
$\boldsymbol{\eta}_k$	Measurement noise vector at time t_k	[various]
θ	True anomaly	$[\mathrm{rad}]$
θ_B	B-plane clock angle	$[\mathrm{rad}]$
$\boldsymbol{\Theta}$	Vector of problem unknowns	[various]
κ	Unscented Transform scaling parameter	[-]
λ	Longitude	$[\mathrm{rad}]$
μ	Gravitational parameter	$[\mathrm{m^3\,s^{-2}}]$
$\boldsymbol{\mu}$	Mean state vector	[various]
ν	Shadow factor	[-]
$\boldsymbol{\rho}$	Position vector of the infinitesimal mass element	$[\mathrm{m}]$
$\boldsymbol{\sigma}_y$	Vector of the standard deviations of $\boldsymbol{y}$	[various]
$\boldsymbol{\Sigma}_y$	Covariance matrix of random variable $\mathbf{y}$	[various]
ϕ	Latitude	$[\mathrm{rad}]$
Φ	Terminal cost (Mayer term)	[various]
Φ_G	Cumulative distribution function of a standard Gaussian distribution	[-]

Symbol	Definition	Unit
$\boldsymbol{\Phi}(t, s)$	State-transition matrix from time s to time t	[-]
$\boldsymbol{\psi}$	Boundary-constraint function	[various]
ψ_0	Reference solar irradiance at $1\ \mathrm{au}$	$[\mathrm{W\,m^{-2}}]$
Ψ_n	Tail probability function associated with a standard χ^2 distribution with n degrees of freedom	[-]
ω	Argument of periapsis	$[\mathrm{rad}]$
Ω	Right ascension of the ascending node	$[\mathrm{rad}]$

NOTATION

Notation	Definition
$\mathcal{N}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_y)$	Gaussian distribution with mean $\bar{\boldsymbol{y}}$ and covariance $\boldsymbol{\Sigma}_y$.
$\|\cdot\|$	Euclidean norm.
$\boldsymbol{x} \cdot \boldsymbol{y}$	Vector dot-product
$\boldsymbol{x} \times \boldsymbol{y}$	Vector cross-product
$\mathrm{diag}(d_1, d_2, \ldots, d_n)$	Diagonal matrix of size n with its i–th diagonal term equal to d_i.
A^*	Non-zero elements of the set A, i.e., $A \setminus \{0\}$.
$(\cdot)^T$	Transposition operator.
$(\cdot)_j$	j–th column vector of the underlying matrix.
$p_G(\boldsymbol{y}; \bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_{\boldsymbol{y}})$	Probability density function (PDF) of a multivariate Gaussian distribution of mean $\bar{\boldsymbol{y}}$ and covariance $\boldsymbol{\Sigma}_y$ evaluated at $\boldsymbol{y}$.
$\mathbb{N}_n$	Set $\{0, 1, 2, \ldots, n\}$
$\mathbb{E}[\cdot]$	Expectation operator
$\mathbb{P}\{\cdot\}$	Probability operator
$x \sim \mathcal{D}$	Random variable x is distributed according to the distribution $\mathcal{D}$
$\bar{x}$	Mean (expected) value of x
$\dot{x}$	First time derivative
$\ddot{x}$	Second time derivative
x_k	Time-dependent function value evaluated at the discrete time instant t_k
x^{conv}	Non-dimensionalisation conversion factor for quantity x
$x\|_c$	Property x is limited to c
$x_{A/B}$	x expresses a property of A with respect to B
x_a	x expresses a property at apoapsis
x_p	x expresses a property at periapsis
$\boldsymbol{x}$	Vector
$\boldsymbol{x}_i$	Vector's i-th component
$\hat{\boldsymbol{x}}$	Unit vector
$\boldsymbol{x}^A$	Vector $\boldsymbol{x}$ is expressed in A-frame components
$\boldsymbol{x} \preceq \boldsymbol{y}$	Componentwise inequality between vectors
$\mathcal{F}_A$	Reference frame A
$\mathbf{X}$	Matrix
$\mathbf{X_{ij}}$	Term at the i–th row and j–th column of a matrix
$\boldsymbol{C}_{B/A}$	Rotation matrix from frame A to frame B
$\mathbf{X}^{-1}$	Matrix $\mathbf{X}$ is inverted
$\mathbf{X} \succeq \mathbf{0}$	Matrix $\mathbf{X}$ is positive semi-definite

Notation	Definition
$\rho(\mathbf{X})$	Positive square root of the spectral radius of matrix $\mathbf{X}$
$\mathcal{O}(\cdot)$	Asymptotic computational complexity
Δx	Finite change or difference in the quantity x

1

INTRODUCTION

The computational demands of astrodynamics predate the digital computer by several centuries. In 1705, Edmond Halley predicted that the comet observed in 1531, 1607, and 1682 would return around the year 1758. Refining this prediction, however, required accounting for the gravitational perturbations induced by Jupiter and Saturn. In 1757, Alexis Clairaut, Jérôme Lalande, and Nicole-Reine Lepaute undertook the lengthy calculations needed to evaluate these effects. The workload was divided among them, allowing a problem of otherwise prohibitive scale to be completed before the comet reappeared. Their calculations predicted a perihelion passage in April 1759, with an uncertainty of approximately one month. The comet reached perihelion on 13 March 1759 (Broughton, 1985). Reflecting on this effort, Lalande (1759) wrote:

It must be acknowledged that this immense sequence of detailed calculations would have appeared daunting to me, had Madame Lepaute, who had long applied herself successfully to astronomical calculations, not shared the work.

The episode illustrates a principle that remains central to scientific computing: when a computational problem becomes too large to be treated sequentially, progress often depends on decomposing it into smaller tasks that can be evaluated concurrently. In the eighteenth century, this decomposition was performed manually by human calculators. Today, graphics processing units (GPUs) provide a hardware architecture capable of executing large numbers of computational tasks in parallel.

Robust trajectory optimisation is a particularly relevant application of this paradigm. When uncertainty is incorporated directly into the mission-design process, each optimisation iteration may require repeated propagation of the spacecraft dynamics and of the associated uncertainty representation, together with the evaluation of objective functions, constraints, and, where required, their sensitivities. The resulting computational burden grows with the fidelity of the dynamical model, the temporal resolution of the transcription, and the complexity of the uncertainty representation. The resulting structure, however, also exposes several forms of parallelism that may be exploited using GPUs.

1.1. UNCERTAINTIES IN MISSION DESIGN

Spacecraft trajectories are inevitably affected by uncertainty arising from navigation errors, random disturbances, partially known dynamical environments, inaccurate models, and imperfect control execution, including full missed-thrust events (MTEs) (Rubinsztejn et al., 2020). A control law optimised for a single deterministic reference trajectory may therefore become inadequate during operations, particularly for long-duration missions and nonlinear dynamical systems, in which initially small deviations can grow substantially over time. Low-thrust missions are especially sensitive because their limited control authority often requires extended intervals of near-maximum thrust, leaving reduced capacity for corrective manoeuvres. The operational relevance of this issue is illustrated by MTEs experienced during missions such as Dawn and Hayabusa.

The need for robustness is further reinforced by the increasing use of lightweight spacecraft, cost-constrained mission architectures, and extended autonomous operations under long communication delays. During an interplanetary transfer, uncertainty must be managed not only to preserve performance but also to reduce the probability of hazardous events, such as collision with a planetary body during a close flyby, with implications for both spacecraft safety and planetary protection. In current practice, these risks are often

mitigated heuristically by allocating additional propellant for correction manoeuvres, enlarging deterministic safety margins such as planetary flyby impact radii, and enforcing coasting arcs along sensitive portions of the reference trajectory (Oh et al., 2008; Rayman et al., 2007; Rayman and Williams, 2002).

Within the traditional mission design process, the nominal trajectory is first optimised without explicitly accounting for the statistical properties of the uncertain parameters. Conservative margins are subsequently introduced, while the guidance, navigation, and control (GNC) system is developed to reject disturbances and correct deviations during flight (Ridolfi et al., 2012). High-fidelity simulations and statistical analyses, commonly based on iterative Monte Carlo campaigns with the GNC system in the loop, are then used to update subsystem requirements and trajectory-design margins. Although this progressive refinement is well established, it is computationally expensive and may fail to identify rare but critical scenarios leading to mission failure.

This separation between trajectory optimisation and uncertainty analysis also places a substantial burden on the GNC subsystem. If the resulting stability or control requirements cannot be satisfied, trajectory margins and subsystem requirements may need to be revised, increasing development time, testing effort, system complexity, and cost. A more systematic alternative is to treat the reference trajectory itself as a pre-emptive robustness mechanism by incorporating uncertainty information directly into the optimisation process. This approach can reduce the corrective effort required during operations and partially transfer the management of uncertainty from the GNC system to the mission design phase. Its practical adoption, however, depends on reducing the computational cost associated with robust trajectory optimisation.

1.2. PARALLEL COMPUTING FOR ROBUST TRAJECTORY OPTIMISATION

The potential to use trajectory design as a pre-emptive robustness mechanism has motivated growing interest in optimisation methods that explicitly account for model errors and uncertainties (Chai et al., 2019). Because direct Monte Carlo propagation may require a large number of trajectory simulations, alternative uncertainty-propagation methods have been developed to approximate the evolution of probability distributions at lower computational cost while retaining sufficient accuracy (Luo and Yang, 2017). These methods have enabled the integration of uncertainty analysis into trajectory optimisation. Nevertheless, an important trade-off remains. Computationally efficient formulations, such as those based on linearisation techniques, are generally most effective when uncertainties remain comparatively small. By contrast, methods capable of accurately representing larger uncertainty sets and strongly nonlinear dynamics typically incur substantially higher computational costs.

Parallel computing offers a means of mitigating this trade-off. Many operations arising in stochastic trajectory planning can be decomposed into independent computational tasks and evaluated concurrently on multicore central processing units (CPUs) or general-purpose graphics processing units (GPGPUs), which contain large numbers of lightweight computing cores and are designed to execute many threads in parallel. The term GPGPU was introduced when GPUs began to be used for applications beyond graphics rendering. Since modern GPUs are routinely employed for general-purpose computation, the terms GPU and GPGPU are often used interchangeably in this context. Their use in scientific computing has become increasingly accessible through programming interfaces such as the Compute Unified Device Architecture (CUDA) and the Open Computing Language (OpenCL), which expose the underlying hardware through general-purpose parallel-programming abstractions.

GPU-assisted computation has already been investigated for uncertainty propagation (Massari et al., 2017) and for deterministic trajectory optimisation and mission-analysis problems (Zou and Jiang, 2025; Izzo, 2012; Wittig et al., 2016; Kaleem and Leitner, 2011; Wagner et al., 2012). GPU acceleration has also been applied to robust trajectory optimisation in other application domains, including robotics (Wang et al., 2026). However, within the literature surveyed for this thesis, the use of GPU computing for robust trajectory optimisation for low-thrust space-mission design remains comparatively underexplored. This gap motivates the development of formulations whose computational structure is explicitly designed to benefit from GPU acceleration.

1.3. PROBLEM STATEMENT

The previous sections highlight the growing interest in incorporating uncertainty analysis directly into the trajectory design process. This enables the optimisation of reference trajectories that are not only deterministically optimal but also resilient to navigation errors, random disturbances, modelling inaccuracies, control-execution errors, MTEs and dynamical perturbations. However, stochastic trajectory optimisation introduces a substantial computational burden. These costs become particularly significant for extended low-thrust trans-

fers characterised by large uncertainties and nonlinear dynamical environments.

At the same time, many of the most computationally expensive operations involved in robust trajectory optimisation can be decomposed into a large number of independent operations. This structure makes the problem well suited to GPU computing. Based on these considerations, the objective of this thesis is to *develop and evaluate a robust low-thrust trajectory optimisation methodology that explicitly accounts for uncertainties and perturbations while being structured to expose and exploit parallelism*. The main research question that will drive this thesis is stated hereafter:

> How can GPU computing be leveraged to accelerate low-thrust trajectory optimisation in the presence of uncertainty and perturbations while maintaining numerical accuracy and solution optimality?

Three research sub-questions (RSQs), which clarify the scope of the project and help address the main research question, have also been identified:

- **RSQ-1:** How can stochastic trajectory optimisation methods be reformulated to expose and exploit parallelism?
- **RSQ-2:** What computational speedup can be achieved relative to multicore CPU implementations?
- **RSQ-3:** How does GPU acceleration affect optimisation accuracy, solution robustness, and computational scalability relative to multicore CPU implementations?

1.4. REPORT OUTLINE

The remainder of this report is organised as follows...

2

MISSION HERITAGE

This chapter establishes the technical and mission context of the thesis. Section 2.1 identifies the main sources of uncertainty affecting interplanetary spacecraft trajectories. Section 2.2 then introduces the general structure of stochastic optimisation for mission analysis, emphasising the roles of uncertainty propagation, transcription, and deterministic optimisation solvers.

The state of the art is subsequently reviewed in Section 2.3, with particular attention to robust trajectory planning and GPU acceleration in mission analysis, together with relevant research heritage from Delft University of Technology.

Section 2.4 then places the selected reference scenario within the broader context of low-thrust interplanetary missions by summarising representative successful deep-space missions that employed solar electric propulsion or solar sailing. Finally, in Section 2.5, NASA's Psyche mission is introduced as the reference case for this study, and the target asteroid, spacecraft, and uncertainty models are described. The chapter concludes with Section 2.5.3 by defining the mission requirements, system requirements, and boundary conditions that will guide the development of this research.

2.1. SOURCES OF UNCERTAINTY

Uncertainties affecting interplanetary spacecraft trajectories can be grouped into four main categories:

- **Modelling errors**: these arise from imperfect knowledge of the system dynamics. A representative example is the uncertainty in the solar radiation pressure coefficient, which can affect the accuracy of the predicted acceleration.
- **Navigation errors**: these describe the discrepancy between the true spacecraft state and the state estimated either on the ground, through orbit determination, or onboard, through sequential filtering.
- **Thrusting errors**: while these may be interpreted as a subset of modelling errors, they are typically treated separately given their direct influence on trajectory evolution. They generally arise from discrepancies accumulated between command computation and execution, or from inaccurate thruster performance predictions. The most critical manifestation is a complete missed-thrust event (MTE). Furthermore, attitude errors can introduce a misalignment between the commanded and realised thrust direction.
- **Random disturbances**: these include stochastic perturbations that cannot be deterministically predicted during the mission. In asteroid missions, such disturbances may arise from outgassing, dust-particle impacts near active small bodies, or other unmodelled accelerations.
- **Launch injection errors**: these arise from dispersions in the state delivered by the launch vehicle relative to the nominal injection conditions.

2.2. STOCHASTIC SOLVERS FOR MISSION ANALYSIS

Stochastic solvers are used in astrodynamics to compute trajectory profiles that remain robust under uncertainty. Most stochastic solvers can be decomposed into three main building blocks:

- **Uncertainty propagation:** Random state and control variables are propagated from time t_k to t_{k+1}, yielding their associated probability distributions. The corresponding stochastic constraints and cost functions are then evaluated.
- **Transcription method:** The stochastic constraints $\boldsymbol{c}$ and cost function J are reformulated as tractable deterministic counterparts, denoted by $\widetilde{\boldsymbol{c}}$ and $\widetilde{J}$. This step is often less emphasised in the stochastic optimisation literature, despite being critical for computational tractability and constraint satisfaction. In practice, transcription methods may introduce significant conservatism, leading to suboptimal solutions with excessively large safety margins.
- **Deterministic optimisation solver:** After uncertainty propagation and transcription, the original stochastic optimisation problem is converted into a deterministic optimisation problem that can be solved with standard numerical methods. Many approaches rely on off-the-shelf solvers, such as the MATLAB function *fmincon*[1], used by Greco et al. (2022); the ESA WORHP solver[2] (Büskens and Wassel, 2012), used in (Marmo et al., 2023); or the convex solver MOSEK[3] (Andersen, 2000), used in (Ridderhof et al., 2020; Benedikter et al., 2022). These solvers reduce development effort because they are well documented, implemented, and supported by existing user communities. They are also general enough to address a broad class of optimisation problems. However, this generality may come at the cost of computational efficiency, especially for repeated solves or onboard trajectory design. For this reason, *ad hoc* solvers can be advantageous when common operations across the three blocks can be exploited to reduce runtime. Examples include the work of Ozaki et al. (2018), where a stochastic version of the differential dynamic programming (DDP) solver is implemented, and the solver of Oguri and McMahon (2022), which relies on primer vector theory (Conway et al., 2010).

The workflow is summarised in Figure 2.1. Here, $\boldsymbol{X}$ and $\boldsymbol{U}$ denote the complete sequences of discrete state and control vectors over the optimisation horizon, respectively, while the superscript $(\cdot)^*$ identifies the converged solution returned by the deterministic optimiser. Beyond the limitations of each individual block, stochastic solvers may be computationally intensive (Greco et al., 2022), difficult to deploy in embedded applications, overly conservative (Ridderhof et al., 2020), or dependent on strong assumptions, such as linear dynamics and/or Gaussian uncertainty over the full time of flight (ToF) (Benedikter et al., 2022; Ozaki et al., 2018; Marmo et al., 2023).

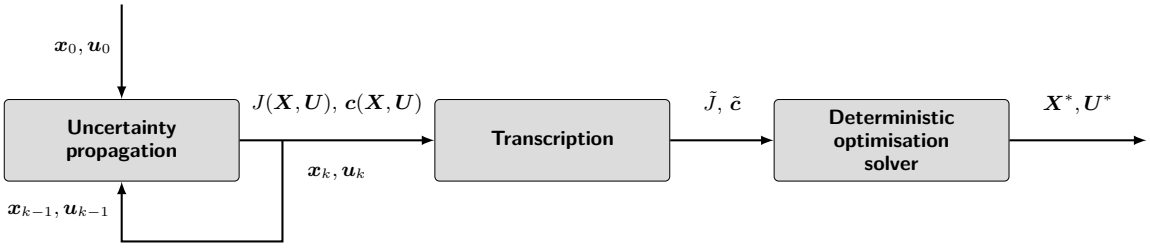


Figure 2.1: Workflow of a stochastic optimisation solver.

2.3. STATE OF THE ART

This section reviews the research status of the areas most relevant to the methodology developed in this thesis. First, robust trajectory optimisation methods are discussed, with emphasis on how uncertainty is represented, propagated, and incorporated into the optimal control problem. Particular attention is given to the distinction between open-loop and closed-loop control-policy formulations, as this distinction affects the ability of a trajectory to remain feasible in the presence of uncertainties and errors. The section then identifies and describes the framework selected as the methodological basis of this work. Finally, previous applications of GPU acceleration in mission analysis are reviewed, together with related research heritage from Delft University of Technology.

[1] See `https://www.mathworks.com/help/optim/ug/fmincon.html`. Accessed: 7 June 2026.
[2] See `https://worhp.de`. Accessed: 7 June 2026.
[3] See `https://www.mosek.com`. Accessed: 7 June 2026.

2.3.1. ROBUST TRAJECTORY PLANNING

Robust trajectory planning aims to account for uncertainty directly within the trajectory-design process, rather than addressing deviations from the nominal solution only after optimisation. As introduced in Section 2.2, existing methods can be classified according to three main aspects: the approach used to formulate probabilistic cost functions and constraints in a tractable form; the numerical strategy employed to solve the resulting optimal control problem; and the uncertainty-propagation method adopted during optimisation. A further important distinction lies in the formulation of the control policy as open-loop or closed-loop.

2.3.1.1. CONTROL-POLICY FORMULATIONS

The treatment of uncertainty and error sources in trajectory optimisation depends not only on how they are represented and propagated, but also on how the control policy is formulated. In particular, the distinction between open-loop and closed-loop control determines whether the optimised trajectory can adapt to deviations from the nominal state during flight. This choice directly influences robustness, control-margin allocation, and the ability of the spacecraft to satisfy mission constraints.

In open-loop formulations, the optimised control history is prescribed a priori and does not explicitly depend on future navigation estimates. In closed-loop or model-predictive-control formulations, by contrast, state deviations, navigation updates, and the associated state-estimation errors are explicitly accounted for during the design process (Gustafson, 2010; Richards and How, 2006). Closed-loop methods can be further divided into two main categories. In the first, the feedforward control profile and feedback law are optimised simultaneously. In the second, the nominal control profile is optimised while the feedback structure, correction policy, or available corrective-control authority is prescribed in advance.

This distinction is particularly important for low-thrust rendezvous trajectories. During the final approach, the minimum-fuel solution may require a thrust magnitude equal or close to the maximum available thrust. The nominal thrust profile must therefore retain sufficient control authority for corrective manoeuvres (Ridderhof and Tsiotras, 2019; Rayman et al., 2007; Laipert and Longuski, 2015). More generally, minimum-fuel guidance may require the nominal trajectory and feedback strategy to be designed jointly so that corrective-control margins are allocated efficiently throughout the transfer using probabilistic, or chance, constraints. When uncertainties are represented by random variables, these margins directly affect the probability that the required corrective control exceeds the available actuation capability, resulting in control saturation.

Current research in robust space-trajectory design increasingly seeks to incorporate expected uncertainties and errors directly into the optimisation process. The objective is to design nominal control profiles and feedback laws that are tailored to the expected mission environment while reducing the propellant margin required to accommodate stochastic corrections.

2.3.1.2. RESEARCH DIRECTIONS

Research on robust trajectory optimisation has developed along several complementary directions, the principal approaches to which are reviewed in this section.

Greco et al. (2022) proposed a belief-based approach in which the optimal control problem is formulated in terms of uncertainty distributions, or beliefs, rather than specific realisations of the system state. Although the exact state realisation is unknown, the belief state is assumed to be fully observable because the corresponding probability distribution is known at each time. Related work addressed optimal control under uncertainty using polynomial-algebra-based multiple shooting (Greco et al., 2020), while subsequent research incorporated nonlinear navigation analysis under both aleatoric and epistemic uncertainty (Greco and Vasile, 2020).

Robust low-thrust trajectory optimisation under epistemic uncertainty was also investigated by da Graça Marto and Vasile (2021, 2022). Epistemic uncertainty arises from incomplete knowledge of system and launch-vehicle parameters rather than from a fully characterised stochastic process. The authors represented uncertain parameters using probability boxes and assessed trajectory robustness through the lower expectation, corresponding to the worst-case expected satisfaction of a prescribed performance threshold over the admissible family of probability distributions.

This approach was applied to both a multi-asteroid fly-by tour and an asteroid-rendezvous problem, considering uncertainties in the launch hyperbolic excess velocity, thrust magnitude, and specific impulse. The lower expectation requires the solution of an embedded global optimisation problem, the authors therefore combined Bernstein-polynomial representations, quasi-Monte Carlo sampling, surrogate models, dimensionality-reduction mappings, and the Multi-Agent Collaborative Search algorithm. The resulting control and threshold mappings improved the quality of the approximated Pareto front. However, the asteroid-tour optimisation still required approximately eight hours on the tested CPU platform, highlighting the substantial computational

cost of the framework. The authors also noted that the lower-expectation estimates were approximate, with an average error below 10%, rather than guaranteed outer bounds, and that the resulting trajectories were optimal only within the reduced control space.

Covariance information has also been incorporated directly into the objective function. Zimmer et al. (2010) introduced an indirect optimal control approach in which navigation performance is represented through terms depending on the spacecraft state-estimation covariance. Their results showed that the orbit covariance could be significantly reduced during continuous-thrust transfers with only a modest increase in fuel consumption. Subsequently, Oguri and McMahon (2022) extended Lawden's primer-vector theory to robust low-thrust trajectory design. This formulation provides an analytical open-loop optimal control law that satisfies probabilistic path constraints under state uncertainty. Despite the fast convergence and reduced computational cost often associated with indirect formulations, their practical application to advanced trajectory-optimisation problems may be limited by the need to derive analytical expressions, such as state and costate derivatives. This requirement increases formulation complexity and reduces versatility.

Xiong et al. (2015) introduced a method for transforming a stochastic trajectory-optimisation problem into a deterministic framework by combining Polynomial Chaos Expansion with an adaptive pseudospectral collocation scheme. Polynomial Chaos Expansion was also applied to robust mission analysis by Giordano and Topputo (2023), who proposed an integrated preliminary mission-analysis framework in which navigation assessment and stochastic costs are embedded directly within the trajectory-design process.

Another class of methods is Stochastic Differential Dynamic Programming (SDDP) (Ozaki et al., 2018, 2020). SDDP inherits many of the advantages and limitations of deterministic Differential Dynamic Programming (DDP). DDP was originally introduced to mitigate the exponential growth in state-space dimensionality associated with Dynamic Programming by solving Bellman's optimal control problem locally around a reference trajectory. Consequently, both DDP and SDDP are generally less suitable for problems involving large uncertainties or substantial deviations from the nominal trajectory.

Machine-learning approaches based on Reinforcement Learning (RL) have also been proposed for robust optimal trajectory design (Federici et al., 2021; Zavoli and Federici, 2021). In RL-based formulations, a deep neural network maps spacecraft states to an optimal control policy and an associated value function, which quantifies trajectory performance with respect to the mission objectives and requirements. The network is trained through repeated interactions with multiple realisations of the environment, progressively improving the control policy to maximise the expected cumulative reward. Once trained, the method can provide both a robust reference trajectory and an optimal state-feedback control law. The trained network may therefore be deployed onboard the spacecraft to support autonomous guidance and control during mission operations. However, RL generally requires longer training times than the other methods considered, and its ability to accommodate multiple terminal constraints remains an open research question. RL has also demonstrated potential for robust deep-space transfers subject to different sources of error (LaFarge et al., 2021).

Behavioural Cloning (BC) has also been investigated as a means of improving autonomous guidance, navigation, and control capabilities (Izzo et al., 2019; Sanchez-Sanchez et al., 2016; Cheng et al., 2020). In these methods, a set of previously optimised trajectories, referred to as expert demonstrations, is generated by perturbing the initial or terminal conditions of a deterministic problem. These demonstrations are then used to improve training quality or reduce the computational cost of the learning process. Nevertheless, the effectiveness of BC depends strongly on the size and representativeness of the demonstration set, which must adequately cover the relevant range of possible trajectory deviations. Generating such a comprehensive dataset can therefore present a significant computational challenge.

In parallel, synergistic approaches combining convex optimisation and covariance steering have been introduced for several classes of robust continuous-control problems in astrodynamics, including interplanetary transfers (Benedikter et al., 2022).

Covariance steering represents an important line of research in which the nominal control primarily governs the mean trajectory, while the feedback gain shapes the evolution of the state covariance. It therefore provides a natural framework for designing trajectories that satisfy both performance and uncertainty requirements.

2.3.1.3. SELECTED METHODOLOGY

A particularly relevant application of covariance steering is the chance-constrained robust-optimisation framework developed across the series of studies by Marmo and Zavoli (2022); Marmo et al. (2023); Marmo and Zavoli (2024); Marmo et al. (2025). The framework jointly optimises the nominal trajectory and a closed-loop control policy while accounting for the uncertainty affecting the spacecraft state. Its objective function

minimises the cumulative closed-loop control effort, including both deterministic and stochastic control contributions, up to a prescribed probability level. Navigation errors are incorporated by applying feedback to a filtered estimate of the spacecraft state, thereby accounting for the uncertainty introduced by the navigation system. In particular, the formulation presented by Marmo and Zavoli (2024) was tailored to continuous low-thrust propulsion, providing a more accurate estimate of the state uncertainty induced by propulsion inaccuracies.

The numerical solution is based on a hybrid multiple-shooting approach. A single-shooting scheme is used to propagate the spacecraft state distribution, while the mean trajectory and deterministic open-loop controls are optimised through multiple shooting. This strategy provides a larger convergence radius than a fully single-shooting formulation, while avoiding numerical discontinuities in the covariance matrix at the boundaries between shooting segments.

The inclusion of a feedback component allows the method to overcome the intrinsic limitations of robust open-loop approaches, which can tolerate only limited deviations from the nominal trajectory. At the same time, the adoption of a simple linear feedback term based on the filtered spacecraft state preserves computational efficiency. Linear feedback laws have also been employed for the simultaneous optimisation of control gains and nominal trajectories in several related studies (Ridderhof et al., 2020; Ridderhof and Tsiotras, 2022; Ridderhof et al., 2022; Benedikter et al., 2021, 2022; Oguri and Lantoine, 2022; Akman et al., 2024).

A further advantage of this framework is its compatibility with nonlinear uncertainty propagation. The use of the Unscented Transform (UT) avoids the need to linearise the dynamics about a reference solution, as required by many convex-based approaches. This reduces implementation complexity and eliminates the need to derive explicit linearised dynamics, a process that can be both time-consuming and prone to error. Compared with linear covariance propagation based on first-order Taylor expansions, the UT generally provides more accurate estimates of the propagated mean and covariance in nonlinear dynamical systems, particularly when uncertainties are large.

Nevertheless, the UT remains a moment-matching technique and assumes that the state distribution can be adequately characterised by its first two moments. It may therefore fail to capture strongly non-Gaussian features, such as multimodality. The method is nonetheless selected in this work, and this limitation is addressed through Gaussian Mixture Model (GMM) decompositions. For each mixture component, the UT propagates a carefully selected set of sigma points through the nonlinear system, thereby improving the representation of the propagated uncertainty while retaining a tractable moment-based description.

This increased accuracy is particularly important when large uncertainties may develop, for instance during long coasting arcs or under off-nominal conditions such as prolonged missed-thrust events. The resulting approach therefore trades part of the computational efficiency of convex covariance-steering methods, which rely on linear expressions for covariance propagation, for a more accurate representation of nonlinear uncertainty evolution. In this context, GPU acceleration can mitigate the additional computational cost while preserving the advantages of nonlinear propagation.

In conclusion, as will be described in detail in Section 5.2, the selected methodology exposes two levels of parallelism. The first arises across mixands, since all GMM components can be propagated independently. The second arises within each mixand, since the time evolutions of the UT sigma points are mutually independent.

2.3.2. GPU ACCELERATION IN MISSION ANALYSIS

GPU acceleration has revolutionised a wide range of scientific and industrial domains. The ability to process vast amounts of data in parallel has enabled breakthroughs in space mission analysis, where parallel propagation of multiple initial states or orbit correction scenarios can drastically reduce computational time.

In many space applications, GPU-based implementations achieve speedups of one to two orders of magnitude compared with CPU-only solutions. The combination of high memory bandwidth, low-latency shared memory, and thousands of concurrent threads allows GPUs to outperform even high-end multicore CPUs for suitable workloads.

For example, regarding uncertainty-propagation applications, MC simulations using GPU-based parallel-computation techniques have been carried out to demonstrate how the multivariate uncertainty distribution loses its normality (Arora et al., 2015), for orbit estimation, collision-probability calculation, and visualisation (Liu et al., 2015), and for nonlinear uncertainty propagation based on differential-algebra (DA) techniques (Massari et al., 2017). GPUs have been used in other space-related applications, such as space-debris collision prediction (Lin et al., 2017), asteroid aggregation problems (Ferrari et al., 2020), aerodynamics simulations in low-Earth orbit (Omar, 2017), and parallel image processing of remote-sensing data (Liu et al., 2014). In

conclusion, parallel computing has become a cornerstone of modern scientific and engineering computation. The resulting computational capabilities have enabled significant advancements across numerous domains, from artificial intelligence to orbital mechanics.

2.3.3. DELFT UNIVERSITY OF TECHNOLOGY RESEARCH HERITAGE

Several research efforts related to GPU computing and robust trajectory optimisation have been conducted at TU Delft in recent years. This section summarises the most relevant heritage from these works. In the context of GPU computing, Geda (2019) investigated the efficient use of GPUs for massively parallel trajectory propagation and developed CUDAjectory, a CUDA-based propagation tool that remains in use and under development at ESA/ESOC.

The work applied optimisation techniques at both instruction and algorithmic levels to maximise computational performance. A sample-clustering strategy was introduced to partially mitigate memory-coalescence bottlenecks arising when trajectory samples have different initial epochs. In this context, sample clustering consists of grouping trajectory samples with similar dynamical behaviour, particularly similar initial epochs. Since GPU memory access is most efficient when neighbouring threads access neighbouring memory locations, grouping samples with similar epochs increases the likelihood that adjacent threads retrieve ephemeris data from nearby memory addresses, thereby improving memory-access efficiency. In addition, asynchronous output handling was implemented to reduce simulation runtime by overlapping GPU propagation with file-writing operations. This avoids frequent synchronisation between computation and disk output, which is comparatively slow. Ephemeris retrieval was also optimised by storing the required data in texture memory. Finally, the performance of two gravity models, namely spherical harmonics and point-mascon models, was compared, showing that the spherical-harmonics formulation is faster in all tested cases. Depending on the application, speedups of up to two orders of magnitude were achieved with respect to single-core CPU execution of the same software. The largest relative runtime reduction was observed when integration and output times were comparable, as in low-end gaming GPUs. For high-end GPUs, integration time is typically much smaller than output time, so asynchronous output handling has a more limited impact on the total runtime.

A related study by Nascimento (2025) investigated GPU-accelerated Model Predictive Path Integral control (MPPI) for high-speed navigation on uneven and cluttered lunar terrain. The authors developed an MPPI controller using the WARP framework (Macklin, 2024), which facilitates the exploitation of GPU parallelism. The controller was tested in simulation using a Husky rover on realistic lunar terrain generated from NASA data. Since many MPPI implementations assume flat ground when sampling trajectories, they may generate inaccurate, suboptimal, or unsafe paths on sloped terrain. To address this limitation, Nascimento (2025) proposed a lightweight mathematical method for three-dimensional trajectory projection and compared it with a standard two-dimensional MPPI formulation. The results showed that the 3D-enhanced MPPI planner outperforms the 2D approach in terms of traversal speed and obstacle avoidance on sloped, rock-scattered terrain. Conversely, the conventional 2D formulation remains robust on flat terrain. These findings suggest that an adaptive strategy, switching between 2D and 3D trajectory projection according to local terrain characteristics, could provide a useful trade-off between performance and power efficiency.

In the context of robust trajectory optimisation, Silva (2025) investigated the robustification of a deterministically optimal reference re-entry trajectory for the HORUS-2B vehicle. The work combined non-intrusive Polynomial Chaos Expansions (PCEs) with Gaussian Mixture Models (GMMs) to address the high dimensionality of the uncertainty space and the strongly nonlinear dependence of the trajectory constraints on the uncertain inputs. Owing to the curse of dimensionality and the nonlinear re-entry dynamics, the hybrid GMM–PCE uncertainty-propagation method does not, when considered independently, outperform Monte Carlo (MC) simulation. The main contribution of the thesis was therefore to investigate how the method could become computationally advantageous when embedded within a stochastic optimisation framework. In particular, the hybrid GMM–PCE formulation provides analytical expressions for trajectory statistics as functions of the optimisation decision variables. These expressions can be used to determine relationships between decision variables that satisfy the stochastic constraints, thereby reducing the dimensionality of the design space and avoiding repeated uncertainty propagation within parts of the robustification process.

2.4. LOW-THRUST INTERPLANETARY MISSIONS

Low-thrust propulsion has become an important enabling technology for interplanetary missions requiring large cumulative velocity changes and efficient propellant utilisation. Although the available thrust is typically several orders of magnitude lower than that of chemical propulsion systems, its continuous application over

Table 2.1: Summary of representative low-thrust deep-space missions. Failed missions are not included.

Mission	Launch	Launcher	Target	Propulsion type	Approach	Encounter epoch[a]
SMART-1[b]	2003	Ariane 5G	Moon	SEP	Orbiter/ Impactor	2005–2006
Deep Space 1[c]	1998	Delta II 7326-9.5	Braille	SEP	Flyby	1999
			Borrelly		Flyby	2001
Hayabusa[d]	2003	M-V	Itokawa	SEP	Rendezvous/ Sampler	2005–2010
Dawn[e]	2007	Delta II Heavy 7925H-9.5	Vesta	SEP	Orbiter	2011–2012
			Ceres		Orbiter	2015–2018
IKAROS[f]	2010	H-IIA F17	Venus	Solar sail	Flyby	2010
Hayabusa2[g]	2014	H-IIA	Ryugu	SEP	Rendezvous/ Sampler	2018–2020
			2001 CC21		Flyby	2026
			1998 KY26		Rendezvous	2031
BepiColombo[h]	2018	Ariane 5 ECA	Mercury	SEP	Orbiter	2026
Psyche[i]	2023	Falcon Heavy	16 Psyche	SEP	Orbiter	2029

[a] Encounter epochs refer to the main target encounter or operational period at the target. For ongoing missions, the listed year corresponds to the planned encounter epoch.
[b] (Ayala and Rigger, 2007); [c](Rayman and Varghese, 2001); [d](Kuninaka and Kawaguchi, 2011); [e](Rayman, 2020); [f](Tsuda et al., 2013); [g](Watanabe et al., 2017; Hirabayashi et al., 2021); [h](Benkhoff et al., 2021); [i](Polanskey et al., 2025).

extended periods can provide substantial mission-design flexibility and enable otherwise impractical transfers.

Table 2.1 places the present study within this broader mission context by summarising representative successful deep-space missions that employed low-thrust propulsion beyond Earth orbit. Most of these missions used solar electric propulsion (SEP), in which electric thrusters are powered by solar arrays, whereas IKAROS demonstrated solar-sail propulsion.

For consistency, the mission approaches listed in the table are defined as follows. In this work, *rendezvous* denotes matching the heliocentric motion of the target and conducting proximity operations without necessarily entering a gravitationally bound orbit around it. By contrast, *orbiter* denotes a mission architecture involving orbit insertion and sustained controlled motion about the target body. The term *sampler* refers to a spacecraft that performs a touch-and-go (TAG) manoeuvre to collect material from the target surface.

2.5. Reference Mission

The mission envisioned for this study is NASA's Psyche mission, the 14th mission of the Discovery Program. The spacecraft was launched from Cape Canaveral Space Force Station, Florida, on 13 October 2023 at 14:19:43 UTC, with the objective of investigating the metal-rich main-belt asteroid (16) Psyche (Dibb et al., 2024). The mission is designed to test whether Psyche is the remnant core of an early differentiated planetesimal that was disrupted by collisions or, alternatively, primordial unmelted material. Following a 5.6-year interplanetary cruise, including a Mars gravity assist in May 2026, the orbital science phase at Psyche is planned to begin in August 2029. A schematic representation of the nominal Psyche mission trajectory is shown in Figure 2.2.

The mission addresses two goals of NASA's Science Mission Directorate Planetary Science Division, which are refined into the following five science objectives:

- determine whether Psyche is a core or primordial unmelted material;
- determine the relative ages of regions on Psyche's surface;
- determine whether small metallic bodies incorporate the same light elements expected in Earth's high-pressure core;
- determine whether Psyche formed under conditions more oxidising or more reducing than Earth's core;
- characterise Psyche's morphology.

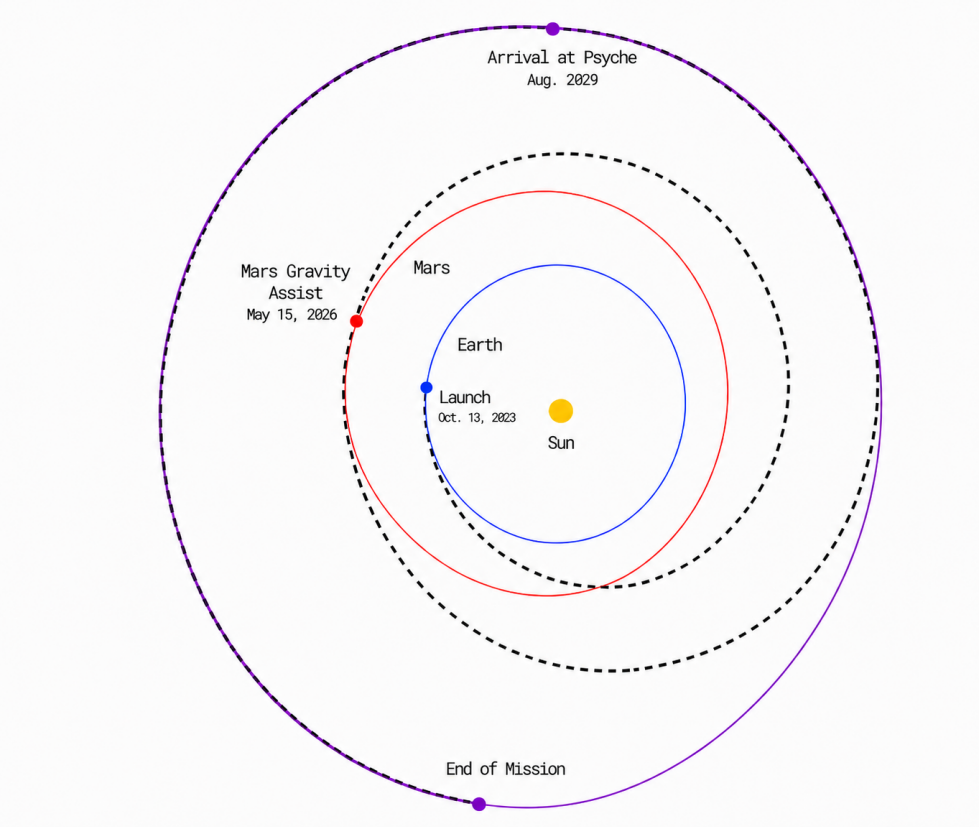


Figure 2.2: Schematic representation of the Psyche mission trajectory. Adapted from NASA/JPL-Caltech/ASU.[4]

Prior to launch, SpaceX provided nominal launch trajectories for each launch opportunity, designed to meet the Psyche launch targets within prescribed tolerances. The launch targets were defined at the Targeting Interface Point (TIP), 15 min after the second second-stage engine cut-off (SECO-2), using Earth-relative orbital parameters in the International Celestial Reference Frame (ICRF): characteristic energy (C_3), defined as the square of the hyperbolic excess velocity; declination of the launch asymptote (δ_∞); and right ascension of the launch asymptote (α_∞). For the 13 October 2023 launch, the targeted values at the TIP were $C_3 = 34.0\ \mathrm{km^2\,s^{-2}}$, $\delta_\infty = 0.4888475\ \mathrm{rad}$, and $\alpha_\infty = 1.9845214\ \mathrm{rad}$. The delivery requirements were imposed as bounds on deviations from these target values: $\Delta C_3 \in [-0.25, 0.25]\ \mathrm{km^2\,s^{-2}}$, $\Delta\delta_\infty \in [-0.0043633, 0.0043633]\ \mathrm{rad}$, and $\Delta\alpha_\infty \in [-0.0087266, 0.0087266]\ \mathrm{rad}$ (Bradley et al., 2026). This comparatively large delivery tolerance was acceptable because the spacecraft's fuel-efficient solar electric propulsion (SEP) system made launch-injection-error correction substantially less propellant-expensive than it would have been for a chemically propelled spacecraft (Bradley et al., 2026).

2.5.1. History and Characteristics of (16) Psyche

Psyche was discovered on 17 March 1852 at the Astronomical Observatory of Naples by the Italian astronomer Annibale de Gasparis (1819–1892) (de Gasparis, 1852). De Gasparis named the asteroid after Psyche, the Greek mythological figure, mortal wife of Cupid, and personification of the Greek word for "soul". Psyche was the 16th asteroid to be discovered, the first being Ceres, and one of eight asteroids discovered by de Gasparis (Badolati, 2007).

For more than a century, little was known about Psyche beyond its orbital elements. Progress accelerated during the 1960s and 1970s, when improved telescope and detector technologies, together with renewed interest in asteroids and their possible links to meteorites, enabled new observations. Optical and infrared radiometry, polarimetry, and photometry showed that Psyche is a relatively large and rapidly rotating body, with a spin period slightly above 4 h and an average diameter exceeding 200 km; its mass corresponds to approximately 1% of the mass of the main asteroid belt (Polanskey et al., 2025). Early broadband multispectral

[4] `https://psyche.ssl.berkeley.edu/wp-content/uploads/2023/09/Psyche-trajectory.svg`. Accessed: 7 June 2026.

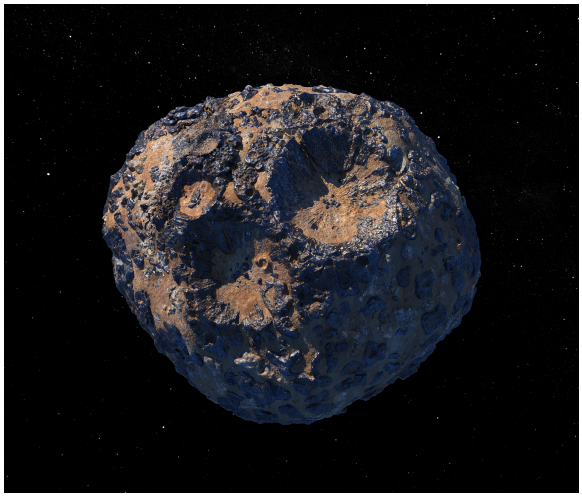


Figure 2.3: Artist's illustration of asteroid (16) Psyche.[5]

observations further revealed a featureless, reddish visible-wavelength spectrum, qualitatively consistent with the appearance shown in Figure 2.3, and similar to iron–nickel meteorites; this contributed to Psyche's classification as a prominent M-class asteroid (Polanskey et al., 2025). Psyche is the largest M-class asteroid and has been interpreted as a possible remnant core of an early differentiated planetesimal disrupted by collisions. As the third-largest asteroid to be visited by a spacecraft, it is expected to provide important constraints on the formation and evolution of planetesimals and planets.

Radar observations of Psyche date back to the 1980s (Ostro et al., 1985). The asteroid has one of the highest measured radar albedos among main-belt and near-Earth asteroids, 0.37 ± 0.09 (Shepard et al., 2017), which is among the strongest evidence that its surface and regolith are rich in iron–nickel metal. Radar, ground-based adaptive optics, and stellar occultation observations have also refined Psyche's size and shape. The current shape is best approximated by a triaxial ellipsoid with dimensions of approximately $280 \times 240 \times 170\ \mathrm{km}$, corresponding to an effective spherical diameter of approximately $222\ \mathrm{km}$ (Shepard et al., 2021). However, current shape models also indicate significant concavities, probably associated with large impact craters or basins (Shepard et al., 2021).

Mass estimates derived from orbital perturbations, combined with size estimates, suggest a bulk density between 3400 and $4100\ \mathrm{kg\,m^{-3}}$ (Elkins-Tanton et al., 2020), with a more recent estimate of $4172 \pm 145\ \mathrm{kg\,m^{-3}}$ (Farnocchia et al., 2024). These values are consistent with a mixture of metals, silicates, and porosity rather than a purely metallic composition. Additional evidence for a complex composition, structure, and origin comes from telescopic indications of small abundances of iron-bearing silicates, such as olivine and pyroxene, as well as hydrated mineral phases on the surface. Improvements in spatial and spectral resolution have also revealed surface heterogeneities (Polanskey et al., 2025). Despite extensive observations with modern ground- and space-based optical, infrared, and radar instruments, substantial uncertainties remain in Psyche's physical properties, composition, and origin. The Psyche spacecraft investigation is intended to resolve these uncertainties through close-range orbital measurements.

2.5.2. SPACECRAFT

The Psyche spacecraft uses solar electric propulsion (SEP) for all deterministic manoeuvres after launch, including interplanetary cruise, asteroid approach, and orbital operations. Its SEP system comprises two solar arrays, power electronics, power-processing units, four SPT-140 Hall-effect thrusters (HETs), xenon storage and distribution hardware, and two dual-axis positioning mechanisms (DAPMs). The single-axis-gimballed solar arrays maintain Sun pointing and generate approximately $21\ \mathrm{kW}$ at $1\ \mathrm{AU}$ and $2.7\ \mathrm{kW}$ at Psyche's heliocentric distance of approximately $3.1\ \mathrm{AU}$. The power electronics regulate the spacecraft-bus voltage and manage battery charging and discharging, while the power-processing units supply the HETs. Seven composite-overwrapped pressure vessels, connected through a common plenum, provide a total xenon-storage capacity of $1085\ \mathrm{kg}$, with each vessel holding up to $155\ \mathrm{kg}$ (Oh et al., 2025). Two thrusters are

[5] Source: NASA/JPL-Caltech/ASU, *Asteroid Psyche (Illustration)*, PIA24471. Available at: NASA Science Photojournal. Accessed: 12 June 2026.

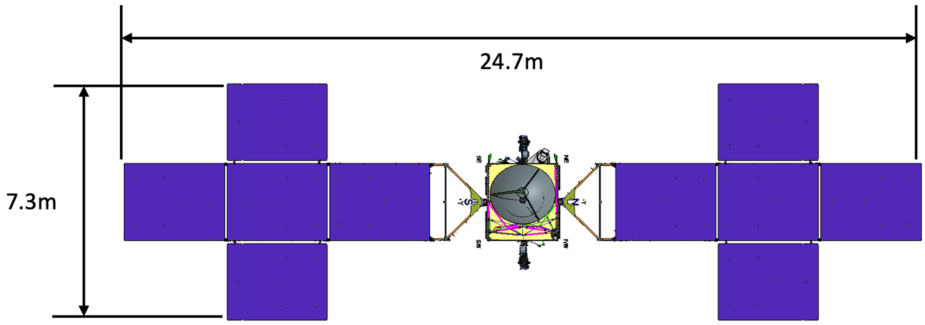


Figure 2.4: Psyche spacecraft with its large five-panel solar arrays deployed (Oh et al., 2025).

mounted on each DAPM, which can be gimballed about two axes, although only one thruster is operated at a time.

The complete flight system, including the two five-panel solar arrays, is $24.76\ \mathrm{m}$ long and $7.34\ \mathrm{m}$ wide, as shown in Figure 2.4. The spacecraft bus is approximately $4.9\ \mathrm{m}$ tall, $2.2\ \mathrm{m}$ wide, and $2.4\ \mathrm{m}$ deep, and the launch mass was approximately $2747\ \mathrm{kg}$.[6]

Although the SPT-140 produces substantially less thrust than a chemical primary-propulsion system, this is offset by its considerably higher propulsive efficiency. Its specific impulse varies with input power, ranging from $1283\ \mathrm{s}$ at a thrust of $58.2\ \mathrm{mN}$ and an input power of $0.91\ \mathrm{kW}$ to $1794\ \mathrm{s}$ at $279.3\ \mathrm{mN}$ and $4.57\ \mathrm{kW}$ (Snyder et al., 2020). The trajectory design must also account for activities that interrupt optimal thrusting. These primarily include weekly high-gain-antenna Earth pointing for command uplink and data downlink, together with shorter and more frequent interruptions for reaction-wheel momentum unloading caused mainly by electric-propulsion swirl torque. The resulting thrusting duty cycle is approximately 85%, although its value depends on the thrust level and spacecraft activities.

The interplanetary cruise is divided into Cruise 1, before the Mars gravity assist (MGA), and Cruise 2, after the MGA. NASA's Mission Design and Navigation team employs performance models that describe the available solar-array power, thrust, and xenon mass-flow rate as functions of power-processing-unit input power. The models also account for thrust-dependent swirl torque, other spacecraft electrical loads, and the available duty cycle. Because the effects of low-thrust manoeuvres accumulate over extended timescales, the thrust applied at a given time depends on the predicted spacecraft performance over the following weeks, months, and years. Robust performance margins are therefore required to reduce the probability of failing to satisfy future trajectory requirements.

The Psyche trajectory is designed to tolerate a minimum amount of missed thrust while maintaining the required arrival date. Cruise 1 is particularly robust to such interruptions. For instance, an anomaly on 1 April 2025 resulted in 77 days of missed thrust during Cruise 1, with minimal effects on the MGA timing and arrival at Psyche. Cruise 2 is generally more sensitive, particularly during a specific interval; nevertheless, the mission continues to meet its requirements and objectives. Additional robustness is being incorporated into the trajectory design, including an arrival earlier than strictly required, to reduce the effect of any future missed-thrust event on the science mission.

The mission requires a post-launch propulsive ΔV of $6.5\ \mathrm{km\,s^{-1}}$, provided entirely by the SEP system. By comparison, NASA's previous deep-space SEP missions, Deep Space 1 and Dawn, achieved ΔV values of $4.3\ \mathrm{km\,s^{-1}}$ and $11.5\ \mathrm{km\,s^{-1}}$, respectively. Of Psyche's total requirement, $5.8\ \mathrm{km\,s^{-1}}$ is needed to reach capture at the asteroid and a further $0.2\ \mathrm{km\,s^{-1}}$ is required for orbital transfers, resulting in an electric-propulsion deterministic ΔV requirement of $6.0\ \mathrm{km\,s^{-1}}$.

Table 2.2: Execution-error model for electric-propulsion thrust-acceleration magnitude (Bradley et al., 2023).

Mission sub-phase	**Per-arc bias** (3σ)	**Stochastic acceleration** (3σ)
Cruise 1 & 2	2.3%	0.5%
Approach	2.7%	0.5%

[6]NASA Science, "Psyche Spacecraft". `https://science.nasa.gov/mission/psyche/spacecraft/`. Accessed: 13 June 2026.

Table 2.3: Execution-error model for electric-propulsion thrust direction (Bradley et al., 2023).

Mission sub-phase	Per-axis bias per arc (3σ)	Stochastic uncertainty per axis (3σ)
Cruise 1 & 2	5.236×10^{-3} rad	2.618×10^{-2} rad
Approach	5.236×10^{-3} rad	2.618×10^{-2} rad

An additional $0.5\,\mathrm{km\,s^{-1}}$ is allocated to statistical manoeuvres, angular-momentum management, and missed-thrust accommodation. The MGA increases the spacecraft speed by approximately $0.4\,\mathrm{km\,s^{-1}}$ and provides a heliocentric orbit-plane change that would be costly to achieve propulsively. Its net effect is equivalent to a heliocentric velocity change of $2.0\,\mathrm{km\,s^{-1}}$ (Bradley et al., 2026).

Psyche's four HETs constitute its sole source of deterministic thrust. By contrast, NASA's two previous deep-space SEP missions, Deep Space 1 and Dawn, employed electrostatic ion thrusters. HETs have a simpler hardware architecture but provide less direct control over the ion beam, resulting in greater navigation-relevant uncertainty in thrust performance than ion thrusters (Bradley et al., 2023). Accurate characterisation of thrust-magnitude and thrust-direction uncertainties is therefore essential for SEP navigation, particularly because Psyche must perform precision orbital manoeuvres using HETs. Although HETs have substantial flight heritage on Earth-orbiting spacecraft and ESA's SMART-1 lunar mission, Psyche is the first deep-space mission to employ them. The navigation tools and error models developed for Deep Space 1 and Dawn provide important heritage; however, the adoption of HETs required new error models and uncertainty-quantification methods, which were developed by Bradley et al. (2023).

Bradley et al. (2023) presented pre-launch electric-propulsion manoeuvre-execution error models for Psyche. These models are adopted in the present framework to validate the robustness of the control policy obtained from the optimal control problem developed later in this work. Since the present study considers acceleration-magnitude and acceleration-direction errors during Cruise 1, Cruise 2, and the approach phase, the corresponding values are reproduced in Tables 2.2 and 2.3. The approach phase is defined as the interval extending from 100 days before capture to the capture date.

The thrust-magnitude error model includes burn-to-burn repeatability uncertainty, electric-propulsion hardware set-point uncertainty, spacecraft-mass uncertainty, and mass-flow-rate uncertainty. As reported by Bradley et al. (2023), the hardware set-point uncertainty is included in both the bias and stochastic components, making the resulting model conservative. The thrust-direction uncertainties are likewise taken from the pre-launch model of Bradley et al. (2023) and are reported in Table 2.3; the reader is referred to the original article for their detailed derivation.

The initial standard deviation of the spacecraft mass is set to an almost negligible value of $10^{-2}\,\mathrm{kg}$ because the launch mass is accurately known. Navigation uncertainty associated with measurement updates is initially represented by a position standard deviation of $200\,\mathrm{km}$ and a velocity standard deviation of $0.1\,\mathrm{m\,s^{-1}}$. These values are consistent with those reported by Oguri et al. (2022) and are based on orbit-determination analyses. They are varied in subsequent numerical experiments to assess the robustness of the proposed stochastic solver.

In this work, operational interruptions are not modelled. Moreover, the maximum available thrust and specific impulse are assumed to remain constant at $279.3\,\mathrm{mN}$ and $1794\,\mathrm{s}$, respectively, regardless of heliocentric distance. This simplification is appropriate because the objective is not to reconstruct the Psyche mission trajectory at high fidelity, but to develop and assess a robust optimisation methodology.

2.5.3. REQUIREMENTS

Based on the considerations developed throughout this chapter, mission requirements, simulator requirements, system requirements, and boundary conditions are defined to guide the design of the robust trajectory optimisation. **Mission requirements**

MIS-REQ-01: The mission time window shall be prescribed, with the launch epoch set to 13 October 2023 at 14:19:43 UTC and the arrival epoch set to 01 August 2029 at 12:00:00 UTC.
MIS-REQ-02: The reference trajectory shall include a Mars gravity assist, with the closest-passage state constrained in time, position, and velocity using Psyche spacecraft ephemeris data.
MIS-REQ-03: The MGA shall satisfy a minimum altitude of TBD with a probability greater than 99%.

Simulator requirements
SIM-REQ-01: The selected integrator–propagator combination shall achieve a maximum integration error of $10\,\mathrm{m}$.

SIM-REQ-02: The physical-model accuracy shall achieve a maximum error of $100\ \mathrm{m}$.
SIM-REQ-03: The uncertainty-propagation method shall be validated against Monte Carlo simulations and yield a maximum error of TBD.

System requirements
SYS-REQ-01: The guidance system shall be robust to launch-injection errors bounded by $\Delta C_3 \in [-0.25, 0.25]\ \mathrm{km^2\,s^{-2}}$, $\Delta\delta_\infty \in [-0.0043633, 0.0043633]\ \mathrm{rad}$, and $\Delta\alpha_\infty \in [-0.0087266, 0.0087266]\ \mathrm{rad}$.
SYS-REQ-02: The guidance system shall be robust to navigation errors of $200\ \mathrm{km}$ in position and $0.1\ \mathrm{m\,s^{-1}}$ in velocity, both expressed as 1σ uncertainties.
SYS-REQ-03: The guidance system shall be robust to an initial spacecraft-mass uncertainty of $10^{-2}\ \mathrm{kg}$, expressed as a 1σ uncertainty.
SYS-REQ-04: The guidance system shall be robust to thrust-acceleration magnitude errors, expressed at 3σ, consisting of a per-arc bias of 2.3% during Cruise 1 and Cruise 2, a per-arc bias of 2.7% during approach, and a stochastic acceleration uncertainty of 0.5% in all three mission sub-phases.
SYS-REQ-05: The guidance system shall be robust to thrust-direction errors, expressed at 3σ, consisting of a per-axis and per-arc bias of $5.236 \times 10^{-3}\ \mathrm{rad}$ and a per-axis stochastic uncertainty of $2.618 \times 10^{-2}\ \mathrm{rad}$.
SYS-REQ-06: The commanded thrust magnitude shall not exceed the maximum available thrust, set to $279.3\ \mathrm{mN}$.
SYS-REQ-07: The propulsion system shall use a constant specific impulse of $1794\ \mathrm{s}$.
SYS-REQ-08: The guidance system shall be able to counteract perturbative forces.
SYS-REQ-09: The guidance system shall remain robust to stochastic perturbations characterised by diffusion-matrix intensities of TBD for position, TBD for velocity, and TBD for mass.
SYS-REQ-10: The flight system shall have a total initial mass of $2747\ \mathrm{kg}$, including a xenon propellant mass of $1085\ \mathrm{kg}$.

Finally, the boundary conditions adopted for the mission scenario and the methodology are listed below.

BC1: The mission sequence and timing are prescribed. The optimisation problem considered in this work is formulated as a single-objective optimal control problem with a predefined mission sequence and time window. In general, such mission definitions are obtained during the early stages of preliminary mission design through broad searches based on simplified dynamical models and methods, such as patched-conic approximations. Global optimisation algorithms are typically employed during this phase to explore the discrete combinatorial design space and determine the sequence in which candidate targets or fly-by bodies are visited. The transfer between consecutive bodies additionally requires the solution of a continuous optimal control problem to determine the corresponding trajectory and thrust profile. When the discrete sequencing problem and the continuous trajectory-optimisation problem are solved simultaneously, the resulting formulation is referred to as a mixed-integer optimal control problem.

In particular, the mean state vector at MGA closest passage is constrained in time, position, and velocity using the spacecraft ephemeris provided by a Spacecraft and Planet Kernel (SPK) file.[7] The file was obtained from NASA's Navigation and Ancillary Information Facility (NAIF) server (Acton, 1996). The kernel was generated using Monte 171.1 (Smith et al., 2016) on 18 May 2026 at 21:56 UTC and provides an approximately weeks-long short-term prediction of the Psyche spacecraft ephemeris. At the time of writing, the higher-accuracy reconstructed trajectory produced by the Psyche Mission Design and Navigation team at the Jet Propulsion Laboratory had not yet been distributed as a separate SPK file and could therefore not be used in this work.
BC2: Attitude dynamics and attitude control are not simulated. The spacecraft is assumed to be capable of autonomously pointing its thrusters in the required direction, irrespective of operational constraints. The interplanetary point-mass trajectory-generation problem is therefore treated independently of attitude control, which lies outside the scope of this work. In reality, spacecraft motion is governed by six-degree-of-freedom (6-DOF) rigid-body dynamics. For preliminary deep-space trajectory design, however, translational and rotational motion can often be decoupled because the forces governing translation have a negligible direct influence on rotational dynamics, and vice versa. This decoupling substantially simplifies the design process.
BC3: According to Oh et al. (2025), Psyche's Hall-thruster system provides 235 predefined throttle levels. Operators select one of these levels rather than directly commanding the xenon mass-flow rate. The Hall-thruster power-processing unit then uses autonomous feedback control to adjust the xenon flow rate and

[7] Available at: `https://naif.jpl.nasa.gov/pub/naif/PSYCHE/kernels/spk/psyche_sc-eph_260314-261001_260518_v1.bsp`. Accessed: 16 June 2026.

achieve the corresponding discharge-current setpoint. Although the hardware can, in principle, operate at any setpoint, mission-design requirements led to its discretisation into throttle levels. This choice reflects the fact that deep-space navigation prioritises thrust predictability and repeatability over continuously variable thrust magnitude.

In this work, however, the thrust magnitude is assumed to vary continuously between 0 and $279.3\ \mathrm{mN}$, while the specific impulse is fixed at a representative constant value. Operational constraints are also neglected. These assumptions are commonly adopted in the literature (Oguri and Lantoine, 2022; Marmo et al., 2025) and preserve problem tractability while introducing only a limited modelling error. In reality, solar electric propulsion systems can operate in several modes, and their performance is coupled to the power generated by the solar arrays. Since the available solar-array power depends on heliocentric distance, which is itself determined by the optimised trajectory, modelling the complete set of thruster operating modes would introduce an additional coupling that lies outside the scope of this work. The present study instead focuses on the development of a robust trajectory-optimisation methodology augmented by GPU acceleration.

The assumption of continuously variable thrust is therefore adopted as a deliberate simplification suitable for preliminary mission design. As described in Section 5.5, the stochastic optimal control problem is transcribed using a hybrid multiple-shooting formulation in which the control input is represented as piecewise constant over each control interval. Each control value can therefore be interpreted as the average thrust applied during that interval. Consequently, the continuous control solution does not necessarily imply that the thruster must physically generate every intermediate thrust level. It may instead represent a duty-cycled or pulse-width-modulated command whose total impulse over the interval is equivalent to that prescribed by the continuous solution. This approximation is reasonable when the thrust-switching timescale is much shorter than the timescale over which the heliocentric trajectory evolves.

BC4: The simulation of a complete navigation architecture lies outside the scope of this work. The navigation error is therefore modelled as an additive Gaussian-noise component.

BC5: A less conservative risk-allocation method, discussed in detail in Section 5.3.6, could be employed instead of the naive method adopted in this work. This choice is motivated by the objective of the study, which is to exploit parallelism in robust trajectory-optimisation algorithms. More advanced risk-allocation methods would require a transition to sequential optimisation, which is not the approach pursued in this work.

In conclusion, a detailed project plan divided into work packages, together with a complete Gantt chart, is provided in Appendix A1.

3

FLIGHT DYNAMICS

This chapter discusses the kinematics and dynamics that characterise the motion of a spacecraft on an interplanetary transfer. Section 3.1 defines the reference frames used throughout the thesis, while Section 3.2 presents different sets of elements to represent the spacecraft's translational state. In Section 3.3, the transformations that describe the relative orientation between the reference frames are introduced. Section 3.4 then presents the method of patched conics, which is used to define the departure conditions of the interplanetary transfer and the B-plane for the Mars gravity assist. Section 3.5 provides an overview of the models used to describe the environment models and forces acting on the spacecraft. Finally, the equations of motion are presented in Section 3.6.

3.1. REFERENCE FRAMES

The state of the spacecraft at a certain moment has to be expressed with respect to a given reference frame, which is defined by the position of its origin and the orientation of its three axes[1]. Usually, reference frames are based on the rotation axis of a given celestial body, whose points of intersection with the body's surface are referred to as the *north* and *south* poles.

Earth Mean Equator and Mean Equinox of J2000 (EME2000)

The EME2000 realisation is based on distant stars. The origin of this reference frame is primarily located at the solar system barycentre (SSB), with the $X_{EME2000}$-axis pointing towards the mean vernal equinox. The $Z_{EME2000}$-axis is aligned with the Earth's rotation axis as it was at the J2000 Barycentric Dynamical Time (TDB) epoch[2], while the $Y_{EME2000}$-axis completes a right-handed coordinate system. The EME2000 reference frame is identified by the symbol $\mathcal{F}_{EME2000}$, and its basis vectors are labelled as $\hat{\boldsymbol{x}}_{EME2000}$, $\hat{\boldsymbol{y}}_{EME2000}$, and $\hat{\boldsymbol{z}}_{EME2000}$. For the sake of precision, it should be noted that it would be more appropriate to refer to this reference frame as *pseudo-inertial* when the origin is placed, for example, at the centre of mass of (16) Psyche, rather than inertial. In fact, the asteroid has an acceleration about the Sun as it orbits around it. The Earth-centred version of the J2000 frame is often referred to as the Earth-Centred Inertial (ECI) frame.

International Celestial Reference Frame (ICRF)

The International Celestial Reference System (ICRS) is a space-time coordinate system within the framework of the theory of general relativity, with its origin primarily located T the SSB and its axes directions fixed with respect to distant quasars and active galactic nuclei, closely aligned with those of the historical EME2000 reference frame. The International Celestial Reference Frame (ICRF) constitutes the practical realisation of the ICRS through the adopted coordinates observed using Very Long Baseline Interferometry (VLBI). Furthermore, since the defining radio sources are assumed to be so distant that their angular motions, seen from Earth, are negligible, there is no epoch associated with the ICRF (Wakker, 2015). Additionally, because the distance from the planets to the solar system barycentre is negligible compared with the distance to the frame-defining quasar radio sources, no rotation is needed when translating this system to another body-centred definition, e.g. an Earth-centred one.

[1] It is noted that the definition given here is technically that of a *reference system* rather than of a *reference frame*.
[2] J2000 TDB is 2000 JAN 01 12:00:00 TDB, or JD 2451545.0 TDB.

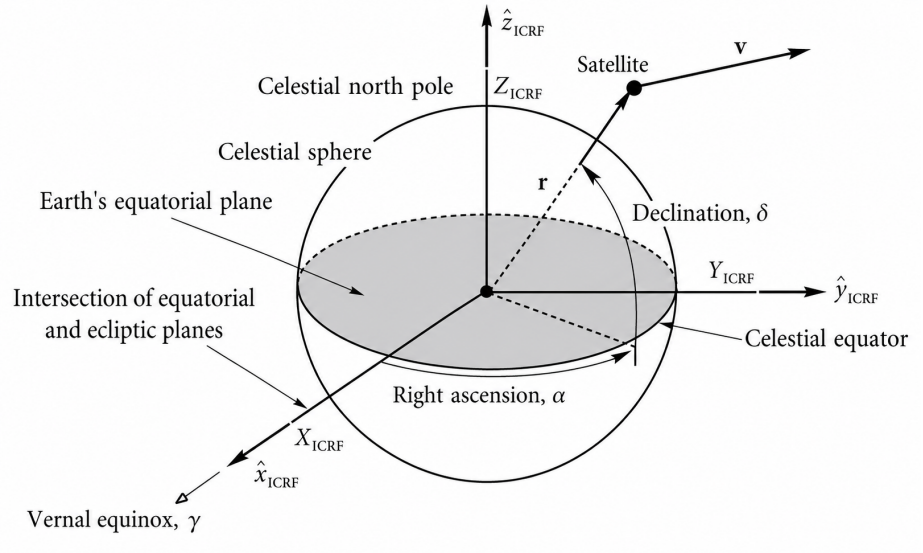


Figure 3.1: Representation of $\mathcal{F}_{ICRF}$. Adapted from (Curtis, 2005).

The orientation difference between the historical EME2000 frame and the ICRS is smaller than 0.1 arcseconds. For this reason, in many applications these two reference frames are considered to be equivalent. The main reason for the difference is that EME2000 is defined with an associated epoch. In the SPICE toolkit, the frame name `J2000` is used as the conventional label for the ICRF orientation. Consequently, modern SPICE ephemerides and orientation data labelled as being expressed in the `J2000` frame are, in practice, referenced to the ICRF. The name `J2000` is retained for historical and backward-compatibility reasons, whereas `ICRF` is not recognised as a separate built-in SPICE frame name. Therefore, no transformation between `J2000` and the ICRF is required when using SPICE kernels containing ICRF-referenced data[3].

The ICRF reference frame is identified by the symbol $\mathcal{F}_{ICRF}$, and its basis vectors are labelled as $\hat{\boldsymbol{x}}_{ICRF}$, $\hat{\boldsymbol{y}}_{ICRF}$, and $\hat{\boldsymbol{z}}_{ICRF}$. In this work, an ICRF-oriented frame may be defined with its origin either at the solar system barycentre or at the centre of mass of (16) Psyche, with the same implications discussed above for the EME2000 frame.

Body-Fixed Reference Frame

Body-fixed frames are, by definition, non-inertial, since they follow the rotation of the body about its axis. The origin is placed at the centre of the celestial body, the X_{BF}-axis intersects a chosen meridian, the Z_{BF}-axis points towards the north pole, and the Y_{BF}-axis completes the right-handed system. Figure 3.2 shows an example for Earth, whose fixed frame is named the International Terrestrial Reference Frame (ITRF). The latter has its X_{ITRF}-axis pointing towards the Greenwich meridian, its Z_{ITRF}-axis directed along the mean pole of Earth's rotation, and the Y_{ITRF}-axis directed to complete the right-handed orthonormal trihedron. Several conversions are needed to transition from the ICRF to body-fixed reference frames. The rotation matrix is generally built from the body's pole orientation and prime-meridian angle, including precession, nutation, and libration when relevant. Detailed formulations for constructing the transformations from the ICRF/EME2000 orientation to the body-fixed frames of solar system bodies are provided by the IAU Working Group on Cartographic Coordinates and Rotational Elements (Archinal et al., 2018), whereas the transformation to the International Terrestrial Reference Frame is defined in the IERS Conventions (Petit and Luzum, 2010). The general body-fixed reference frame is identified by the symbol $\mathcal{F}_{BF}$, and its basis vectors are labelled as $\hat{\boldsymbol{x}}_{BF}$, $\hat{\boldsymbol{y}}_{BF}$, and $\hat{\boldsymbol{z}}_{BF}$.

Perifocal (PQW)

Its origin O_P is located at the SSB. The X_P-axis is directed towards the periapsis of the spacecraft orbit, positive away from the SSB, whereas the Z_P-axis is parallel to the angular momentum vector, and thus orthogonal to the plane in which the spacecraft orbits the SSB. The Y_P-axis completes the right-handed system. This frame is identified by the symbol $\mathcal{F}_P$, and its basis vectors are labelled as $\hat{\boldsymbol{p}}$, $\hat{\boldsymbol{q}}$, and $\hat{\boldsymbol{w}}$.

[3] Source: *MSOPCK User's Guide*, Section "ICRF vs. J2000". Available at: `NAIF MSOPCK User's Guide`. Accessed: 17 June 2026.

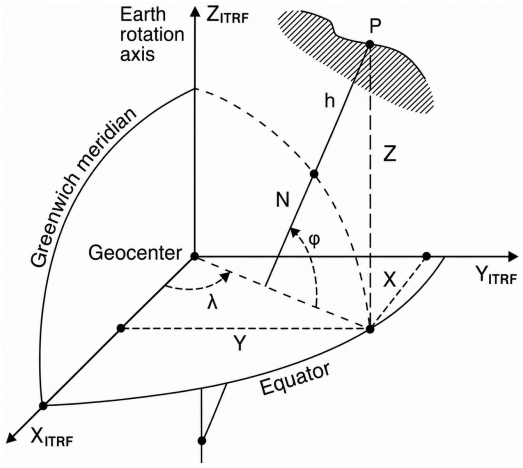


Figure 3.2: Representation of $\mathcal{F}_{ITRF}$.

Radial-Transverse-Normal (RTN)
Its origin O_R is located at the centre of mass of the spacecraft. The R-axis (radial) points away from the centre of the central body and along the radial component of the gravitational acceleration, while the N-axis (normal, or cross-track) is normal to the orbit plane in the direction of the angular momentum. The T-axis (transverse, or along-track) completes the right-handed system. In general, the along-track direction does not coincide with the velocity, although this is the case for circular orbits. This reference frame is identified by the symbol $\mathcal{F}_R$, and its basis vectors are labelled as $\hat{\boldsymbol{r}}$, $\hat{\boldsymbol{t}}$, and $\hat{\boldsymbol{n}}$.

3.2. STATE REPRESENTATION

Since only the translational motion will be modelled, the state of the spacecraft is uniquely identified by six components, three for the position and three for the velocity. Three coordinate sets will be used throughout this work to represent the spacecraft's state:

- Cartesian coordinates ($\mathcal{C}$): these provide the simplest and most flexible way to describe the equations of motion in an inertial frame. They are employed both for propagating dynamics and computing guidance commands.
- Spherical coordinates ($\mathcal{S}$): these are employed exclusively to reconstruct the outgoing/ingoing hyperbolic excess velocity relative to the central body under analysis.
- Orbital elements ($\mathcal{O}$): this representation will improve the quality of the analysis, as it offers deeper insights into the physics of the problem.

Cartesian coordinates
Cartesian coordinates are generally used to describe the position and velocity of the spacecraft with respect to the selected frame origin and orientation. Position $\boldsymbol{r}$ and velocity $\boldsymbol{v}$ are identified by the following variables:

$$\boldsymbol{r} = \begin{pmatrix} x & y & z \end{pmatrix}^T \qquad \boldsymbol{v} = \begin{pmatrix} v_x & v_y & v_z \end{pmatrix}^T \tag{3.1}$$

The norms of the position and velocity are identified by r and v.

Spherical coordinates
The spherical position is defined as:

$$\boldsymbol{r} = \begin{pmatrix} r & \phi & \lambda \end{pmatrix}^T \tag{3.2}$$

where r is the distance between the centre of mass of the vehicle and the origin of the reference frame, ϕ is the latitude, and λ is the longitude. The longitude is measured positively from west to east ($0 \leq \lambda \leq 2\pi$), while the latitude is positive in the north direction and negative to the south ($-\frac{\pi}{2} \leq \phi \leq \frac{\pi}{2}$). As will be shown later, it is common to express spherical coordinates with respect to a rotating frame such as the BF frame.

The relative departure/arrival conditions are specified through the characteristic energy C_3, the right ascension α, and the declination δ of the outgoing/incoming hyperbolic asymptote:

$$\boldsymbol{v}_\infty = \begin{pmatrix} C_3 & \alpha & \delta \end{pmatrix}^T \tag{3.3}$$

These quantities are generally expressed along the axes of the ICRF and they can be visualised in Figure 3.1. The right ascension α is measured in the X_{ICRF}–Y_{ICRF} reference plane, from the positive X_{ICRF}-axis towards the positive Y_{ICRF}-axis. The declination δ is measured from the X_{ICRF}–Y_{ICRF} plane towards the hyperbolic-excess-velocity vector.

Orbital elements

The solution to the two-body problem is a conic section which is uniquely identified by a set of six orbital elements, or Kepler elements. The state vector in terms of Kepler elements is the following:

$$\boldsymbol{x} = \begin{pmatrix} a & e & i & \Omega & \omega & \theta \end{pmatrix}^T \tag{3.4}$$

where a is the semi-major axis, e is the eccentricity, i the inclination, Ω the right ascension of the ascending node (RAAN), ω the argument of periapsis, and θ the true anomaly. The semi-major axis describes the size of the orbit, while the eccentricity defines its shape. The orientation of the orbital plane in space is given by i and Ω, while ω describes how the orbit is oriented within the orbital plane. Finally, θ describes the position of the spacecraft along the orbit. Other relevant quantities to describe a Kepler orbit are the eccentricity vector $\boldsymbol{e}$ and the angular momentum vector $\boldsymbol{h}$, defined as:

$$\boldsymbol{e} = \frac{\boldsymbol{v} \times \boldsymbol{h}}{\mu} - \frac{\boldsymbol{r}}{r} \qquad \boldsymbol{h} = \boldsymbol{r} \times \boldsymbol{v} \tag{3.5}$$

3.3. COORDINATE AND FRAME TRANSFORMATIONS

The spacecraft state may be expressed using different coordinate representations and reference frames, depending on the specific analysis being performed. A state conversion changes the mathematical representation of the same physical state while leaving the reference frame unchanged. By contrast, a frame transformation expresses the same physical quantity with respect to a different origin or set of basis vectors.

3.3.1. STATE CONVERSIONS

Cartesian–Orbital ($\mathcal{C} \leftrightarrow \mathcal{O}$)

To compute the orbital elements from a given Cartesian state, the following procedure has to be followed:

$$a = \frac{\mu r}{2\mu - r v^2} \tag{3.6}$$

$$e = \|\boldsymbol{e}\| \tag{3.7}$$

$$i = \arccos\left(\frac{\boldsymbol{h} \cdot \hat{\boldsymbol{z}}_{\text{ICRF}}}{h}\right) \tag{3.8}$$

$$\Omega = \begin{cases} \arccos\left(\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{x}}_{\text{ICRF}}\right) & \hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{y}}_{\text{ICRF}} \geq 0 \\ 2\pi - \arccos\left(\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{x}}_{\text{ICRF}}\right) & \hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{y}}_{\text{ICRF}} < 0 \end{cases} \tag{3.9}$$

$$\omega = \begin{cases} \arccos\left(\dfrac{\boldsymbol{e} \cdot \hat{\boldsymbol{n}}}{e}\right) & \boldsymbol{e} \cdot \hat{\boldsymbol{z}}_{\text{ICRF}} \geq 0 \\ 2\pi - \arccos\left(\dfrac{\boldsymbol{e} \cdot \hat{\boldsymbol{n}}}{e}\right) & \boldsymbol{e} \cdot \hat{\boldsymbol{z}}_{\text{ICRF}} < 0 \end{cases} \tag{3.10}$$

$$\theta = \begin{cases} \arccos\left(\dfrac{\boldsymbol{r} \cdot \boldsymbol{e}}{re}\right) & \boldsymbol{r} \cdot \boldsymbol{v} \geq 0 \\ 2\pi - \arccos\left(\dfrac{\boldsymbol{r} \cdot \boldsymbol{e}}{re}\right) & \boldsymbol{r} \cdot \boldsymbol{v} < 0 \end{cases} \tag{3.11}$$

where $\hat{\boldsymbol{n}}$ is the unit vector along the line of nodes, defined as

$$\hat{\boldsymbol{n}} = \frac{\hat{\boldsymbol{z}}_{\mathrm{ICRF}} \times \boldsymbol{h}}{\|\hat{\boldsymbol{z}}_{\mathrm{ICRF}} \times \boldsymbol{h}\|} \tag{3.12}$$

The opposite procedure instead requires rotating the state into the PQW frame:

$$\boldsymbol{r}^P = \frac{a\left(1-e^2\right)}{1+e\cos\theta}\begin{pmatrix}\cos\theta\\ \sin\theta\\ 0\end{pmatrix} \qquad \boldsymbol{v}^P = \sqrt{\frac{\mu}{a\left(1-e^2\right)}}\begin{pmatrix}-\sin\theta\\ e+\cos\theta\\ 0\end{pmatrix} \tag{3.13}$$

Cartesian–Spherical ($\mathcal{C} \leftrightarrow \mathcal{S}$)
The following relations can be used to move between spherical and cartesian position representations:

$$\begin{cases} r = \sqrt{x^2+y^2+z^2} \\ \phi = \arcsin\left(\dfrac{z}{r}\right) \\ \lambda = \begin{cases} \arccos\left(\dfrac{x}{\sqrt{x^2+y^2}}\right) & y \geq 0 \\ 2\pi - \arccos\left(\dfrac{x}{\sqrt{x^2+y^2}}\right) & y < 0 \end{cases} \end{cases} \qquad \begin{cases} x = r\cos\phi\cos\lambda \\ y = r\cos\phi\sin\lambda \\ z = r\sin\phi \end{cases} \tag{3.14}$$

To compute the Cartesian hyperbolic excess velocity from the characteristic energy C_3, the right ascension α, and the declination δ, the following equation is used:

$$\boldsymbol{v}_\infty = \sqrt{C_3}\begin{pmatrix}\cos\delta\cos\alpha\\ \cos\delta\sin\alpha\\ \sin\delta\end{pmatrix} \tag{3.15}$$

The inverse conversion from the Cartesian hyperbolic excess velocity to the spherical parameters is given by

$$C_3 = \|\boldsymbol{v}_\infty\|^2 \tag{3.16}$$

$$\alpha = \begin{cases} \arccos\left(\dfrac{v_{\infty,x}}{\sqrt{v_{\infty,x}^2+v_{\infty,y}^2}}\right) & v_{\infty,y} \geq 0 \\ 2\pi - \arccos\left(\dfrac{v_{\infty,x}}{\sqrt{v_{\infty,x}^2+v_{\infty,y}^2}}\right) & v_{\infty,y} < 0 \end{cases} \tag{3.17}$$

$$\delta = \arcsin\left(\frac{v_{\infty,z}}{v_\infty}\right) \tag{3.18}$$

3.3.1.1. FRAME ROTATIONS

The transformation from a generic frame A to a generic frame B can be expressed using the rotation matrix $\boldsymbol{C}_{B/A}$. Since this matrix is orthogonal, its inverse is equal to its transpose, such that $\boldsymbol{C}_{A/B} = \boldsymbol{C}_{B/A}^{-1} = \boldsymbol{C}_{B/A}^{T}$. Any three-dimensional rotation can be represented as a sequence of elementary rotations about the coordinate axes. Therefore, a general rotation matrix can be obtained as the product of at most three elementary rotation matrices:

$$\boldsymbol{C}_X(\Theta) = \begin{bmatrix}1 & 0 & 0\\ 0 & \cos\Theta & \sin\Theta\\ 0 & -\sin\Theta & \cos\Theta\end{bmatrix} \quad \boldsymbol{C}_Y(\Theta) = \begin{bmatrix}\cos\Theta & 0 & -\sin\Theta\\ 0 & 1 & 0\\ \sin\Theta & 0 & \cos\Theta\end{bmatrix} \quad \boldsymbol{C}_Z(\Theta) = \begin{bmatrix}\cos\Theta & \sin\Theta & 0\\ -\sin\Theta & \cos\Theta & 0\\ 0 & 0 & 1\end{bmatrix} \tag{3.19}$$

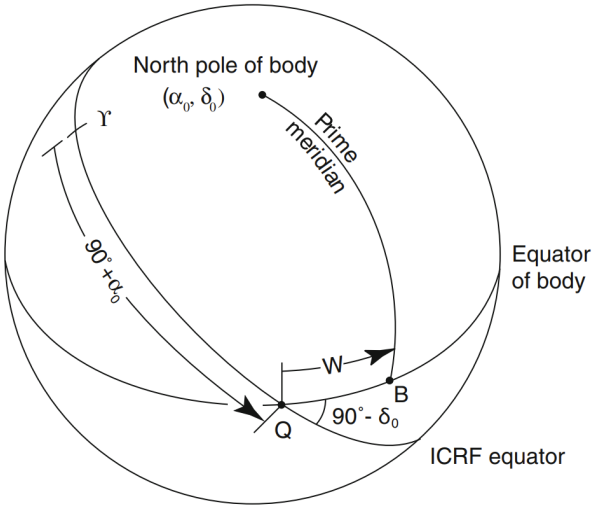


Figure 3.3: Reference system used to define the orientation of the planets and their satellites (Archinal et al., 2011).

The matrices above describe rotations through an arbitrary angle Θ about the X-, Y-, and Z-axes, respectively, of a generic reference frame. The specific rotation matrices associated with the reference frames used in this work are described below.

Radial-transverse-normal to EME2000/ICRF ($\mathcal{F}_R \rightarrow \mathcal{F}_{EME2000}/\mathcal{F}_{ICRF}$)
The unit vectors defining the RTN frame can be expressed with respect to the EME2000/ICRF frame directly from the spacecraft position and velocity vectors at a given epoch:

$$\hat{\boldsymbol{r}} = \frac{\boldsymbol{r}}{\|\boldsymbol{r}\|} \qquad \hat{\boldsymbol{n}} = \frac{\boldsymbol{r} \times \boldsymbol{v}}{\|\boldsymbol{r} \times \boldsymbol{v}\|} \qquad \hat{\boldsymbol{t}} = \hat{\boldsymbol{n}} \times \hat{\boldsymbol{r}} \tag{3.20}$$

The components of these unit vectors are the direction cosines of the RTN basis with respect to the inertial basis. Therefore, the corresponding rotation matrix is obtained by arranging the RTN unit vectors as columns:

$$\boldsymbol{C}_{EME2000/R} = \boldsymbol{C}_{ICRF/R} = \begin{bmatrix} \hat{\boldsymbol{r}} & \hat{\boldsymbol{t}} & \hat{\boldsymbol{n}} \end{bmatrix} \tag{3.21}$$

Perifocal to EME2000/ICRF ($\mathcal{F}_P \rightarrow \mathcal{F}_{EME2000}/\mathcal{F}_{ICRF}$)
The transformation from the perifocal frame to the EME2000/ICRF frame, is obtained through three consecutive rotations defined by Ω, i, and ω:

$$\boldsymbol{C}_{ICRF/P} = \boldsymbol{C}_{EME2000/P} = \boldsymbol{C}_Z(-\Omega)\boldsymbol{C}_X(-i)\boldsymbol{C}_Z(-\omega) \tag{3.22}$$

EME2000/ICRF to Body-Fixed ($\mathcal{F}_{EME2000}/\mathcal{F}_{ICRF} \rightarrow \mathcal{F}_{BF}$)
The transformation between two reference frames can be done by applying a translation and a rotation. If the origin of the two frames is the same, then only the rotation is necessary. For planets and satellites, the origin of their reference frames is usually the barycentre of the body; therefore, to transform between inertial and body-fixed frames, only a rotation is required. The IAU Working Group has defined planetary coordinate systems relative to their mean axis of rotation and various definitions of longitude depending on the body (e.g. references to a surface feature such as a crater) (Archinal et al., 2018).

The three rotational angles ($\pi/2 + \alpha_0$, $\pi/2 - \delta_0$, W) that determine the transformation between a body-fixed frame and the ICRF can be seen in Figure 3.3.

The north pole is defined as the pole of rotation that lies on the north side of the invariable plane of the solar system. The position of the north pole is given by the value of its right ascension α_0 and declination δ_0. The two intersection points of the body's equator and the ICRF equator are $\alpha_0 \pm \pi/2$. The point at $\alpha_0 + \pi/2$ was chosen to define node Q as one of the intersection points. Point B was chosen as the intersection of the body's prime meridian and equator. The location of the prime meridian is specified by providing a value for W, which is the angle measured easterly along the body's equator between node Q and point B. The right ascension of the point Q is $\alpha_0 + \pi/2$, and the inclination of the planet's equator to the celestial equator is

$\pi/2 - \delta_0$ (Archinal et al., 2011). The constants (α_0, δ_0, W) are given for each planet and its satellites in the form of polynomials at epoch J2000.0 TDB.

The rotation between the position vector $\boldsymbol{r}^{\mathrm{ICRF}}/\boldsymbol{r}^{\mathrm{EME2000}}$ and the body-fixed position vector $\boldsymbol{r}^{\mathrm{BF}}$ can be written in the form of rotation matrices as follows (Montenbruck and Gill, 2001):

$$\boldsymbol{r}^{\mathrm{BF}} = \boldsymbol{R}(t)\boldsymbol{r}^{\mathrm{ICRF}} = \boldsymbol{R}(t)\boldsymbol{r}^{\mathrm{EME2000}} \tag{3.23}$$

where $\boldsymbol{R}(t)$ is the time-dependent rotation matrix, which has the following form:

$$\boldsymbol{R}(t) = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ u_{21} & u_{22} & u_{23} \\ u_{31} & u_{32} & u_{33} \end{bmatrix} \tag{3.24}$$

with

$$\begin{aligned} u_{11} &= \cos\psi\cos\phi - \cos\theta\sin\phi\sin\psi \\ u_{12} &= \cos\psi\sin\phi + \cos\theta\cos\phi\sin\psi \\ u_{13} &= \sin\psi\sin\theta \\ u_{21} &= -\sin\psi\cos\phi - \cos\theta\sin\phi\cos\psi \\ u_{22} &= -\sin\psi\sin\phi + \cos\theta\cos\phi\cos\psi \\ u_{23} &= \cos\psi\sin\theta \\ u_{31} &= \sin\theta\sin\phi \\ u_{32} &= -\sin\theta\cos\phi \\ u_{33} &= \cos\theta \end{aligned} \tag{3.25}$$

where

$$\begin{aligned} \phi &= \pi/2 + \alpha_0 \\ \theta &= \pi/2 - \delta_0 \\ \psi &= W \end{aligned} \tag{3.26}$$

3.4. METHOD OF PATCHED CONICS

In this section, the method of patched conics is introduced, following the treatment of Curtis (2005), as it will be used to define the departure conditions of the interplanetary transfer under analysis and the B-plane for the Mars gravity assist.

The term conics refers to the fact that two-body, or Keplerian, orbits are conic sections with the focus at the attracting body. To study an interplanetary trajectory, it is assumed that when the spacecraft is outside the sphere of influence of a planet, it follows an unperturbed Keplerian orbit around the Sun. Because interplanetary distances are so vast, for heliocentric orbits the size of the spheres of influence may be neglected, so that they can be treated, like the planets they surround, as points in space coinciding with the planetary centres. Within each planetary sphere of influence, the spacecraft travels an unperturbed Keplerian path about the planet. Although the sphere of influence is negligibly small on the scale of the solar system, from the point of view of the planet it is very large and may be considered to lie at infinity.

To analyse a mission from planet 1 to planet 2 using the method of patched conics, the heliocentric trajectory that intersects the desired positions of the two planets in their orbits is first determined. This trajectory takes the spacecraft from the sphere of influence of planet 1 to that of planet 2. At the spheres of influence, the heliocentric velocities of the transfer orbit are computed relative to the planet to establish the velocities "at infinity", which are then used to determine the planetocentric departure trajectory at planet 1 and the arrival trajectory at planet 2. In this way, the three conics, one centred at the Sun and the other two centred at the planets in question, are "patched" together.

For completeness, the radius of the classical Laplace sphere of influence of a secondary body is given by

$$r_{\mathrm{SOI}} = r\left(\frac{m}{M}\right)^{2/5}, \tag{3.27}$$

where r is the heliocentric distance of the orbit of the secondary body around the primary body, m is the mass of the secondary body, and M is the mass of the primary body.

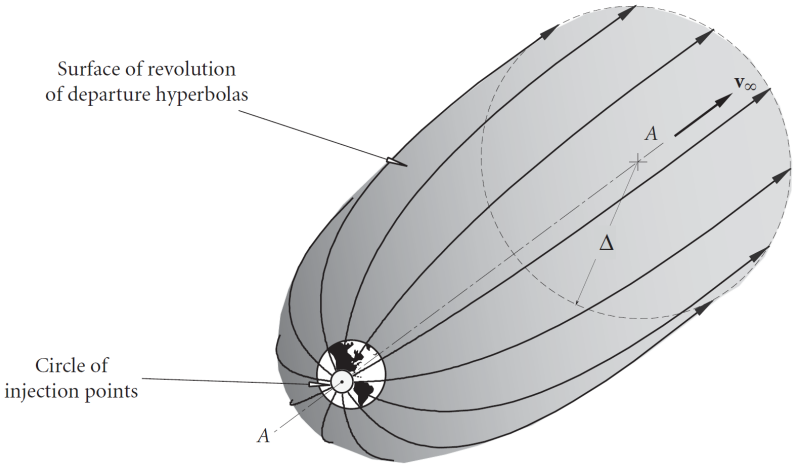


Figure 3.4: Locus of possible departure trajectories for a given $\boldsymbol{v}_\infty$ and r_p (Curtis, 2005).

3.4.1. PLANETARY DEPARTURE

In order to escape the gravitational pull of a planet, the spacecraft must travel a hyperbolic trajectory relative to the planet, arriving at its sphere of influence with a relative velocity $\boldsymbol{v}_\infty$ greater than zero. A space vehicle is ordinarily launched into an interplanetary trajectory from a parking orbit. The periapsis radius is given by

$$r_p = \frac{h^2}{\mu_1}\frac{1}{1+e}, \tag{3.28}$$

where h is the angular momentum of the departure hyperbola (relative to the planet), e is the eccentricity of the hyperbola, and μ_1 is the departure planet's gravitational parameter.

The angular momentum is given by:

$$h = \frac{\mu_1\sqrt{e^2-1}}{v_\infty}. \tag{3.29}$$

Substituting Equation (3.29) into Equation (3.28) and solving for the eccentricity yields

$$e = 1 + \frac{r_p v_\infty^2}{\mu_1}. \tag{3.30}$$

Substituting this result back into Equation (3.28) gives the following expression for the specific angular momentum:

$$h = r_p\sqrt{v_\infty^2 + \frac{2\mu_1}{r_p}}. \tag{3.31}$$

Since the hyperbolic excess speed is generally specified by the mission requirements (Equation (3.16)), choosing a departure periapsis r_p yields the parameters e and h of the departure hyperbola. From the angular momentum, the periapsis speed is obtained,

$$v_p = \frac{h}{r_p} = \sqrt{v_\infty^2 + \frac{2\mu_1}{r_p}}. \tag{3.32}$$

The location of the periapsis is given by:

$$\beta = \cos^{-1}\left(\frac{1}{e}\right) = \cos^{-1}\left(\frac{1}{1+\dfrac{r_p v_\infty^2}{\mu_1}}\right). \tag{3.33}$$

The angle β characterises the orientation of the hyperbola's apsidal line relative to the outgoing asymptote, whose direction is associated with the hyperbolic excess velocity vector $\boldsymbol{v}_\infty$.

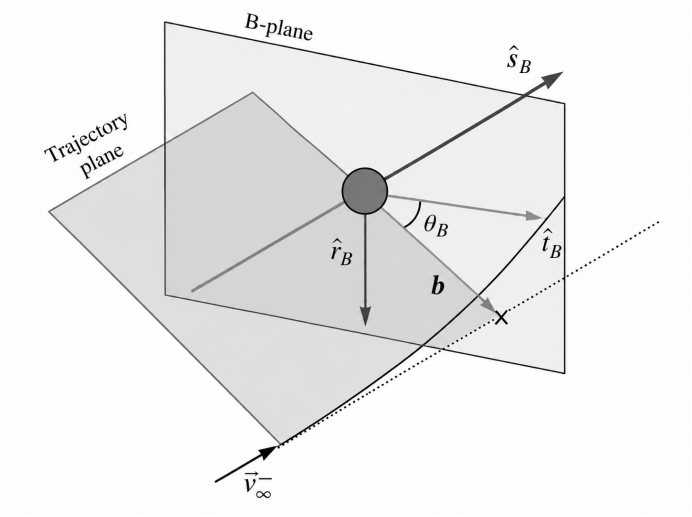


Figure 3.5: Geometrical representation of $\mathcal{F}_B$.

It should be pointed out that the only requirement on the orientation of the plane of the departure hyperbola is that it contains the centre of mass of the planet as well as the relative velocity vector $\boldsymbol{v}_\infty$. Therefore, as shown in Figure 3.4, the hyperbola can be rotated about a line A–A which passes through the planet's centre of mass and is parallel to $\boldsymbol{v}_\infty$. Rotating the hyperbola in this way sweeps out a surface of revolution on which lie all possible departure hyperbolas. The periapses of these hyperbolas trace out a circle. If injection is assumed to occur at the periapsis of the departure hyperbola, this circle represents the locus of all geometrically admissible injection points. This circle is the base of a cone with vertex at the centre of the planet; its radius is $r_p \sin\beta$.

3.4.2. THE B-PLANE

The B-plane provides a convenient geometrical representation of a hyperbolic encounter with a celestial body. It is commonly employed for the design and targeting of planetary flybys.

The B-plane is defined as the plane passing through the centre of the body and perpendicular to the incoming hyperbolic asymptote.

A right-handed reference frame $\mathcal{F}_B$, whose basis vectors are $\left(\hat{\boldsymbol{s}}_B, \hat{\boldsymbol{t}}_B, \hat{\boldsymbol{r}}_B\right)$, is introduced to describe the B-plane. It is introduced here and not in Section 3.1 since its definition is tied to the patched-conics method. The unit vector $\hat{\boldsymbol{s}}_B$ is aligned with the incoming hyperbolic asymptote. Let $\hat{\boldsymbol{n}}_B$ denote a reference direction, generally selected as the pole direction of the encounter body. It also typically lies on the ecliptic plane of the solar system. The first axis contained in the B-plane is defined as

$$\hat{\boldsymbol{t}}_B = \frac{\hat{\boldsymbol{s}}_B \times \hat{\boldsymbol{n}}_B}{\|\hat{\boldsymbol{s}}_B \times \hat{\boldsymbol{n}}_B\|}. \tag{3.34}$$

The second B-plane axis is subsequently obtained as

$$\hat{\boldsymbol{r}}_B = \hat{\boldsymbol{s}}_B \times \hat{\boldsymbol{t}}_B. \tag{3.35}$$

The basis vectors $\hat{\boldsymbol{t}}_B$ and $\hat{\boldsymbol{r}}_B$ therefore lie in the B-plane, whereas $\hat{\boldsymbol{s}}_B$ is normal to it.

The principal quantity used to characterise the encounter is the miss-parameter vector $\boldsymbol{b}$. This vector extends from the centre of the encounter body to the point at which the incoming asymptote intersects the B-plane. It can therefore be interpreted as the position at which the spacecraft would cross the B-plane if it continued along its unperturbed incoming asymptote, without being deflected by the gravitational attraction of the body. The vector $\boldsymbol{b}$ lies both in the B-plane and in the hyperbolic trajectory plane.

The incoming-asymptote direction may then be reconstructed from the hyperbolic-orbit geometry according to

$$\hat{\boldsymbol{s}}_B = \cos\beta\, \hat{\boldsymbol{e}} + \sin\beta \left(\hat{\boldsymbol{h}} \times \hat{\boldsymbol{e}}\right). \tag{3.36}$$

The direction of the miss-parameter vector is defined as

$$\hat{\boldsymbol{b}} = \hat{\boldsymbol{s}}_B \times \hat{\boldsymbol{h}}. \tag{3.37}$$

The magnitude of the miss parameter is

$$b = |a|\sqrt{e^2 - 1}. \tag{3.38}$$

Conservation of angular momentum gives

$$bv_\infty = r_p v_p. \tag{3.39}$$

The orientation of the aim point within the B-plane is described by the angle θ_B, defined as the angle between the $\boldsymbol{b}$ vector and the $\hat{\boldsymbol{t}}_B$ basis vector:

$$\theta_B = \arccos\left(\frac{\boldsymbol{b} \cdot \hat{\boldsymbol{t}}_B}{\|\boldsymbol{b}\|}\right). \tag{3.40}$$

Modifying the B-plane aim point therefore makes it possible to target a desired flyby altitude, geographical location, or outgoing asymptotic direction.

3.5. INTERPLANETARY ENVIRONMENT

Thus far, the analysis of interplanetary transfers and planetary flybys has been conducted within a Keplerian framework. In reality, however, celestial bodies cannot always be treated as point masses, spacecraft possess finite mass and dimensions, planetary systems comprise multiple gravitating bodies, and gravitational fields are neither perfectly spherical nor the sole sources of acceleration. Any effect that causes the actual trajectory to depart from its Keplerian reference is referred to as a perturbation. Once such effects are included, closed-form analytical solutions are generally no longer available, and the equations of motion must be propagated numerically. This section therefore provides an overview of the perturbations most relevant to the modelling of an interplanetary mission.

3.5.1. GRAVITY FIELD

Gravitational field modelling is the foundation for analysing orbital motion, particularly for interplanetary trajectories. The irregular shape and non-homogeneous mass distribution of small bodies result in a highly irregular gravitational field, characterised by rapid variations in intensity and direction which lead to complex, nonlinear dynamics in their proximity. Typically, the gravitational field is described through the gravitational potential:

$$\mathcal{U}(\boldsymbol{r}) = G\int_A \frac{\mathrm{d}m(\boldsymbol{\rho})}{|\boldsymbol{r} - \boldsymbol{\rho}|} \tag{3.41}$$

where G is the gravitational constant, $\boldsymbol{r}$ is the position vector of the spacecraft, $\boldsymbol{\rho}$ is the position vector of the infinitesimal mass element $\mathrm{d}m$, and A is the collection of all mass elements. Gravitational field modelling can be performed using three main approaches, which exhibit different degrees of accuracy and complexity: point mass(es), series approximation, or three-dimensional approximation. Series approximation methods model the gravitational potential energy by an infinite series of specific functions, while three-dimensional methods, mainly used for asteroids, approximate the geometry of the body as a simple triaxial ellipsoid or as a complex polyhedron (Cui and Qiao, 2014). In the current study, only the first two approaches will be explored, as they yield a degree of accuracy which is largely sufficient.

Point mass

The simplest formulation that can be used to model the celestial body's gravity field is that of a point mass. Assuming a spherical distribution of mass M and constant density, the gradient of Equation (3.41) becomes:

$$\boldsymbol{a}_g = \nabla\mathcal{U}(\boldsymbol{r}) = -\frac{GM}{\|\boldsymbol{r}\|^3}\boldsymbol{r} = -\frac{\mu}{\|\boldsymbol{r}\|^3}\boldsymbol{r} \tag{3.42}$$

where μ is the gravitational parameter of the body. Despite its attractive simplicity, the point-mass formulation does not consider latitudinal or longitudinal variations in the body's mass distribution. To capture such variations, the gravitational potential can be expanded in a series, as illustrated in the following paragraph.

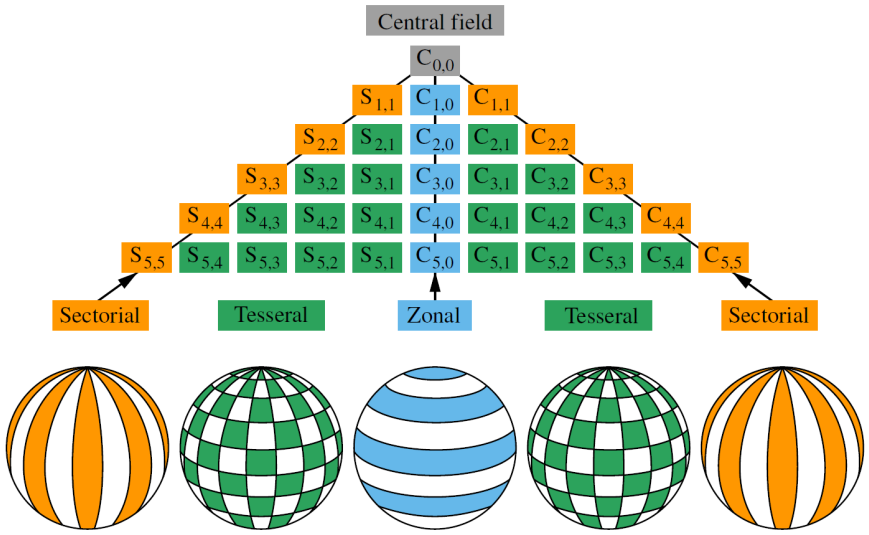


Figure 3.6: Graphical representation of spherical-harmonics coefficients and their relation to mass offsets in the (spherical) body of interest (Mooij, 2024).

Spherical harmonic expansion

The gravitational potential in Equation (3.41) satisfies Laplace's equation $\nabla^2 \mathcal{U} = 0$ outside of the body. This can be exploited by constructing a set of orthogonal solutions to Laplace's equation and choosing the coefficients of these expansions to match the actual potential function. Multiple harmonic series models exist, each obtained by adopting a particular coordinate system to solve Laplace's equation through separation of variables. This methodology identifies a region of convergence for the corresponding series where the models are subject only to omission errors representing the residual field variations not accounted for by the finite-degree expansions. Outside these convergence regions, the models are susceptible to amplification of omission errors and possible divergence effects.

The most common series is known as the *spherical harmonic expansion* (SHE), which is obtained by solving Laplace's equation through separation of variables in terms of spherical coordinates. The general form for the spherical harmonic potential of a gravity field is then:

$$\mathcal{U}(r,\phi,\lambda) = \frac{\mu}{r}\left(1 + \sum_{n=1}^{\infty}\sum_{m=0}^{n}\left(\frac{R_e}{r}\right)^n P_{nm}(\sin\phi)\left[C_{nm}\cos(m\lambda) + S_{nm}\sin(m\lambda)\right]\right) \tag{3.43}$$

where R_e is a reference radius of the body, P_{nm} are the associated Legendre functions of the first kind, and C_{nm} and S_{nm} are referred to as gravity field harmonic coefficients of degree n and order m.

Each combination of n and m represents a distinct kind of gravity field deviation from a perfectly spherical one. For example, when the order m is set to zero, only the coefficients C_{n0} remain. These are referred to as *zonal* terms, as they represent the variations of the gravity field with the latitude, as indicated in Figure 3.6. Among them, the most relevant is usually C_{20}, which represents the gravitational disturbance caused by the oblateness of a body. The remaining terms, which are not longitudinally symmetric, are referred to as *tesseral*, if $n \neq m$, or *sectorial*, in the opposite case. In particular, second-order tesseral coefficients S_{21} and C_{21} vanish if the body rotation axis is aligned with its principal inertia axis. It is also noted that the terms of degree 1 (C_{10}, C_{11}, S_{11}) describe the location of the centre of mass, hence they are also zero when this is assumed to be located at the origin of the reference frame. A slightly different formulation than Equation (3.43) can be used to reduce numerical errors, since the conventional gravitational coefficients tend to become very small when n, m become large (Melman et al., 2009). This formulation replaces C_{nm}, S_{nm}, and P_{nm} with normalised coefficients and normalised Legendre polynomials, as defined by Kaula (1966):

$$C_{nm} = \bar{C}_{nm}\bar{\mathcal{N}}_{nm}, \qquad S_{nm} = \bar{S}_{nm}\bar{\mathcal{N}}_{nm}, \qquad P_{nm} = \frac{\bar{P}_{nm}}{\bar{\mathcal{N}}_{nm}} \tag{3.44}$$

$$\bar{\mathcal{N}}_{nm} = \sqrt{\frac{(2-\delta_{0m})\,(2n+1)\,(n-m)!}{(n+m)!}} \tag{3.45}$$

where δ_{0m} is the Kronecker delta. The Cartesian components of the acceleration in the body-fixed frame can then be computed based on the following equations:

$$a_{gx} = \left(\frac{1}{r}\frac{\partial \mathcal{U}}{\partial r} - \frac{z}{r^2\sqrt{x^2+y^2}}\frac{\partial \mathcal{U}}{\partial \phi}\right)x - \left(\frac{1}{x^2+y^2}\frac{\partial \mathcal{U}}{\partial \lambda}\right)y \tag{3.46}$$

$$a_{gy} = \left(\frac{1}{r}\frac{\partial \mathcal{U}}{\partial r} - \frac{z}{r^2\sqrt{x^2+y^2}}\frac{\partial \mathcal{U}}{\partial \phi}\right)y + \left(\frac{1}{x^2+y^2}\frac{\partial \mathcal{U}}{\partial \lambda}\right)x \tag{3.47}$$

$$a_{gz} = \frac{z}{r}\frac{\partial \mathcal{U}}{\partial r} + \frac{\sqrt{x^2+y^2}}{r^2}\frac{\partial \mathcal{U}}{\partial \phi} \tag{3.48}$$

$$\frac{\partial \mathcal{U}}{\partial r} = -\frac{\mu}{r^2}\left[1 + \sum_{n=1}^{\infty}\sum_{m=0}^{n}\left(\frac{R_e}{r}\right)^n (n+1)P_{nm}(\sin\phi)\Big(C_{nm}\cos(m\lambda) + S_{nm}\sin(m\lambda)\Big)\right] \tag{3.49}$$

$$\frac{\partial \mathcal{U}}{\partial \lambda} = \frac{\mu}{r}\sum_{n=1}^{\infty}\sum_{m=0}^{n}\left(\frac{R_e}{r}\right)^n mP_{nm}(\sin\phi)\Big(S_{nm}\cos(m\lambda) - C_{nm}\sin(m\lambda)\Big) \tag{3.50}$$

$$\begin{aligned}\frac{\partial \mathcal{U}}{\partial \phi} = \frac{\mu}{r}\sum_{n=1}^{\infty}\sum_{m=0}^{n}\left(\frac{R_e}{r}\right)^n &\Big[P_{n,m+1}(\sin\phi) - m\tan\phi\, P_{nm}(\sin\phi)\Big] \\ &\times \Big(C_{nm}\cos(m\lambda) + S_{nm}\sin(m\lambda)\Big)\end{aligned} \tag{3.51}$$

Spherical harmonics can theoretically represent any potential field, even for highly non-spherical bodies, provided the series is sufficiently long. In practice, the series is truncated to a given degree and order (D/O) to balance accuracy and complexity. For planetary bodies, whose shape is generally quite similar to a sphere, high accuracy can be achieved with relatively few terms. However, for highly irregular bodies such as asteroids, the mismatch between the mass-enclosing reference sphere (the Brillouin sphere) and the body's actual shape causes the series to diverge within a significant region of *flyable space*, making this formulation unsuitable for landing scenarios.

Alternative formulations address these limitations by using other basis functions to construct a solution to the Laplace equation, such as the *ellipsoidal harmonic expansion* (EHE) and the *spheroidal harmonic expansion*, but these formulations will not be explored in the current work, since they introduce a significant computational burden without appreciably increasing the fidelity of the mission sub-phases which are explored in this thesis. As a final remark, it is important to note that the accelerations derived in Equations (3.46)–(3.51) are calculated in the body-fixed frame of the celestial body exerting the gravitational acceleration under analysis.

Third-body effects

The models described previously hold for propagation with respect to an inertial origin (e.g. the SSB). In some situations, propagations are not done relative to an inertial origin, but rather with respect to some central body. Here, two different cases are distinguished:

(i) The computation of $(\boldsymbol{a}_{BA})_B$: acceleration due to body B, acting on body A, with the frame origin at body B's centre of mass: the acceleration due to the central body itself;

(ii) The computation of $(\boldsymbol{a}_{BA})_C$: acceleration due to body B, acting on body A, with the frame origin at some other body C: the so-called third-body perturbation.

For the first case, we have:

$$(\boldsymbol{a}_{BA})_B = \boldsymbol{a}_{BA} - \boldsymbol{a}_{AB} \tag{3.52}$$

The second term on the right-hand side of Equation (3.52) takes into account the fact that the body A may also exert a gravitational acceleration on body B. This term is negligible for the cases where $\mu_A/\mu_B \to 0$, for instance when body A is a spacecraft and body B is a celestial body, as is always the case in this study. Therefore, from here onwards, we have:

$$(\boldsymbol{a}_{BA})_B \to \boldsymbol{a}_{BA} \tag{3.53}$$

For case (ii), the third-body perturbation, we have:

$$(\boldsymbol{a}_{BA})_C = \boldsymbol{a}_{BA} - \boldsymbol{a}_{BC} \tag{3.54}$$

which, for interacting point masses, reduces to the following:

$$(\boldsymbol{a}_{BA})_C = \mu_B \left(-\frac{\hat{\boldsymbol{r}}_{BA}}{\|\boldsymbol{r}_{BA}\|^2} + \frac{\hat{\boldsymbol{r}}_{BC}}{\|\boldsymbol{r}_{BC}\|^2} \right) \tag{3.55}$$

Although less familiar in typical applications, Equations (3.52) and (3.54) apply equally to point-mass and extended-body accelerations. For example, they can be used directly to compute a third-body spherical-harmonic acceleration, which will not be explored in this work as it has a negligible effect on interplanetary trajectory design.

3.5.2. SOLAR RADIATION PRESSURE

Solar radiation pressure (SRP) arises from photon momentum exchange when electromagnetic radiation impacts a surface with a non-zero projected area in the emission direction. Various methods exist to model SRP, ranging from macro models, such as those used in the Gravity Recovery and Climate Experiment (GRACE) mission (Cheng et al., 2008), which still rely on analytical formulations, to finite element models for the Global Positioning System (GPS) (Ziebart, 2004), which employ Monte Carlo ray-tracing to simulate photon paths. As detailed SRP modelling lies beyond the scope of this work, a simplified first-order approximation, known as the *cannonball model*, is adopted to qualitatively capture the destabilising effects of SRP.

The cannonball model assumes that the spacecraft is a perfectly spherically symmetric object with uniform optical properties, such that incident radiation is absorbed or specularly reflected uniformly over its surface. The radiation source is assumed to be sufficiently distant relative to the spacecraft dimensions that the incident rays can be considered parallel. Consequently, the effective cross-sectional area is independent of the spacecraft orientation. According to this model, the acceleration due to SRP does not depend on the spacecraft attitude and can be computed as

$$\boldsymbol{a}_{SRP} = \nu \frac{C_r A}{m} \frac{\psi_0}{c} \frac{1\,\mathrm{au}^2}{\|\boldsymbol{r} - \boldsymbol{r}_\odot\|^2} \frac{\boldsymbol{r} - \boldsymbol{r}_\odot}{\|\boldsymbol{r} - \boldsymbol{r}_\odot\|} = \nu C_{SRP} \frac{\boldsymbol{r} - \boldsymbol{r}_\odot}{\|\boldsymbol{r} - \boldsymbol{r}_\odot\|^3}. \tag{3.56}$$

Here, $\psi_0 = 1367\ \mathrm{W\,m^{-2}}$ is the reference solar irradiance at $1\ \mathrm{au} = 1.495978707 \times 10^{11}\ \mathrm{m}$, A/m is the spacecraft area-to-mass ratio, C_r is the dimensionless radiation-pressure coefficient, and ν is the shadow factor. The shadow factor can be determined by the eclipse conditions as follows:

$$\nu = \begin{cases} 0, & \text{umbra (total eclipse)} \\ 1, & \text{sunlight} \\ 0 < \nu < 1, & \text{penumbra (partial eclipse), annular eclipse} \end{cases} \tag{3.57}$$

Eclipse conditions can be determined by the dual-cone shadow model. This model assumes a finite size and distance of the Sun, creating the penumbra and umbra regions in the occulting body's shadow. In this work, the shadow factor ν is assumed to remain equal to unity throughout the propagation. Since the trajectory is propagated from the Earth sphere of influence to the encounter with asteroid Psyche and is predominantly heliocentric, any planetary occultations are expected to be of short duration. The resulting omission of solar-radiation-pressure acceleration is therefore considered negligible relative to the physical requirement specified in Section 2.5.3.

3.6. EQUATIONS OF MOTION

To account for uncertainty and to describe the statistical evolution of the system state, the dynamics can be formulated as a *stochastic differential equation* (SDE). A common representation, widely adopted in the literature, is based on Itô calculus (Øksendal, 2003), which extends the deterministic model by adding a stochastic term that captures process noise and unmodelled effects. This stochastic augmentation leads to a system response that evolves as a stochastic process governed by probabilistic laws, rather than a single deterministic trajectory.

Accordingly, the stochastic system dynamics are expressed as

$$d\boldsymbol{x}(t) = \boldsymbol{f}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\, dt + \boldsymbol{g}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\, d\boldsymbol{w}(t), \tag{3.58}$$

where $\boldsymbol{g} : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \times \mathbb{R} \to \mathbb{R}^{n_x \times n_w}$ is the *diffusion matrix*, which maps instantaneous random perturbations, $d\boldsymbol{w}(t)$, into the state space (Øksendal, 2003). Through its coupling with the Wiener process, it directly determines how uncertainty enters each state component.

Here, $\boldsymbol{w}(t) \in \mathbb{R}^{n_w}$ is the process-noise vector composed of independent *Wiener processes* (or standard *Brownian motions*). It can be written explicitly as

$$\boldsymbol{w}(t) = \big[w_1(t), w_2(t), \ldots, w_{n_w}(t)\big]^\top.$$

A Wiener process represents the continuous-time integral of Gaussian white noise, offering a rigorous and tractable framework for modelling stochastic disturbances in dynamical systems. Unlike white noise, which is an idealised, distribution-valued process without a continuous realisation, the Wiener process has continuous trajectories and normally distributed increments, yet its sample paths are nowhere differentiable. It can be interpreted as a continuous but irregular random walk, accumulating random fluctuations over time.

For any $t \geq 0$, the continuous-time Wiener process vector, $\boldsymbol{w}(t)$, satisfies the following properties:

- *Initial condition:* The process starts from the origin with probability one, $\boldsymbol{w}(0) = \boldsymbol{0}$.
- *Independent increments:* The increments of $\boldsymbol{w}(t)$ over non-overlapping time intervals are statistically independent, implying that the future evolution of the process is independent of its past given its current state. That is, for any $0 \leq t_1 < t_2 < t_3 < t_4$, the random vectors $\big[\boldsymbol{w}(t_2) - \boldsymbol{w}(t_1)\big]$ and $\big[\boldsymbol{w}(t_4) - \boldsymbol{w}(t_3)\big]$ are independent.
- *Stationary increments:* The distribution of each increment depends only on the duration of the interval, not on its absolute position in time. Specifically, for any $\Delta t > 0$,

  $$\big[\boldsymbol{w}(t + \Delta t) - \boldsymbol{w}(t)\big] \equiv \big[\boldsymbol{w}(\Delta t) - \boldsymbol{w}(0)\big],$$

  where $\equiv$ denotes equality in distribution.
- *Gaussian increments:* Each increment of the process is multivariate normally distributed with zero mean and covariance proportional to the interval length

  $$\big[\boldsymbol{w}(t + \Delta t) - \boldsymbol{w}(t)\big] \sim \mathcal{N}\big(0, \Delta t\, \mathbb{I}_{n_w}\big),$$

  where $\mathbb{I}_{n_w}$ denotes the $n_w \times n_w$ identity matrix.
- *Cumulative behaviour:* The uncertainty of the process grows linearly with time as independent random fluctuations accumulate. Consequently, the second moment of $\boldsymbol{w}(t)$ increases proportionally to the elapsed time,

  $$\mathbb{E}\left[\boldsymbol{w}(t)\boldsymbol{w}^\top(t)\right] = t\, \mathbb{I}_{n_w},$$

  while the expected value remains zero, $\mathbb{E}[\boldsymbol{w}(t)] = 0$, reflecting the absence of deterministic drift.

The deterministic part of the translational equations of motion describes the dynamics of the centre of mass of the spacecraft in the ICRF/EME2000 frame according to Newton's second law. Together with the mass dynamics, it can be expressed as a single first-order vector differential equation by defining the drift function as:

$$\boldsymbol{f}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big) = \begin{bmatrix} \boldsymbol{v}(t) \\ \dfrac{\sum \boldsymbol{F}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)}{m(t)} \\ -\dfrac{\big\|\boldsymbol{T}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\big\|}{I_{sp} g_0} \end{bmatrix} \tag{3.59}$$

where $\boldsymbol{u}(t)$ denotes the control input, $\boldsymbol{f}$ represents the deterministic dynamics of the system, $\|\boldsymbol{T}\|$ is the norm of the thrust vector, and g_0 is the standard gravitational acceleration at the Earth's surface. Specifically, the deterministic part of the velocity temporal derivative in Equation (3.59) can be explicitly formulated as:

$$\frac{\sum \boldsymbol{F}}{m} = \boldsymbol{a}_{g,pm}^{ICRF} + \frac{\sum \boldsymbol{F}_{pert}}{m} = \boldsymbol{a}_{g,pm}^{ICRF} + \boldsymbol{C}_{ICRF/BF}\, \boldsymbol{a}_{g,ns}^{BF} + \boldsymbol{a}_{srp}^{ICRF} + \boldsymbol{a}_{T}^{ICRF} \tag{3.60}$$

where the subscripts pm and ns have been used to indicate the point-mass and the non-spherical components of the overall gravitational acceleration, and $\boldsymbol{a}_T$ is the thrust acceleration. The superscript indicates the frame in which the acceleration is expressed.

The translational-motion part of Equation (3.58) is a stochastic differential equation based on propagating Cartesian elements, but it is just one possible representation of the dynamics. When it comes to choosing a propagation scheme, many options are available, such as Cartesian elements (Cowell and Encke propagators), Kepler elements (Gauss planetary equations), modified equinoctial elements (MEEs), and the unified state model (Vittaldev et al., 2012). It has been decided to employ the simplest formulation (Cowell) since it is generally argued that in the presence of highly perturbed motion, the other methods lose their advantages, while Cowell's simplicity and straightforward implementation make it more competitive. Nevertheless, it is acknowledged that the principal limitations of this method arise from the large magnitudes and rapid variations of the state derivatives, which may introduce significant numerical errors and complicate adaptive time-step selection. To partially mitigate these issues, a detailed time-step sensitivity analysis will be performed to identify a discretisation that balances computational efficiency and numerical accuracy, therefore remaining sufficiently small to resolve the highest-frequency dynamics of the system.

4

PARALLEL COMPUTING

4.1. HISTORY AND MOTIVATION

Historically, improvements in CPU performance were achieved primarily by increasing processor clock frequency. By the mid-2000s, however, further increases became increasingly constrained by power consumption, heat dissipation, and transistor-scaling limitations. Processor development therefore shifted towards multi-core architectures, in which several processing units are integrated within a single chip and execute tasks concurrently.

Parallel computing is a form of computation in which multiple operations are performed simultaneously. It is based on decomposing a large problem into smaller tasks that can be executed concurrently. Efficient parallel computing requires both suitable hardware and appropriate software. Computer architecture provides the resources needed for concurrent thread execution, while parallel programming organises the workload to exploit those resources effectively. Hardware and software design are therefore closely interdependent (Cheng et al., 2014).

4.2. FORMS OF PARALLELISM

Parallelism can be exploited at different levels of a computing system; the principal forms considered here are bit-level, instruction-level, task, and data parallelism (Cheng et al., 2014).

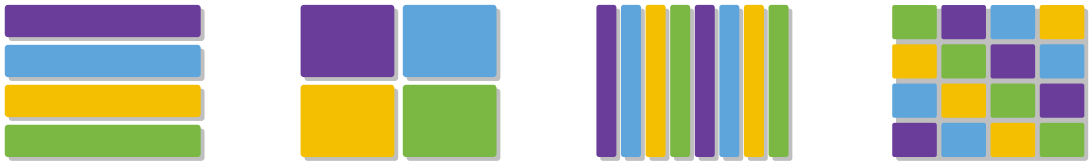
Bit-level parallelism refers to the ability of a processor to operate on wider binary words within a single instruction. Increasing the processor word size can reduce the number of instructions required for operations involving large numerical values or memory addresses. Whereas early microprocessors commonly used 8-bit words, contemporary general-purpose processors typically implement 64-bit instruction-set architectures. Nevertheless, word size alone does not determine the overall computational performance of a processor.

Instruction-level parallelism concerns the concurrent execution of independent instructions from a single instruction stream. Modern high-performance CPUs use dependency analysis and branch prediction to identify instructions that can be executed before preceding instructions have completed, while preserving the behaviour specified by the program.

GPUs adopt a different architectural balance. Rather than relying primarily on sophisticated control logic to maximise the performance of a single instruction stream, they prioritise aggregate throughput by providing many execution units and supporting a large number of concurrent threads. GPUs hide instruction and memory latency by scheduling other available warps whenever one warp is unable to proceed. A warp is a group of 32 threads that are scheduled together.

Task parallelism distributes distinct functions or computational tasks across multiple processing elements. The concurrent tasks may execute different instruction sequences and may operate on either the same data or separate datasets. The achievable degree of parallelism depends on how the original problem is decomposed and on the dependencies between its constituent tasks.

Data parallelism applies the same or similar operations concurrently to different elements of a dataset. The data are divided into smaller partitions, which are subsequently mapped to parallel threads or groups of threads.



Block partitioning in one dimension | *Block partitioning in both dimensions* | *Cyclic partitioning in one dimension* | *Cyclic partitioning in both dimensions*

Figure 4.1: Different partitioning techniques in data parallelism.

A simple example is element-wise array addition, for which separate array elements can be processed independently.

Common data-distribution strategies include block, cyclic, and block-cyclic partitioning, as illustrated in Figure 4.1. Partitioning may also be performed in multiple dimensions, making it suitable for matrices, structured computational grids, images, and other multidimensional datasets. Data parallelism is particularly well suited to GPU execution when the workload contains many independent data elements and exhibits regular control-flow and memory-access patterns.

4.3. Flynn's Taxonomy

Flynn's taxonomy classifies computer architectures according to the number of instruction and data streams that can be processed concurrently. It defines four principal categories (Cheng et al., 2014):

Single Instruction, Single Data (SISD) describes an architecture in which one instruction stream operates on one data stream. A conventional single-core processor executing a sequential program is the simplest example.

Single Instruction, Multiple Data (SIMD) describes the concurrent application of one instruction stream to multiple data elements. SIMD execution is used by vector processors and by the vector instruction sets of modern CPUs, making it particularly suitable for regular data-parallel operations.

Multiple Instruction, Single Data (MISD) describes an architecture in which multiple instruction streams operate on the same data stream. This category is uncommon in general-purpose computing, although related arrangements may be used in specialised fault-tolerant or redundant-processing systems.

Multiple Instruction, Multiple Data (MIMD) describes an architecture in which multiple processing elements execute independent instruction streams on different data.

Modern GPUs cannot be described completely by a single category because different forms of parallelism occur at different architectural levels. The phrase Single Instruction, Multiple Threads (SIMT) was coined by NVIDIA Corporation (2026) and is usually mentioned as an addition to Flynn's taxonomy.

SIMT is frequently compared with SIMD parallelism, but there are significant differences. In SIMD, a single control-flow path is followed by a fixed-width vector of data, whereas SIMT assigns a logical control flow to each thread. Different SIMT threads may therefore follow different branches, even though this divergence can reduce execution efficiency.

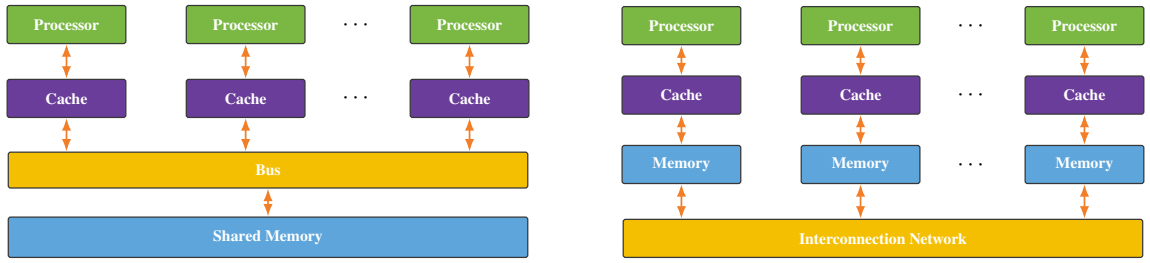
4.4. Memory Organisation Models

Computer architectures can be classified according to how processing elements access memory and exchange data. The three principal models are shared memory, distributed memory, and hybrid memory (Cheng et al., 2014). The first two are compared schematically in Figure 4.2.

4.4.1. Shared-Memory Systems

In a shared-memory system, multiple threads or processes access a common address space. Although sharing memory implies a shared address space, it does not necessarily mean that there is a single physical memory. Synchronisation mechanisms are required when concurrent accesses could conflict and lead to race conditions. POSIX Threads and OpenMP are widely used programming interfaces for shared-memory parallelism.

Multi-core (CPU) and many-core (GPU) architectures directly support this model; parallel-programming languages and libraries therefore often exploit it. OpenCL and CUDA are the most popular APIs for GPU parallelism. Since CUDA is the one used in this work, it is described in more detail in Section 4.9.



(a) Shared-memory organisation.

(b) Distributed-memory organisation.

Figure 4.2: Comparison between shared- and distributed-memory organisations. Adapted from (Cheng et al., 2014).

4.4.2. Distributed-memory systems

In a distributed-memory system, each process has its own address space. Processes exchange data explicitly through an interconnection network using message-passing operations. The Message Passing Interface (MPI) provides a portable standard for point-to-point and collective communication and is widely used in computing clusters and supercomputers. From a programming perspective, distributed-memory applications typically rely on message-passing libraries whose communication routines are called explicitly within the source code. The programmer is therefore responsible for decomposing the problem, assigning work to processes, and defining the required communication between them.

4.4.3. Hybrid-memory systems

Hybrid systems combine shared- and distributed-memory parallelism. A common configuration uses MPI for communication between computing nodes and OpenMP for parallel execution within each node. Another common hybrid configuration combines MPI for communication between computing nodes with CUDA for parallel execution on one or more GPUs within each node.

4.5. Volatile memory technologies

Random-access memory (RAM) stores the data and instructions required during program execution. It is referred to as random access because any memory location can be accessed directly with approximately the same access time, unlike sequential-access memory. In practice, however, the effective memory access latency experienced by a program depends on the memory hierarchy (e.g., CPU caches) and the access pattern. RAM is volatile, meaning that its contents are lost when power is removed. Two important semiconductor RAM technologies are Static Random-Access Memory (SRAM) and Dynamic Random-Access Memory (DRAM) (Cheng et al., 2014).

4.5.1. Static random-access memory

SRAM stores each bit using a bistable circuit composed of several transistors. The stored state is retained while power is supplied and does not require periodic refresh operations. SRAM provides low access latency but has a relatively low storage density and high cost per bit. It is therefore primarily used for processor caches, register files, and other small on-chip memory structures.

4.5.2. Dynamic random-access memory

This type of volatile memory is usually used as the main memory of a computer. A conventional DRAM cell stores one bit using an access transistor and a capacitor. Because the capacitor gradually loses charge, the stored data must be refreshed periodically. Compared with SRAM, DRAM provides a higher storage density and lower cost per bit, but generally has a higher access latency.

DRAM is organised into banks, rows, and columns. To access data, the corresponding row is activated and loaded into a row buffer. Further accesses to the same open row are relatively fast, whereas accessing another row requires additional precharge and activation operations. The associated timing constraints are defined by memory standards such as DDR5 for system memory.

After a row has been activated, read and write commands transfer several consecutive data words in bursts. Contiguous access patterns therefore make more effective use of the open row and the transferred data, whereas irregular accesses to widely separated locations may require additional memory operations and increase latency.

4.6. PERFORMANCE METRICS

The performance of hardware and software can be assessed using several complementary metrics. The most commonly used measures in parallel computing are introduced below.

Latency is the elapsed time between the initiation and completion of an operation. It is commonly expressed in nanoseconds or microseconds, depending on the operation considered. CPUs are generally designed to minimise the latency of individual instruction streams, although the effective latency also depends on the memory hierarchy and workload.

Bandwidth is the rate at which data can be transferred through a memory system or communication interface. It is commonly expressed in $\mathrm{GB\,s^{-1}}$ and is particularly relevant to memory components and interconnects.

Throughput is the amount of computational work completed per unit time. Depending on the application, it may be expressed as operations per second, tasks per second, or floating-point operations per second (FLOP/s). GPUs are generally designed to maximise aggregate throughput.

Threading efficiency expresses, in relative terms, the extent to which the inclusion of additional threads enhances program performance, and is computed as follows (Soyata, 2018):

$$\eta = \text{Threading Efficiency} = \frac{\text{Single-Thread Execution Time}}{(N\text{-Thread Execution Time}) \times N} \tag{4.1}$$

where N denotes the number of threads employed. It is evident that, were a threading efficiency of 100% to be attained, the execution time would be reduced in direct proportion to the number of threads launched. Nevertheless, owing to inherent constraints in available hardware resources, a threading efficiency of 100% is rarely, if ever, achievable in practice.

Parallelisation overhead represents the cost incurred as a consequence of parallelisation, and is defined as follows (Soyata, 2018):

$$\text{Parallelisation Overhead} = 1 - \text{Threading Efficiency} = 1 - \eta \tag{4.2}$$

Speedup denotes the improvement in execution time achieved when a given task is executed on two distinct systems addressing the same problem. It is calculated, in its simplest form, as the ratio of the respective execution times, although more elaborate definitions of speedup exist that additionally account for latency or throughput.

4.7. MULTI-CORE CPU ARCHITECTURE

Most modern CPUs are designed in such a way that each of their cores is capable of executing more than one thread at the same time. As a consequence of this concurrent execution, the threads have to share some of the resources, such as the cache memory within the core. In most commercially available CPUs, each core can execute two threads. The core and thread counts of a CPU are usually denoted as, e.g., 6C/12T, which stands for six cores and twelve threads. In this work, the term CPU is used to denote a multi-core, multi-thread, out-of-order processor. The architecture of a modern consumer CPU, such as the Intel Core i7 family, as well as its connection with a GPU and the main memory, can be seen in Figures 4.3 and 4.4.

As presented in these figures, the L3 cache is absent in an NVIDIA GPU; in a CPU, however, it is a crucial component, occupying a substantial fraction of the die area. The purpose of the cache memory is to store particular data so that future requests for those data can be served faster. The data stored in a cache might be the result of an earlier computation, or the duplicate of data stored elsewhere. Usually, the programmer has no influence on the way the cache memory manages its data, although the cache efficiency can be controlled to some extent through careful data-access patterns. As shown in Figure 4.3, there is no direct access path from the cores to the main memory. The memory controller takes up a significant chip area; it buffers and aggregates the data coming from the L3 cache to mitigate the inefficiencies of the block-transfer nature of the DRAM. It is also responsible for converting the data streams between the L3 cache and the DRAM into a proper format. The size of the cache line is different for each processor; a typical value is 64 bytes for modern CPUs. Another component is the chipset, a set of electronic components on one or more integrated circuits that manages the data flow between the processor, memory, and peripherals. For instance, it uses PCIe or NVLink to communicate with the GPUs. It is also responsible for queuing and efficiently transferring the data between the L3 cache and the PCIe or NVLink. To summarise, three main parts of the CPU can be distinguished: the cores, the memory, and the I/O. Accordingly, the programs that a CPU executes can be core-intensive, memory-intensive, and/or I/O-intensive.

4.8. GRAPHICS PROCESSING UNIT

During the 1980s and 1990s, it became evident that a new type of processor was needed, especially suitable for 2D and 3D graphics applications, which require a large number of floating-point operations. Graphically driven operating systems such as Microsoft Windows became extremely popular, and there was a growing need for better visualisation in the computer-gaming industry. Silicon Graphics, NVIDIA, ATI Technologies, and other companies started to produce graphics accelerators that were affordable enough to attract widespread attention, giving birth to the term GPU (Sanders and Kandrot, 2010). Originally developed as specialised processors for three-dimensional graphics, GPUs consisted largely of fixed-function hardware designed to accelerate the highly parallel operations required for real-time rendering. Over successive generations, GPUs evolved into programmable computing devices. By 2003, several stages of the graphics pipeline were programmable, allowing user-defined instructions to be executed concurrently across individual elements of a scene or image.

By the mid-2000s, new APIs had emerged that allowed program code to be created on a GPU for general purposes. The two predominant desktop GPU languages are CUDA, which is designed only for NVIDIA platforms, and OpenCL, which can be used on different GPUs and also works on Field-Programmable Gate Arrays (FPGAs) and Digital Signal Processors (DSPs). Only CUDA will be described in more detail, in Section 4.9, since it is the platform used in this thesis project: it is more developer-friendly and provides its own debugger and many other features, despite being compatible only with NVIDIA GPUs. Consequently, this study focuses exclusively on NVIDIA architectures. Unlike the simplified representation commonly used to describe a general-purpose CPU, a GPU cannot be characterised by a single invariant hardware architecture. GPU architectures have evolved substantially over the past decades, and their internal organisation may differ both between architectural generations and among chips belonging to the same architectural family. Although the physical implementations of these architectures differ considerably, the CUDA programming model provides a stable software abstraction through which they can be described and programmed. The CUDA GPU hardware model presented in Section 4.9.2 is therefore used in this work as a unified framework for relating software constructs to the underlying GPU hardware.

4.8.1. ADVANTAGES OF GPU COMPUTING

Within comparable cost and power constraints, a GPU generally provides substantially greater instruction throughput and memory bandwidth than a CPU. Consequently, computational applications capable of exploiting these characteristics can often execute considerably faster on a GPU than on a CPU. Other computing architectures, such as FPGAs, may also achieve high energy efficiency; however, they generally provide less programming flexibility than GPUs.

CPUs and GPUs are designed according to different computational objectives. A CPU is optimised to execute a serial sequence of instructions, referred to as a thread, with the lowest possible latency. Modern CPUs can execute several tens of threads concurrently, but their architecture remains primarily focused on achieving high single-thread performance. In contrast, a GPU is designed to execute thousands of threads simultaneously. This architecture accepts lower performance for each individual thread in exchange for substantially higher aggregate computational throughput.

GPUs are therefore particularly suitable for workloads containing a high degree of data parallelism. A comparatively large proportion of their transistor budget is allocated to data-processing units, whereas CPUs devote a larger share of their transistors to cache memory and instruction-flow control. Figure 4.3 illustrates a representative comparison of how chip resources are distributed within CPU and GPU architectures.

In GPUs, threads are organised into groups known as warps. Since control logic is shared among the threads within each warp, a greater proportion of the available chip area can be allocated to data-processing units.

4.9. CUDA PROGRAMMING MODEL

As CUDA has been selected as the parallel-computing platform for this work, this section introduces its programming model at a high level, independently of any specific programming language.

4.9.1. HETEROGENEOUS SYSTEMS

The CUDA programming model is based on a heterogeneous computing system comprising both CPUs and GPUs. The CPU, together with the memory directly accessible to it, is referred to as the *host* and *host memory*, respectively. Similarly, the GPU and its directly connected memory are referred to as the *device* and *device*

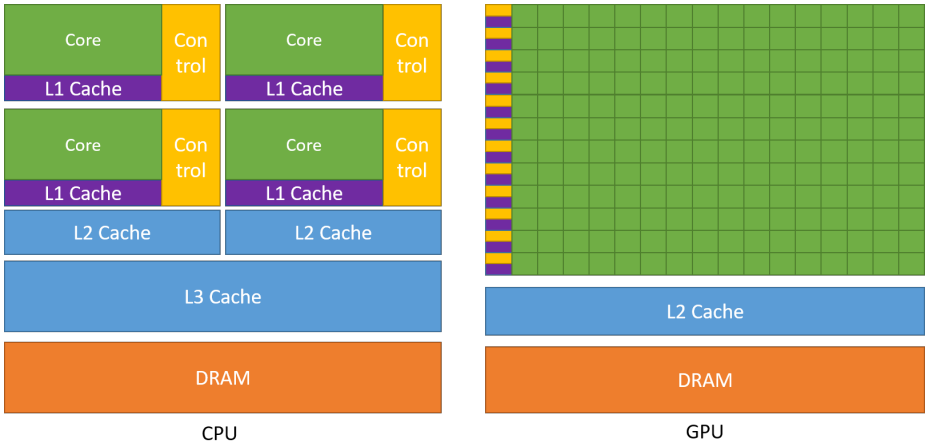


Figure 4.3: Conceptual comparison of CPU and GPU architectures (NVIDIA Corporation, 2026).

memory. In certain system-on-chip (SoC) architectures, the CPU and GPU may be integrated within a single physical package. Larger computing systems may instead contain multiple CPUs, multiple GPUs, or both.

Although CUDA applications execute selected portions of their code on the GPU, program execution always begins on the CPU. The code executed by the CPU is known as *host code*. Through the CUDA application programming interfaces (APIs), the host code can transfer data between host and device memory, initiate the execution of code on the GPU, and wait for memory transfers or GPU computations to complete. The CPU and GPU may execute code concurrently, and high performance is generally achieved by maximising the utilisation of both processing units.

The portion of an application executed on the GPU is known as *device code*. A function invoked for execution on the GPU is historically referred to as a *kernel*, while initiating its execution is known as *launching the kernel*. A kernel launch causes numerous GPU threads to execute the kernel code in parallel. Although GPU threads are conceptually similar to CPU threads, important differences in their execution and organisation affect both computational correctness and performance (NVIDIA Corporation, 2026).

4.9.2. GPU HARDWARE MODEL

Like any programming model, CUDA is based on a conceptual representation of the underlying hardware architecture. For CUDA programming purposes, a GPU may be regarded as a collection of *Streaming Multiprocessors* (SMs), which are arranged into larger groups known as *Graphics Processing Clusters* (GPCs). Each SM contains a local register file, a unified data cache, and several functional units responsible for performing computational operations.

The unified data cache provides the physical storage resources used by both the L1 cache and shared memory. The proportion of this cache allocated to L1 cache and shared memory can be configured at run time. The capacity of the different memory structures, as well as the number and type of functional units contained within an SM, may vary between GPU architectures (NVIDIA Corporation, 2026).

4.9.2.1. COMPUTE CAPABILITY

It was discussed in Section 4.8 that NVIDIA has changed the internal architecture of its GPUs with every new generation. This architectural change at the hardware level requires significant changes in the API; NVIDIA therefore introduced the term *compute capability*. NVIDIA tries to make two improvements with every new architectural generation and compute capability: first, the introduction of a new set of instructions that perform operations that could not be performed with the previous compute capability; and second, performance improvements for all instructions introduced in previous compute-capability generations. This means that a program which was compiled with an older compute capability and runs on a newer architecture will likely perform better than on the architecture for which it was originally compiled. The general specifications and features of a compute device depend on its compute capability. Most compute features introduced with a compute architecture are intended to be available on all subsequent architectures. Beginning with devices of compute capability 9.0, specialised compute features that are introduced with an architecture may not be guaranteed to be available on all subsequent compute capabilities. These features are referred to as architecture-specific features and are designed to accelerate specialised operations, such as Tensor Core computations. They are

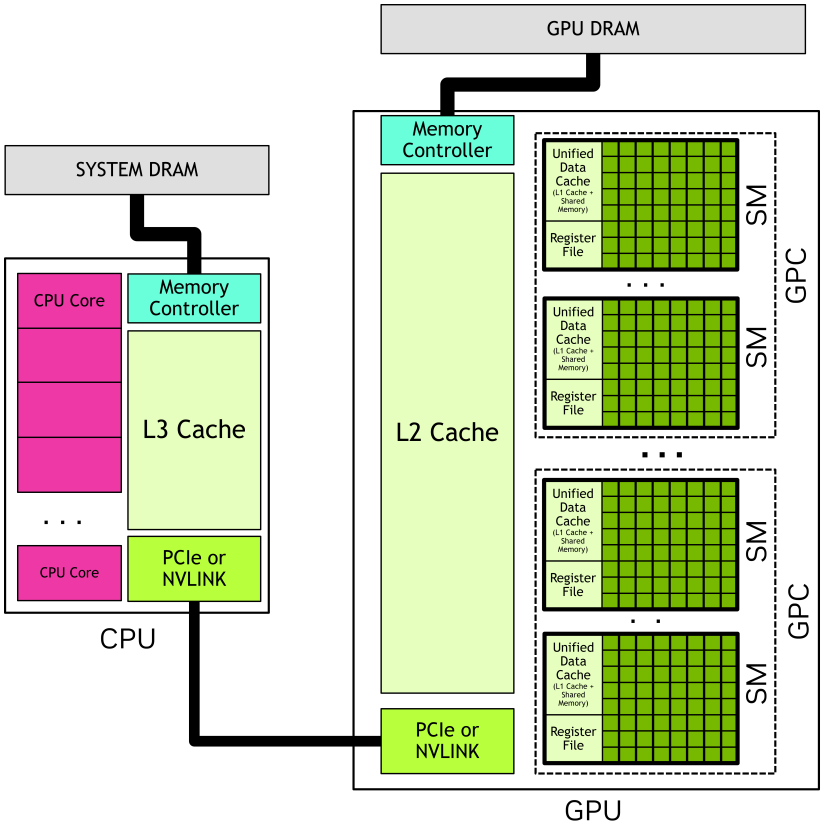


Figure 4.4: Conceptual organisation of a heterogeneous CPU–GPU computing system. Communication between the CPU and GPU is provided through an interconnect such as PCI Express or NVLink. Adapted from (NVIDIA Corporation, 2026).

not supported across all compute capabilities and may change significantly in future GPU generations. To enable them, the code must be compiled for a specific target architecture. Consequently, such code can only run on devices with the compute capability for which it was compiled. Beginning with devices of compute capability 10.0, some architecture-specific features are common to devices of more than one compute capability. The devices that contain these features are part of the same family, and these features can also be called family-specific features. Family-specific features are guaranteed to be available on all devices in the same family. A family-specific compiler target is required to enable family-specific features. Code compiled for a family-specific target can only be run on GPUs which are members of that family. The compute-capability version of a particular GPU should not be confused with the CUDA toolkit version (e.g. CUDA 13.2.1, CUDA 13.3.0), which is the version of the CUDA software platform.

4.9.2.2. THREAD BLOCKS AND GRIDS

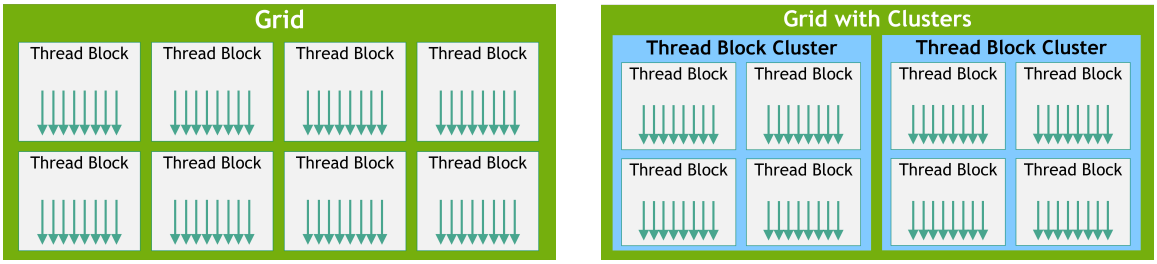
When a kernel is launched, it is executed by a large number of threads, potentially reaching several millions. These threads are arranged into groups known as *thread blocks*, which are themselves organised into a *grid*. All thread blocks belonging to the same grid have identical dimensions and contain the same number of threads, as illustrated in Figure 4.5a.

A kernel is launched using an *execution configuration*, which defines the dimensions of both the grid and the thread blocks. The execution configuration may also specify optional properties, including the stream, the SM configuration settings, and the thread-block cluster dimensions, which are described in the next sections.

Each thread can use built-in variables to determine its position within its thread block, the position of its block within the grid, and the dimensions of both the block and the grid. These quantities provide each thread with a unique logical identity across the kernel launch.

All threads belonging to a thread block are assigned to a single SM. This arrangement allows threads within the block to communicate and synchronise efficiently. In particular, all threads within a block can access the same on-chip shared-memory allocation, which may be used to exchange and reuse data.

A grid may contain millions of thread blocks, even though the executing GPU may contain only tens



(a) Conceptual representation of a grid composed of multiple thread blocks.

(b) Grid organised into thread-block clusters.

Figure 4.5: Comparison between a conventional grid of thread blocks and a grid organised into thread-block clusters (NVIDIA Corporation, 2026).

or hundreds of SMs. Each thread block is assigned to one SM and, under normal execution conditions, remains on that SM until it has completed. In uncommon situations involving features such as CUDA Dynamic Parallelism (NVIDIA Corporation, 2026), a block may instead be suspended: the state associated with its execution is saved to a system-managed region of GPU memory, allowing the SM to execute other blocks. This behaviour is comparable to CPU context switching, but it is not common during ordinary execution.

CUDA does not generally guarantee the order in which different thread blocks are scheduled. A block must therefore not depend on results produced by another block in the same grid, since the second block may not be scheduled until the first has completed. Figure 4.6a illustrates how blocks from a grid may be assigned to an SM.

The CUDA programming model allows grids of different sizes to execute on GPUs ranging from a single SM to thousands of SMs. To preserve this scalability, threads belonging to different blocks should normally have no direct data dependencies and should not require inter-block synchronisation. All threads within one block execute concurrently on the same SM, whereas different blocks may be scheduled on the available SMs in any order, either concurrently or sequentially. A correctly structured kernel must therefore remain valid regardless of the order in which its blocks are executed.

4.9.2.3. THREAD-BLOCK CLUSTERS

On GPUs with compute capability 9.0 or later, CUDA provides an additional, optional level of hierarchy known as a *thread-block cluster*. A cluster is a group of thread blocks. Introducing clusters does not alter the dimensions of the grid or the indices of the blocks within it. Instead, each block acquires an additional position within its containing cluster, as shown in Figure 4.5b.

Clusters group adjacent thread blocks and provide additional opportunities for communication and synchronisation. All blocks within a cluster are scheduled within a single GPC. Because these blocks execute simultaneously on SMs belonging to the same GPC, threads in different blocks of the cluster can communicate and synchronise through specialised CUDA software interfaces.

Threads within a cluster may also access the shared-memory allocations of the other blocks in that cluster. This combined addressable memory is known as *distributed shared memory*. The maximum number of blocks that may belong to a cluster depends on the specific GPU hardware.

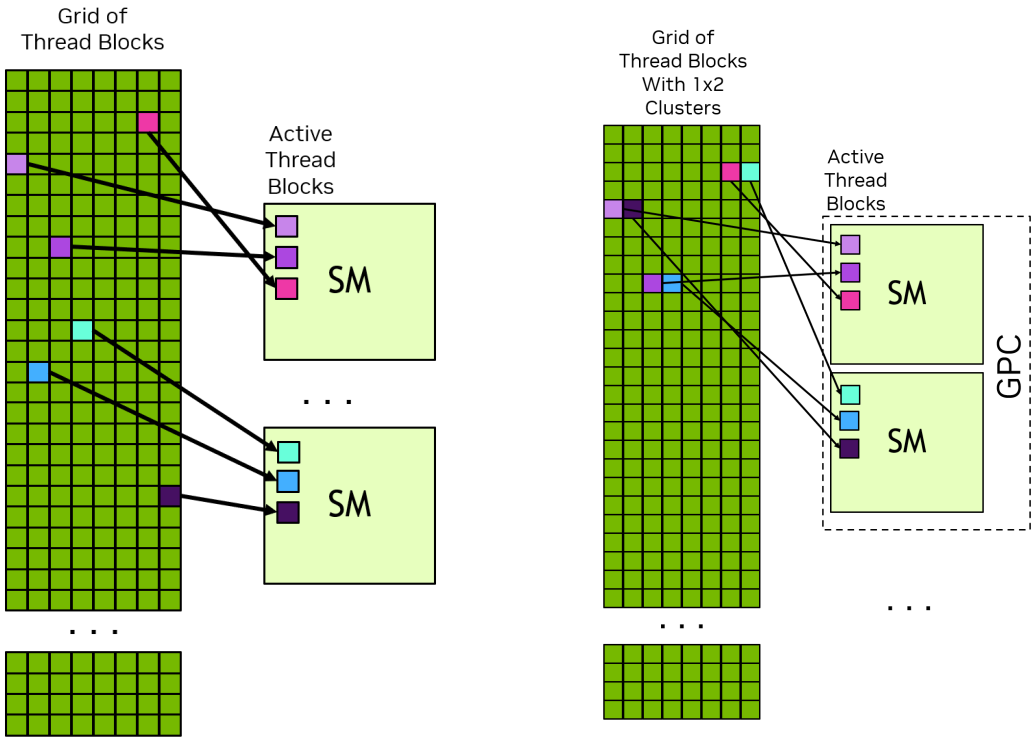
Figure 4.6b illustrates the simultaneous scheduling of the blocks of a cluster on different SMs within the same GPC. Blocks belonging to the same cluster are always adjacent within the grid.

4.9.2.4. WARPS AND SIMT EXECUTION

As previously introduced, within a thread block, threads are divided into warps. Each warp executes kernel instructions according to the SIMT model. All threads in a warp execute the same kernel function, although individual threads may follow different branches through that function.

When threads are assigned to a warp, each one occupies a *warp lane*. The lanes are numbered from 0 to 31, and threads are assigned to warps according to a predictable ordering.

At the programming-model level, all active threads in a warp execute the same instruction together. If only a subset of the threads follows a particular branch, the threads that do not follow that branch are temporarily disabled, or masked, while the relevant instructions are executed. For instance, if a condition is true for only half of the threads in a warp, the remaining half remains inactive while the selected branch is processed. When threads within a warp follow different control-flow paths, the phenomenon is known as *warp divergence*,



(a) Example assignment of thread blocks from a grid to an SM. An SM may have one or more resident thread blocks.

(b) Simultaneous scheduling of the thread blocks of a cluster on SMs of the same GPC

Figure 4.6: Comparison between conventional thread-block scheduling and scheduling with thread-block clusters (NVIDIA Corporation, 2026).

illustrated in Figure 4.7. GPU utilisation is generally highest when threads belonging to the same warp follow the same execution path. An important metric to measure warp divergence is the so-called *branch efficiency*, which is defined as follows:

$$\text{Branch Efficiency} = \frac{\#\text{Branches} - \#\text{Diverged Branches}}{\#\text{Branches}} \tag{4.3}$$

Branch efficiency is the ratio of uniform control-flow decisions over all executed branch instructions.

Under the SIMT model, the threads of a warp conceptually advance through a kernel in lockstep. The details of the physical hardware execution may differ, particularly on architectures supporting independent thread execution. Nevertheless, CUDA programs should be written according to the SIMT abstraction rather than relying on undocumented hardware scheduling behaviour. Violating the programming model may lead to undefined behaviour that differs between GPU architectures. Although explicit consideration of warps is not always required when writing correct CUDA code, understanding warp-level execution is important for

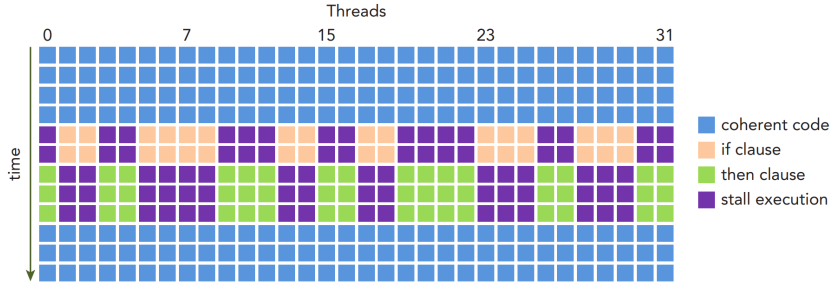


Figure 4.7: Warp divergence (Cheng et al., 2014).

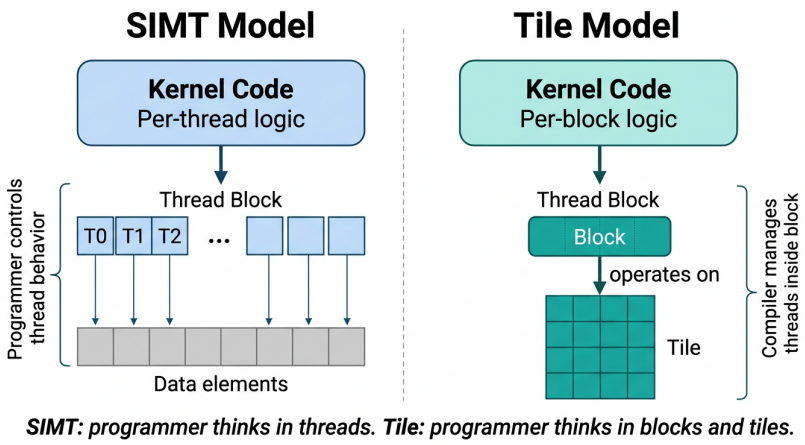


Figure 4.8: Comparison of the programmer's view in the SIMT and tile programming models (NVIDIA Corporation, 2026).

interpreting advanced programming concepts, such as the specialisation of warps within a thread block to limit thread divergence and maximise utilisation. This and other optimisations make use of the knowledge that threads are grouped into warps when executing.

Thread blocks are generally most efficient when their total number of threads is a multiple of 32. Other block sizes are legal, but if the number of threads is not a multiple of the warp size, the final warp is only partially occupied. Its unused lanes remain inactive, potentially reducing functional-unit utilisation and memory-access efficiency.

4.9.2.5. TILE PROGRAMMING

In addition to the SIMT model, CUDA supports a *tile programming* model. Under this model, the programmer writes operations at the level of an entire thread block rather than explicitly describing the behaviour of each individual thread. These operations act on multidimensional collections of values known as *tiles*, and the compiler maps the tile-level operations onto the threads of the block.

Tile kernels are launched over a grid of thread blocks. Each block executes the same tile kernel and uses its position within the grid to determine the portion of the input data for which it is responsible. The programmer specifies the grid dimensions, while the compiler determines the required number of threads per block from the tile operations contained within the kernel.

A tile kernel follows a single block-level control flow. The programmer specifies operations on tiles, and the compiler distributes the associated work across the threads of the block. Conventional control-flow constructs, including conditionals and loops, remain available. Because the block follows one control path, however, the tile model does not expose the same concept of warp divergence to the programmer.

Scalar operations, such as evaluating an index or a loop bound, are performed by a single thread within the block. Tile operations, such as element-wise addition between two tiles, are instead executed collectively and in parallel by the threads of the block.

Thread blocks and tiles must not be confused. A block is an execution unit, whereas a tile is a unit of data. A single thread block may create and manipulate multiple tiles with different shapes and element types.

Arrays and Tiles

Tile kernels operate on two principal forms of data: *arrays* and *tiles*. An array, also referred to as a global array, is a multidimensional container of elements stored in device memory. Arrays are mutable, meaning that their contents may be modified by store operations performed within a kernel. Each array has a defined shape and element type.

A tile is a multidimensional collection of values that exists only within tile code and is local to one thread block. Tiles are immutable: every operation involving a tile produces a new tile rather than modifying the existing one. A tile does not necessarily correspond to a distinct object stored in memory. Instead, the compiler determines how its values are represented and may place them in registers, shared memory, or other SM resources.

Each tile dimension must be a power of two and must be known at compile time. In other words, its dimensions must be determinable before the kernel begins execution rather than being computed dynamically

during execution. Tiles cannot be supplied as kernel arguments; they are created, manipulated, and consumed entirely within tile code.

Relationship between Tile and SIMT Programming
Tile programming and SIMT programming coexist within CUDA. An application may contain both SIMT kernels and tile kernels, and both types of kernel may operate on the same data stored in device memory. The programming model is selected independently for each kernel.

As illustrated in Figure 4.8, SIMT provides fine-grained control over individual threads and remains necessary for algorithms or optimisation methods that require explicit thread-level behaviour, whereas tile programming offers a higher-level abstraction that can simplify kernel implementation. Because thread-level decisions are delegated to the compiler, the same tile kernel may execute on different GPU architectures without requiring modifications to the source code. Both models also use the same device memory spaces, which are introduced in Section 4.9.6.

4.9.3. ASYNCHRONOUS EXECUTION

An important feature of CUDA kernel launches is that they are asynchronous, which means that control returns to the CPU immediately after the CUDA kernel is invoked. However, the programmer can synchronise the GPU and the CPU, such that the CPU idles until the kernel is finished. CUDA allows concurrent, or overlapping, execution of multiple tasks, specifically:

- computation on the host;
- computation on the device;
- memory transfers from the host to the device;
- memory transfers from the device to the host;
- memory transfers within the memory of a given device;
- memory transfers among devices.

The concurrency is expressed via an asynchronous interface, where a dispatching function call or kernel launch returns immediately. Asynchronous calls usually return before the dispatched operation has completed and may return before the asynchronous operation has started. The application is then free to perform other tasks at the same time as the originally dispatched operation. When the final results of the initially dispatched operation are needed, the application must perform some form of synchronisation to ensure that the operation in question has completed. A typical example of a concurrent execution pattern is the overlapping of host and device memory transfers with computation, thus reducing or eliminating their overhead.

In general, asynchronous interfaces typically provide three main ways to synchronise with the dispatched operation:

- **Blocking synchronisation:** the application calls a synchronisation function that prevents the calling host thread from continuing until the dispatched operation has completed. This approach is simple, but the host thread cannot perform other useful work while waiting.
- **Non-blocking synchronisation, or polling:** the application calls a status-query function that returns immediately and indicates whether the dispatched operation has completed. If the operation is still in progress, the application may continue performing other tasks and query its status again at a later time.
- **Callback-based synchronisation:** the application registers a host function in advance. The function is automatically invoked when the dispatched operation, or the preceding work in the associated execution sequence, has completed. This avoids both blocking and repeated status queries, although it introduces a less direct control flow.

While the programming interfaces are asynchronous, the actual ability to carry out various operations concurrently will depend on the version of CUDA and the compute capability of the hardware being used.

The core API components for asynchronous execution in CUDA are CUDA streams and CUDA events. The remainder of this section explains how these elements can be used to express asynchronous execution in CUDA.

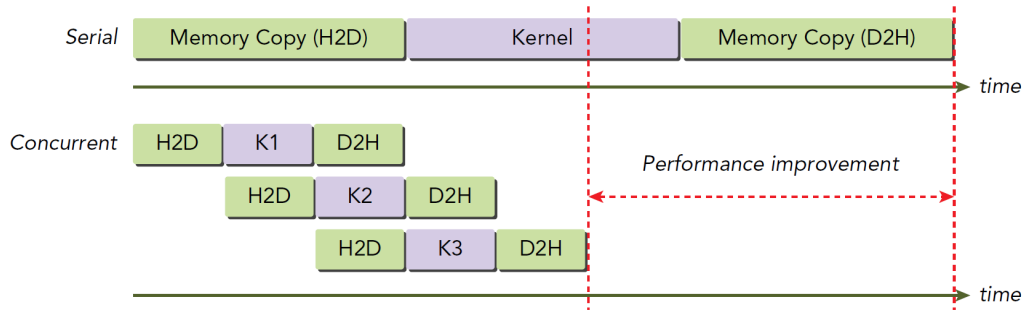


Figure 4.9: Composition of possible CUDA streams (H2D: Host to Device, D2H: Device to Host) (Cheng et al., 2014).

4.9.3.1. CUDA STREAMS

At the most basic level, a CUDA stream is an abstraction which allows the programmer to express a sequence of operations. A stream operates like a work queue into which programs can add operations, such as memory copies or kernel launches, to be executed in order. Operations at the front of the queue for a given stream are executed and then dequeued, allowing the next queued operation to come to the front and be considered for execution. The order of execution of operations in a stream is sequential, and the operations are executed in the order in which they are enqueued into the stream.

An application may use multiple streams simultaneously. In such cases, the runtime will select a task to execute from the streams that have work available, depending on the state of the GPU resources. Streams may be assigned a priority, which acts as a hint to the runtime to influence the scheduling, but does not guarantee a specific order of execution.

The API function calls and kernel launches operating in a stream are asynchronous with respect to the host thread. Applications can synchronise with a stream by waiting for it to be empty of tasks, or they can also synchronise at the device level.

One of the common use cases of streams is to improve the performance of the host-to-device and device-to-host data transfers. As mentioned before, all kernel launches are asynchronous; it is therefore possible for the host to transfer some data to the device while a kernel is running on it. Figure 4.9 illustrates a simple timeline of CUDA operations using three streams. Both data transfer and kernel computation are evenly distributed among three concurrent streams. It can be seen in the figure that the performance improvement can be rather significant when using CUDA streams.

CUDA has a default stream, and operations and kernel launches without a specific stream are queued into this default stream.

4.9.3.2. CUDA EVENTS

CUDA events are a mechanism for inserting markers into a CUDA stream. They behave like tracer particles that can be used to track the progress of tasks in a stream. Consider launching two kernels into a stream. Without such tracking events, it would only be possible to determine whether the stream is empty or not. If an operation depended on the output of the first kernel, it could not be started safely until the stream was known to be empty, by which time both kernels would have completed.

Using CUDA events, this limitation can be overcome. By enqueuing an event into a stream directly after the first kernel, but before the second kernel, it is possible to wait for this event to come to the front of the stream. The dependent operation can then be started safely, knowing that the first kernel has completed but before the second kernel has started. Used in this way, CUDA events can build up a graph of dependencies between operations and streams.

CUDA events also record timing information, which can be used to time kernel launches and memory transfers.

4.9.4. LATENCY HIDING

Latency hiding is an extremely important feature of GPGPU programming. Since every instruction has a non-zero latency, called instruction latency, it is inevitable that a warp stalls due to this latency. However, this latency can be hidden by utilising parallelism. At each instruction-issue cycle, a warp scheduler selects an eligible resident warp and issues its next instruction, thereby keeping the SM busy with useful work at all times. In other words, GPU instruction latency can be hidden by computation performed by other warps. Figure 4.10

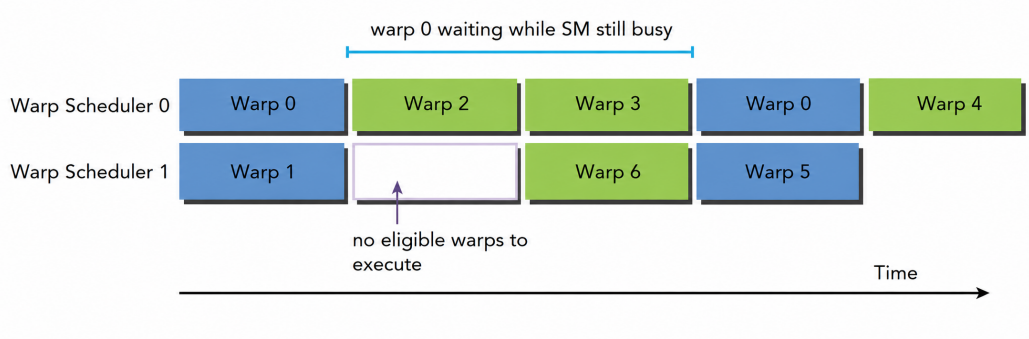


Figure 4.10: Schematic representation of latency hiding by two warp schedulers (Cheng et al., 2014).

shows how an SM hides latency by switching among ready warps. When Warp 0 stalls, for example while waiting for data, its scheduler issues instructions from Warp 2 and Warp 3 instead. Warp 0 resumes once it becomes eligible again. The blank interval indicates that the second scheduler had no ready warp to issue, although the SM remained partially active through the first scheduler.

The number of warps required to hide latency can be reasonably approximated using Little's Law. Originally a theorem in queueing theory, it can also be applied to GPUs:

$$\text{Number of Required Warps} = \text{Latency} \times \text{Throughput} \tag{4.4}$$

4.9.5. DYNAMIC PARALLELISM

CUDA Dynamic Parallelism, often abbreviated CDP, is a feature of the CUDA programming model that enables code running on the GPU to create new GPU work. That is, new work can be added to the GPU in the form of additional kernel launches from device code already running on the GPU. This feature can reduce the need to transfer execution control and data between host and device, as launch-configuration decisions can be made at run time by threads executing on the device.

Data-dependent parallel work can be generated by a kernel at run time. Before CDP was added to CUDA, some algorithms and programming patterns required modifications to eliminate recursion, irregular loop structures, or other constructs that do not fit a flat, single level of parallelism. These program structures may be expressed more naturally using CDP. This transition from repeated host-controlled kernel launches to hierarchical, device-generated parallelism is illustrated in Figure 4.11.

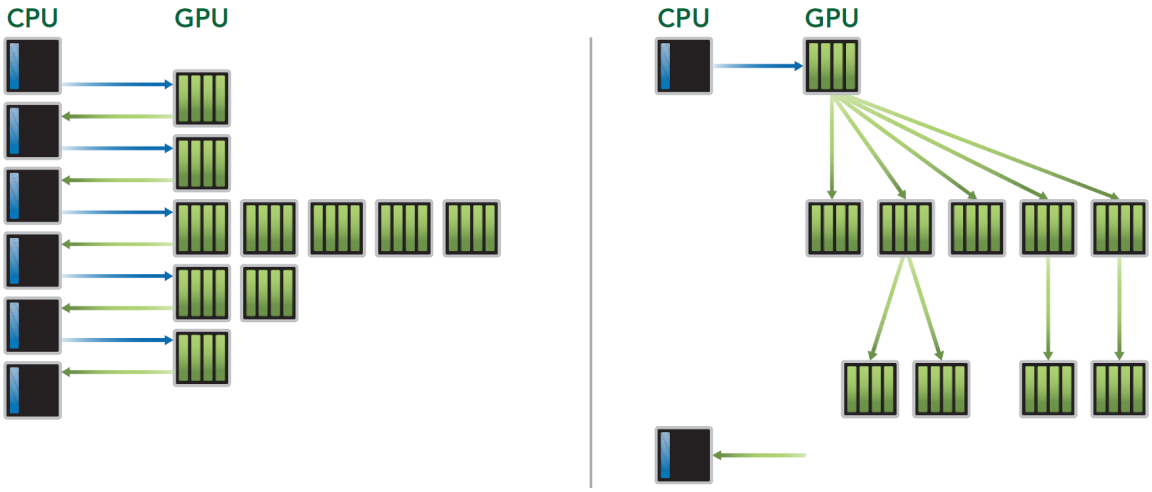


Figure 4.11: Dynamic Parallelism in CUDA (Cheng et al., 2014).

Table 4.1: Programmable memory types, scopes, and lifetimes.
Note: L1 and L2 caches are not included because they are hardware-managed.

Memory Type	Scope	Lifetime	Location
Global	Grid	Application	Device
Constant	Grid	Application	Device
Shared	Block	Kernel	SM
Local	Thread	Kernel	Device
Register	Thread	Kernel	SM

4.9.6. GPU MEMORY

In modern computing systems, efficient use of memory is as important as the effective utilisation of the functional units that perform computations. Heterogeneous systems contain several memory spaces, while GPUs additionally provide different forms of programmable on-chip memory alongside conventional hardware-managed caches.

4.9.6.1. DRAM IN HETEROGENEOUS SYSTEMS

Both CPUs and GPUs have directly attached DRAM chips. From the perspective of device code, the DRAM connected to a GPU is called *global memory* because it is accessible by all SMs within that GPU. The term *global* does not imply that the memory is necessarily accessible by every processor in the complete heterogeneous system. The DRAM connected to the CPU is generally referred to as *host memory*. Global memory is persistent. That is, an allocation made in global memory and the data stored in it persist until the allocation is freed or until the application is terminated.

Like CPUs, GPUs use virtual-memory addressing. On currently supported CUDA systems, the CPU and GPUs operate within a unified virtual address space. For a given address, the system can determine whether the associated allocation resides in host or global memory.

CUDA provides APIs for allocating CPU and GPU memory and for transferring data between CPU and GPU allocations and within a GPU. The locality of data may be managed explicitly when required. Alternatively, CUDA Unified Memory, which will be discussed in Section 4.9.6.3, allows the runtime or system hardware to manage memory placement automatically.

4.9.6.2. ON-CHIP MEMORY IN GPUS

In addition to global memory, each GPU provides on-chip memory resources, summarised in Table 4.1. Every SM has its own register file and shared-memory capacity. Because these resources are located within the SM, threads executing on that SM can access them with very low latency.

The register file stores thread-local variables and is normally managed automatically by the compiler. Shared memory is accessible to all threads within the same thread block or, when thread-block clusters are used, within the relevant cluster. It can therefore be used to exchange data and reuse intermediate results. The register file, shared memory, and L1-cache resources of an SM must be shared among all resident threads and blocks.

Shared-memory allocation differs from register allocation. Registers are assigned individually to threads, whereas a shared-memory allocation belongs to the complete thread block and is accessible by all threads within it.

Local memory is thread-local storage similar to registers and managed by the CUDA compiler, but the physical location of local memory is in the global memory space. The “local” label refers to its logical scope, not its physical location. Local memory is used for thread-local storage during the execution of a kernel. Automatic variables that the compiler is likely to place in local memory are: arrays for which it cannot determine that they are indexed with constant quantities; large structures or arrays that would consume too much register space; and any variable when the kernel uses more registers than are available, a situation known as *register spilling*. Because the local memory space resides in device memory, local-memory accesses have the same latency and bandwidth as global-memory accesses.

In addition to programmable memory resources, GPUs contain both L1 and L2 caches. Each SM has an L1 cache that forms part of its unified data cache. A larger L2 cache is shared among all SMs within the GPU, as shown in Figure 4.3.

Each SM also contains a separate constant cache. This cache stores values located in global memory that have been declared constant for the duration of a kernel. The compiler may also place kernel parameters in constant memory, allowing these parameters to be cached separately from ordinary data handled by the L1 cache. Constant memory is useful for small amounts of data that each thread will use in a read-only fashion. Constant memory is small relative to other memories, typically 64 KB per device.

4.9.6.3. UNIFIED MEMORY

When memory is allocated explicitly on either the CPU or GPU, it is normally accessible only to code executing on that processor. Host code accesses CPU memory, whereas GPU kernels access memory allocated on the corresponding device. CUDA memory-copy operations are therefore required to move data into the appropriate memory space before it is used.

An exception is *mapped host memory*, which is CPU memory allocated with properties that allow direct GPU access. The GPU accesses this memory through an interconnect such as PCI Express or NVLink. However, this access path generally has higher latency and lower bandwidth than local GPU memory, and these penalties cannot always be fully hidden through parallel execution. Mapped host memory should therefore not be regarded as a performance-equivalent substitute for Unified Memory or for placing data in the memory space local to the processor that uses it.

CUDA Unified Memory allows an application to create allocations that are accessible from both the CPU and GPU. The CUDA runtime, or the underlying hardware, manages this shared access and migrates data between physical memory locations when required.

Unified Memory does not remove the importance of data locality. High performance still depends on minimising unnecessary data migration and, whenever possible, accessing data from the processor directly connected to the memory in which those data reside.

The specific hardware capabilities of the system determine how access to, and transfer between, the different memory spaces are implemented (NVIDIA Corporation, 2026).

5

ROBUST TRAJECTORY OPTIMISATION

In this chapter, the theoretical foundations of robust trajectory optimisation are presented, and the problem formulation adopted in this work is derived. Section 5.1 introduces the optimal control problem, first in its deterministic form and then in its stochastic generalisation. Section 5.2 reviews uncertainty-propagation methods and motivates the selection of the hybrid GMM–UT approach. Section 5.3 presents the transcription methods used to convert probabilistic constraints and costs into tractable deterministic conditions, together with the associated risk-estimation, conservatism, and risk-allocation considerations. Section 5.4 then reviews deterministic trajectory-optimisation solvers and justifies the selected solution approach. Finally, Section 5.5 formulates the robust optimal control problem addressed in this thesis.

5.1. THE OPTIMAL CONTROL PROBLEM

This section first introduces the deterministic optimal control problem. The formulation is then extended to the stochastic case, where uncertainty affects the system dynamics, constraints, and objective functional. This distinction provides the theoretical basis for the robust trajectory-optimisation framework developed in the remainder of this chapter.

5.1.1. DETERMINISTIC REPRESENTATION

Optimal control theory provides a systematic framework for determining control inputs that guide a dynamic system toward desired objectives while satisfying physical and operational constraints. The deterministic formulation of the Optimal Control Problem (OCP) defines this baseline setting, where all system parameters and external influences are assumed to be known exactly. The deterministic formulation serves as the conceptual and mathematical foundation upon which the stochastic extensions developed in later sections are built, enabling the explicit treatment of uncertainties at all levels of system dynamics, constraints, and performance metrics.

5.1.1.1. SYSTEM CONSTRAINTS

Constraints capture the physical, operational, and safety limitations that any feasible trajectory must satisfy. They ensure that the system behaviour remains realistic and safe while optimising a prescribed performance index. Depending on their scope, constraints can apply only at specific time instants, such as at the beginning or end of the time horizon, or continuously over the entire trajectory.

The first type, referred to as *boundary constraints*, specifies a set of algebraic relations that define admissible initial and terminal conditions in the form of equality equations

$$\boldsymbol{\psi}\left(\boldsymbol{x}(t_0), \boldsymbol{x}(t_f), \boldsymbol{u}(t_0), \boldsymbol{u}(t_f), t_0, t_f\right) = \mathbf{0}, \tag{5.1}$$

where $\boldsymbol{\psi} \in \mathbb{R}^{n_\psi}$ groups all endpoint requirements. Typical boundary specifications include fixed starting and arrival states, but can also involve free-final-time conditions, continuity relations between dynamic phases, or transversality constraints.

In addition to endpoint requirements, practical systems are often restricted by conditions that must remain valid throughout the motion. These are termed *path constraints*; they are usually expressed as inequality relations and can be written as

$$\mathcal{X}_t\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big) \leq 0 \qquad t \in [t_0, t_f] \tag{5.2}$$

where $\mathcal{X}_t \in \mathbb{R}^{n_{\mathcal{X}}}$ collects the set of continuous-time constraints. Such constraints can describe physical boundaries (e.g., obstacle-avoidance regions or line-of-sight limits) or operational restrictions (e.g., actuator saturation, allowable altitude bands, or distance from planets).

Equations 5.1 and 5.2, together with the deterministic part of the dynamics defined in Equation 3.59, define a deterministic dynamic framework within which the state trajectory is completely determined by the applied control sequence under the given constraints.

5.1.1.2. OBJECTIVE FUNCTION

The evolution of the system is evaluated using a performance index, expressed through a scalar functional termed the *objective functional* or *cost functional*. The objective functional quantitatively measures the performance of a candidate trajectory; the goal of the OCP is therefore to determine the admissible control history $\boldsymbol{u}(t)$ that minimises (or maximises) this cost while satisfying the dynamics and constraints introduced previously.

In its most general formulation, the cost functional is written as

$$J = \Phi\big(\boldsymbol{x}(t_0), \boldsymbol{x}(t_f), t_0, t_f\big) + \int_{t_0}^{t_f} L\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\, dt \tag{5.3}$$

where $\Phi\big(\boldsymbol{x}(t_0), \boldsymbol{x}(t_f), t_0, t_f\big)$ is referred to as the *terminal cost* or *Mayer term*, while $L\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)$ is referred to as the *running cost* or *Lagrange term* (Longuski et al., 2014). The integral term accumulates the performance measure along the trajectory, whereas the Mayer term accounts for the contribution of the endpoint states and times to the objective.

Depending on the control objective and application context, these contributions may appear separately or together. Their combination defines the *Bolza form* of the cost functional, while the Lagrange and Mayer forms correspond to its two limiting cases. The specific choice of cost functional depends on the control objective, mission type, and performance metric to be optimised. These forms can also be converted into one another (Longuski et al., 2014).

Although OCPs are traditionally formulated with a single scalar cost functional, practical scenarios may require the simultaneous optimisation of multiple, potentially conflicting objectives, for instance fuel efficiency versus travel time (Gunantara, 2018). Such problems are addressed using multi-objective optimisation or scalarisation techniques. In the latter approach, several criteria are combined, typically through weighted sums, into a single composite performance index (Gambier and Badreddin, 2007). This scalarisation enables the use of standard optimisation tools, while preserving flexibility in weighting trade-offs between objectives.

In deterministic settings, the minimisation of this cost functional yields the optimal control law and the corresponding nominal trajectory.

5.1.2. STOCHASTIC REPRESENTATION

This section generalises the deterministic optimisation framework into a stochastic one by formally introducing uncertainty into the state dynamics, extending the constraints and objective functional to a probabilistic form. In the stochastic setting, the system state, $\boldsymbol{x}(t)$, is represented as a multivariate random process rather than a deterministic vector, capturing the statistical uncertainty of the system.

Instead of describing a single deterministic trajectory, the stochastic representation considers a family of possible realisations of the state, each weighted by its likelihood of occurrence. This probabilistic viewpoint provides a more realistic and comprehensive representation of the system's behaviour under uncertain environments, particularly relevant in space missions, where perfect knowledge of the spacecraft's state is practically unattainable.

5.1.2.1. PROBABILISTIC CONSTRAINTS

Based on the statistical representation of the system state, the constraints introduced in Section 5.1.1.1 can now be extended to their stochastic counterparts. The admissible region of the system in the deterministic formulation, defined by Equations 5.1 and 5.2, was expressed through equality and inequality relations on

the state and control variables. When uncertainty is present, these constraints cannot be guaranteed to hold exactly for all realisations of the random variables; instead, they are enforced in a probabilistic sense to keep the likelihood of violation below a prescribed acceptable threshold.

The stochastic counterpart of the path inequality constraints is therefore expressed in the literature as

$$\begin{aligned} &\mathbb{P}\left\{\mathcal{X}_t\big(\boldsymbol{x}(t),\boldsymbol{u}(t),t\big)\leq 0\right\}\geq p, \\ \text{or}\qquad &\mathbb{P}\left\{\mathcal{X}_t\big(\boldsymbol{x}(t),\boldsymbol{u}(t),t\big)\leq 0\right\}\geq 1-\beta, \qquad t\in[t_0,t_f], \end{aligned} \tag{5.4}$$

where $p \in [0,1)$ represents a high confidence level, and $\beta = 1-p$ denotes the corresponding low violation probability or risk level. This probabilistic formulation, known as a *chance constraint*, ensures that physical and operational limits are satisfied with confidence level at least $p = 1-\beta$, while allowing for small, infrequent violations due to uncertainty.

Rather than enforcing exact equalities at the endpoints, the boundary constraints are likewise required to lie within admissible sets with a high probability. In the most general form, these stochastic boundary conditions can be written as

$$\mathbb{P}\left\{\boldsymbol{\psi}\big(\boldsymbol{x}(t_0),\boldsymbol{x}(t_f),\boldsymbol{u}(t_0),\boldsymbol{u}(t_f),t_0,t_f\big)=0\right\}\geq 1-\beta, \tag{5.5}$$

where $\beta \in [0,1)$ is again a risk parameter that bounds the violation probability. This formulation provides the flexibility to model coupled or nonlinear boundary conditions involving both state and control variables.

To improve tractability, a common approach is to decompose the general constraint into separate conditions on the initial and final states, which are written as

$$\mathbb{P}\left\{\boldsymbol{x}(t_0)\in\mathcal{X}_i\right\}\geq 1-\beta_i, \tag{5.6}$$

$$\mathbb{P}\left\{\boldsymbol{x}(t_f)\in\mathcal{X}_f\right\}\geq 1-\beta_f, \tag{5.7}$$

where $\mathcal{X}_i, \mathcal{X}_f \subset \mathbb{R}^{n_x}$ are admissible sets centred around the desired initial and final states, and β_i, β_f denote the acceptable violation probabilities for each endpoint.

In addition to probabilistic state and control constraints, limitations may also be imposed directly on the uncertainty level of the system itself. A common example is to bound the maximum eigenvalue of the covariance matrix to ensure that uncertainty remains within acceptable limits:

$$\lambda_{\max}\big(\Sigma(t)\big)\leq\bar{\lambda}, \qquad t\in[t_0,t_f], \tag{5.8}$$

where $\bar{\lambda} \in \mathbb{R}_+$ specifies the maximum allowable spread of the state distribution. This constraint effectively limits the largest principal axis of the covariance ellipsoid, ensuring that the overall uncertainty in the state estimation remains within a prescribed bound.

Together, these probabilistic boundary and path constraints define the feasible domain of an SOCP. They preserve the physical and operational meaning of the deterministic constraints while embedding robustness against uncertainty, ensuring that the optimised solutions remain feasible and reliable with high confidence.

5.1.2.2. PROBABILISTIC OBJECTIVE FUNCTION

The stochastic cost is generally expressed as

$$J=\mathcal{F}\left(\Phi\big(\boldsymbol{x}(t_0),t_0,\boldsymbol{x}(t_f),t_f\big)+\int_{t_0}^{t_f}L\big(\boldsymbol{x}(t),\boldsymbol{u}(t),t\big)\,dt\right), \tag{5.9}$$

where $\mathcal{F}(\cdot)$ represents a statistical operator that maps the random cost into a deterministic scalar objective. The choice of $\mathcal{F}(\cdot)$ determines how uncertainty is incorporated into the optimisation problem. Two common formulations in the literature are the *expected-value* and the *quantile-based* definitions of the cost functional, which are explained next.

5.1.2.3. EXPECTED-VALUE DEFINITION

The most common formulation defines the cost as the expected value of the performance index over all possible realisations of the stochastic processes driving the system dynamics, rather than evaluating performance along a single deterministic path,

$$J=\mathbb{E}\left[\Phi\big(\boldsymbol{x}(t_0),t_0,\boldsymbol{x}(t_f),t_f\big)+\int_{t_0}^{t_f}L\big(\boldsymbol{x}(t),\boldsymbol{u}(t),t\big)\,dt\right]. \tag{5.10}$$

This expected-value definition of the cost function evaluates the mean performance of the control policy. Minimising J yields a policy that optimises the average mission performance while implicitly mitigating the effects of uncertainty in system dynamics and disturbances.

5.1.2.4. QUANTILE-BASED DEFINITION

In many applications, especially when the consequences of rare adverse events are critical, designing a control policy solely based on expected performance may not guarantee sufficient robustness. To address this limitation, a more conservative formulation, known as the quantile-based or risk-sensitive definition, can be adopted. In this approach, the objective shifts from minimising the expected cost to minimising a high-confidence bound (quantile) of the cost distribution. Let $\tilde{J}$ represent the stochastic cost, which depends on the feedback-driven system trajectory. The quantile-based definition seeks to find a threshold J such that the cost does not exceed J with a high probability p, which is written as,

$$\mathbb{P}\{\tilde{J} \leq J\} \geq p, \tag{5.11}$$

which defines J as the p-quantile of $\tilde{J}$, denoted by $Q(\tilde{J}, p)$. The optimisation problem then becomes

$$\min \; Q(\tilde{J}, p), \tag{5.12}$$

where the objective penalises high-cost outcomes with low probability, leading to control policies that are robust against extreme realisations.

In some formulations, the objective function may also include terms that directly penalise state uncertainty (Yi et al., 2019), such as the trace or Frobenius norm of the covariance, in addition to traditional performance metrics.

In summary, the stochastic dynamics defined by Equation 3.58, the probabilistic constraints, and stochastic objective functional define the general form of the Stochastic Optimal Control Problem (SOCP), summarised as

$$\min_{\boldsymbol{x}(t),\boldsymbol{u}(t)} \quad J = \mathcal{F}\left(\Phi\big(\boldsymbol{x}(t_0), t_0, \boldsymbol{x}(t_f), t_f\big) + \int_{t_0}^{t_f} L\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\, dt\right), \tag{5.13a}$$

$$\text{s.t.} \quad d\boldsymbol{x}(t) = \boldsymbol{f}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\, dt + \boldsymbol{g}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big)\, d\boldsymbol{w}(t), \tag{5.13b}$$

$$\mathbb{P}\left\{\boldsymbol{\psi}\big(\boldsymbol{x}(t_0), \boldsymbol{x}(t_f), \boldsymbol{u}(t_0), \boldsymbol{u}(t_f), t_0, t_f\big) = \mathbf{0}\right\} \geq 1 - \beta, \tag{5.13c}$$

$$\mathbb{P}\left\{\boldsymbol{\mathcal{X}}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big) \leq \mathbf{0}\right\} \geq 1 - \beta, \qquad t \in [t_0, t_f]. \tag{5.13d}$$

This general probabilistic formulation provides the foundation for the methodology developed in the following sections.

5.2. UNCERTAINTY PROPAGATION

Everything in life is affected by some degree of uncertainty. The state of a given system cannot be perfectly predicted simply by solving the deterministic equations of motion that govern its evolution. That is why uncertainties must always be accounted for in any engineering design. To this end, a significant effort has been put into the development of uncertainty propagation methods in various fields. The goal of propagating an uncertainty is to be able to predict the distribution of a state at a given time in the future. In orbital mechanics, there are multiple applications of uncertainty propagation. Its most common uses include trajectory estimation, sensitivity analysis, prediction of satellite collisions or robust trajectory optimisation. By the end of this section, the methods considered most promising for the GPU-accelerated robust optimisation of interplanetary trajectories are identified.

The uncertainty of a state $\boldsymbol{x} \in \mathbb{R}^{n_x}$ is described using a Probability Density Function (PDF). The temporal evolution of a given PDF $p(\boldsymbol{x}, t)$ can be analytically computed by solving the Fokker-Planck Equation (FPE) (Fuller, 1969):

$$\frac{\partial p(\boldsymbol{x}, t)}{\partial t} = -\sum_{i=1}^{n} \frac{\partial}{\partial x_i}\left[p(\boldsymbol{x}, t)\boldsymbol{f}^i(\boldsymbol{x}, t)\right] + \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\partial^2}{\partial x_i \partial x_j}\left\{p(\boldsymbol{x}, t)\left[\mathbf{G}(t)\mathbf{Q}(t)\mathbf{G}^T(t)\right]^{ij}\right\} \tag{5.14}$$

where $\boldsymbol{f}(\boldsymbol{x}, t)$ is the differential equation that governs the deterministic evolution of the state, $\mathbf{G}(t)$ is the $n_x \times n_w$ noise diffusion matrix, and $\mathbf{Q}(t)$ is the $n_w \times n_w$ process noise covariance matrix (where n_x is the state dimension and n_w is the dimension of the Brownian motion process).

Solving this equation provides a complete statistical description of the PDF at any given time, which in turn describes the uncertainty of the state. However, when considering a large number of uncertainty variables, and particularly for strongly nonlinear dynamics, solving the FPE becomes extremely difficult (Luo and Yang, 2017; Sun and Kumar, 2016).

In recent times, Acciarini et al. (2024) demonstrated that Galerkin projection allows the time-dependent and state-dependent components of the Fokker–Planck equation to be decoupled, while retaining the complete shape of the probability density function over time in the presence of both deterministic and stochastic dynamics. This offline–online decomposition is attractive because the offline evaluation of derivatives and multidimensional integrals is highly parallelisable, with relatively limited coding effort. With this approach, the values of the probability density function at future times and for different initial conditions can be computed without substantial additional cost, provided that a set of time-independent integrals is evaluated offline in advance.

Acciarini et al. (2024) selected third-order B-spline basis functions. This choice is motivated by the fact that these basis functions are non-negative, which is a fundamental property of probability density functions. Moreover, increasing the number of basis functions only requires increasing the number of polynomials of the same degree over a prescribed interval, without increasing the polynomial degree itself. These basis functions also possess a particularly useful property: the matrices arising from the Galerkin projection become sparse, allowing the number of required computations to be reduced and the corresponding operations to be performed more efficiently. In addition, B-spline basis functions allow both periodic and non-periodic boundary conditions to be treated conveniently.

Although the increased number of zero elements in the matrices arising from the Galerkin projection partially alleviates the computational burden, constructing multivariate basis functions as products of monovariate basis functions causes the total number of basis functions to grow exponentially with the number of dimensions. Therefore, depending on the application and on the number of basis functions required to approximate the dynamical system accurately, it may be necessary to adopt alternative constructions for the basis functions.

An important limitation is that the method is applicable only when both the diffusion tensor and the vector of deterministic forces do not explicitly depend on time. A further requirement for maintaining propagation accuracy is that the selected basis functions remain capable of approximating the evolving probability density function adequately within the prescribed spatial domain. Therefore, this method will not be employed in the present study.

In the last 40 years of the 20th century, Monte Carlo simulations and linear methods were the standard choice for uncertainty propagation (Park, 2007). Monte Carlo simulations provide a very accurate statistical description of the state. However, an extremely high number of simulations is required to achieve a sound statistical level of confidence, making this method very computationally expensive. Linear methods, on the other hand, are not well suited for most real-life applications, since often long propagation times are required. Furthermore, particularly for low-thrust trajectory design applications, large discontinuities and nonlinearities are expected, which further deteriorate the performance of linear methods.

In this section, the goal is to understand which methods are preferred in robust optimisation literature, and the rationale behind the use of each technique. In Section 5.2.1, a summary of uncertainty propagation methods is provided. The application of these methods to spaceflight problems is reviewed in Section 5.2.2. This analysis leads to the choice of a combination of methods in Section 5.2.3.

5.2.1. SUMMARY OF METHODS

The most widely used methods for uncertainty propagation are presented in this section. As shown in Figure 5.1, these methods can be divided into three categories: linear methods, nonlinear methods, and "other" approaches, including Chebyshev interpolation, coordinate transformations, and hybrid methods.

Linear methods were introduced as an alternative to MC simulations because of their conceptual simplicity and low computational cost. LinCov is a local linearisation method developed by Battin (1999) that has been successfully applied to rendezvous problems (Geller et al., 2009) and powered lunar descent and landing applications (Geller and Christensen, 2009). CADET (Covariance Analysis DEscribing Function Technique) is a statistical linearisation method introduced by Gelb and Warren (1973). Originally designed for nonlinear guidance systems (Taylor, 1975), the method has subsequently been applied to the analysis of missile guidance systems (Gurfil, 2001; Zarchan, 1979). However, the accuracy of both LinCov and CADET deteriorates rapidly in strongly nonlinear scenarios, rendering them inadequate for the present work. Furthermore, these methods are not suitable for propagating non-Gaussian uncertainties, which may arise in interplanetary

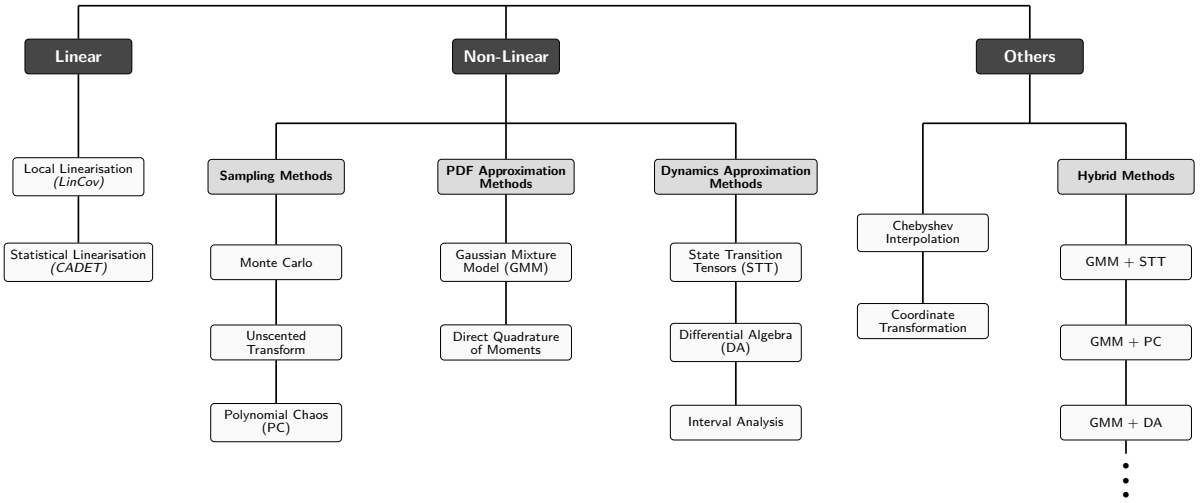


Figure 5.1: Classification of uncertainty-propagation methods.
Adapted from Luo and Yang (2017), Leon Dasi (2021), and Silva (2025).

trajectory-design problems because of factors such as orbital insertion errors. Therefore, linear methods are not considered further in this study.

Since the end of the twentieth century, numerous nonlinear uncertainty propagation methods have been developed to address the limitations associated with linear approaches. However, each of these methods presents specific disadvantages relative to the others, leaving considerable scope for application-dependent trade-offs. Hybrid methods have therefore been developed to combine the advantages of different nonlinear techniques. By appropriately integrating two algorithms, a hybrid method may achieve high propagation accuracy while retaining computational efficiency. Other independent approaches have also been formulated by building upon existing nonlinear techniques. For example, Chebyshev interpolation leverages established propagation methods to improve properties such as numerical stability and robustness. Moreover, coordinate transformations may be employed to simplify the uncertainty propagation problem and improve its accuracy.

The methods shown in Figure 5.1 are introduced in the following sections, with the exception of the linear methods, which have already been excluded from further consideration. This overview provides the basis for the methodological trade-off presented in Section 5.2.3.

Sampling Methods

Sampling methods consist of selecting a set of elements from the initial-state distribution, referred to as a sample, and propagating them through the system dynamics. The final PDF is then estimated from the distribution of the propagated samples. In these methods, the dynamical model may be treated as a black box, and no approximations or simplifications of the governing dynamics are required. Two sampling methods are considered in this work: Monte Carlo simulations and the Unscented Transform.

The classical approach for assessing the robustness of a given system is based on MC simulations. This widespread use stems from the conceptual simplicity of the method and its applicability irrespective of the degree of nonlinearity of the system, the non-Gaussianity of the probability distributions, or the duration of the propagation interval. The accuracy of the approach depends only on the sample size and the fidelity of the numerical simulation, which makes it a computationally expensive technique as the number of simulations approaches infinity.

Within the MC framework, samples are drawn according to the PDF of the initial state and subsequently propagated through the system dynamics. The corresponding final state is therefore accurately computed for each sampled initial condition. When a sufficiently large number of samples is propagated, the resulting ensemble provides an increasingly accurate representation of the final-state distribution. Consequently, MC simulation is among the most widely trusted uncertainty propagation techniques and is particularly valuable for validation purposes.

More recently, in an effort to reduce the runtime of MC simulations, several authors have proposed exploiting GPUs and parallel computing architectures (Arora et al., 2015; Liu et al., 2015).

An alternative means of reducing the computational burden is to decrease the number of propagated samples. The Unscented Transform (UT) is the most widely used approach based on this principle. The UT was

proposed by Julier et al. (1995) and consists of propagating a small set of deterministically selected points, referred to as σ-points, which capture the mean and covariance of the initial PDF. For a system with n degrees of freedom, the standard UT requires only $2n + 1$ σ-points. Although this results in a highly computationally efficient method, the representation of the initial PDF through only its mean and covariance makes the standard UT primarily suitable for the propagation of Gaussian uncertainties.

Polynomial Chaos Expansion (PCE) was first proposed by Wiener (1938) and consists of approximating the response surface of the final state using orthogonal polynomials. To construct this approximation, a sample of initial states is propagated through the system dynamics. The propagation results are then used to establish a mapping between the solution of the system's differential equations and deviations in the initial state. However, when the system response must be evaluated at different points in time, the mapping procedure must be repeated at each time instant, although the underlying trajectories need only be propagated once. The method can be implemented in either intrusive or non-intrusive form, with the latter offering the advantage that the dynamical model can be treated as a black box.

PDF Approximation Methods

PDF approximation methods include the Gaussian Mixture Model and the Direct Quadrature Method of Moments. A Gaussian distribution can be fully characterised by its first two moments: the mean and the variance. By contrast, a non-Gaussian distribution generally requires additional parameters for its complete characterisation. For a given level of uncertainty, the more complex the PDF, the greater the computational burden associated with its propagation. Indeed, even initially Gaussian distributions generally become non-Gaussian when propagated through nonlinear dynamics (Alfriend and Park, 2016). The objective of PDF approximation methods is therefore to decompose a complex PDF into simpler components, thereby simplifying the uncertainty propagation process.

The Gaussian Mixture Model (GMM) is a technique that approximates an arbitrary PDF as a finite weighted sum of Gaussian distributions. This representation simplifies the propagation step because each Gaussian distribution requires only its first two moments to be propagated. This property makes the GMM particularly attractive for use within hybrid methods: the PDF is first approximated using a GMM, after which the Gaussian components constituting the approximation, referred to as Gaussian Mixture Elements (GMEs), are propagated using either a dynamical approximation or a sampling technique.

A related alternative to the GMM is the Direct Quadrature Method of Moments (DQMOM), proposed by Marchisio and Fox (2005). In this approach, the original PDF is approximated by a weighted sum of Dirac delta functions, represented through a discrete set of weights and abscissas. The method can achieve high accuracy and computational efficiency (Xu, 2009; Xu and Xin, 2011). However, DQMOM is less commonly employed for uncertainty propagation than other techniques, particularly in astrodynamics, owing to its greater methodological and implementation complexity relative to the GMM.

Dynamics Approximation Methods

Dynamics approximation methods aim to simplify the propagation process itself by modifying the dynamical model. This generally results in a considerably more computationally efficient approach than sampling methods, since the full dynamical model no longer needs to be evaluated repeatedly, which typically constitutes the most computationally expensive stage of the propagation process.

State Transition Tensors (STTs) were proposed by Park and Scheeres (2007). This approach employs a Taylor expansion of the dynamics about the mean of the initial state. The initial uncertainties are propagated as deviations from the nominal initial state by solving for the state transition tensors, which correspond to the higher-order partial derivatives appearing in the Taylor expansion. The STT formulation requires the Taylor expansion to be integrated only once. By contrast, an MC approach requires multiple independent propagations through the nonlinear dynamics. However, to achieve a satisfactory level of accuracy, the Taylor expansion must be retained up to a sufficiently high order, which can substantially increase the computational burden.

Differential Algebra (DA) provides an alternative framework for manipulating polynomials. Its mathematical foundations can be traced back to Ritt (1950), while a comprehensive formulation of modern DA techniques is provided by Berz (1999), who also introduced the computerised implementation commonly used today (Berz, 1988). DA enables the dynamical flow to be expanded within a computerised environment, resulting in faster Taylor Series Integration (TSI) than conventional numerical integration methods. On its own, however, DA does not necessarily provide a substantial advantage over other approaches and, similarly to the STT method, is restricted to continuous and differentiable dynamics. A method for addressing discontinuities and differentiability issues, such as those associated with atmospheric density models, was proposed

by Bergsma and Mooij (2016).

Interval Analysis (IA), introduced by Moore (1966), consists of propagating only the extreme values of the distribution, thereby providing substantial computational savings compared with a conventional MC approach. By propagating the upper and lower bounds of the state distribution, the method can be used to guarantee that prescribed constraint limits are not exceeded during a trajectory. However, IA provides no information about the state distribution within these bounding values. Moreover, the use of IA may lead to a significant overestimation of the distribution bounds, as discussed in greater detail by Römgens et al. (2013).

Chebyshev Interpolation

All the dynamics approximation methods discussed above share a common feature: they employ polynomials either to expand the dynamical flow or to represent the response surface of the solution to a differential equation. It was therefore natural for researchers to investigate whether these techniques could be generalised to alternative polynomial bases. One of the earliest attempts was based on the so-called ultra-arithmetic framework introduced by Epstein et al. (1982), in which the digital representation of numbers and arithmetic operations was defined using Chebyshev, Legendre, Fourier, and other bases. However, the concept was subsequently abandoned because of difficulties associated with the multiplication of functions represented using non-Taylor approximations and the increase in the number of coefficients, which resulted in larger errors.

Several decades later, Brisebarre and Joldecs (2010) formally compared the effects of expressing polynomials in the Taylor, Newton, and Chebyshev bases, incorporating both developments from previous studies and their own contributions. The results demonstrated that Chebyshev polynomials could achieve smaller errors than Taylor series of the same order, albeit at a slightly greater computational cost.

In this work, Chebyshev Interpolation (CI) refers to the reformulation of the dynamics approximation methods discussed previously, such as DA or PCE, by expressing their polynomial representations in the Chebyshev basis. CI is particularly attractive because of the stability and convergence properties it provides (Ortega et al., 2016). Furthermore, Tardioli et al. (2015) compared several non-intrusive methods and found that non-intrusive CI consistently outperformed alternative techniques, such as PCE, achieving a prescribed level of accuracy with considerably fewer propagations. Nevertheless, its performance may be affected by oscillatory effects, including the Gibbs phenomenon (Gottlieb and Shu, 1997) and the Runge phenomenon (Brisebarre and Joldecs, 2010), although generally to a lesser extent than other interpolation methods.

Coordinate Transformation

The choice of coordinates used to describe and propagate the PDF is of considerable importance in uncertainty propagation. The objective of a coordinate transformation is to identify a state representation that more effectively isolates or simplifies the dominant nonlinear terms in the governing dynamical equations. By appropriately reformulating the state, the distribution may retain its Gaussian character over longer propagation intervals, thereby also extending the validity of linear assumptions (Alfriend and Park, 2016; Weisman et al., 2016).

Hybrid Methods

All the methods reviewed thus far exhibit specific strengths and limitations, giving rise to application-dependent trade-offs. Hybrid approaches aim to combine the advantages of two uncertainty propagation techniques in order to compensate for their respective limitations, thereby considerably improving computational efficiency while maintaining a prescribed level of accuracy. GMM is commonly selected as one of the techniques included in a hybrid formulation because it decomposes the initial PDF into multiple Gaussian distributions. This decomposition allows other methods, such as DA (Sun et al., 2019), PCE (Vittaldev et al., 2016), or STT (Fujimoto and Scheeres, 2015), to be applied using polynomials of significantly lower order, since only the mean and variance of the smaller component distributions need to be propagated.

In summary, hybrid methods can overcome some of the limitations associated with individual propagation techniques, thereby improving the overall efficiency of the uncertainty propagation process.

5.2.2. UNCERTAINTY PROPAGATION IN SPACEFLIGHT APPLICATIONS

Having developed an initial understanding of the strengths and limitations of the different techniques, a more informative analysis of their respective applications can now be conducted. In this section, applications of uncertainty propagation in spaceflight dynamics are reviewed. The objective is to understand the rationale underlying the selection of each method and thereby support an unbiased trade-off in Section 5.2.3.

Uncertainty propagation has been applied to robust guidance problems, particularly through the use of DA, for aerocapture (Di Lizia et al., 2008), atmospheric entry (Lunghi et al., 2018), and planetary landing (Lunghi et al., 2016). In these studies, DA was selected because of its versatility in automatically computing

high-order derivatives, since all three employed simplified environmental models, including an exponential atmosphere without wind, thereby ensuring that the governing functions remained smooth and continuous. These characteristics favoured the use of DA and eliminated the perceived need for comparisons with alternative techniques. Notably, Lunghi et al. (2016, 2018) considered uncertainties in both the initial state and the environmental parameters.

Uncertainty propagation has also been applied to robust control problems. Huang et al. (2019) employed non-intrusive PCE to develop a computationally efficient and accurate optimal-control algorithm. PCE was selected primarily because its non-intrusive formulation is computationally efficient and allows uncertainty propagation and optimisation to be integrated within a single control algorithm. However, only five uncertain variables were considered, which partly explains the computational efficiency of the approach.

Stochastic optimisation problems that explicitly incorporate uncertainty propagation frequently employ PCE (Fisher and Bhattacharya, 2011; Jiang and Li, 2019; Whittle and Sagliano, 2018; Xiong et al., 2015). This preference largely arises from the development of a general robust trajectory-optimisation framework based on intrusive PCE (Fisher and Bhattacharya, 2011; Whittle and Sagliano, 2018). Within this framework, PCE is embedded into the optimisation procedure such that the stochastic problem is transformed into a deterministic optimal-control problem (Darby et al., 2011). This transformation is possible because the effects of the uncertain variables can be represented through convergent polynomial expansions (Whittle and Sagliano, 2018).

Based on the characteristics of the individual uncertainty-propagation techniques, other intrusive approaches, such as STT or DA, could conceivably achieve similar results. However, none of these studies compared PCE with alternative propagation methods. Moreover, the proposed procedure was developed as a general methodology and was initially tested on relatively simple examples, such as the Van der Pol oscillator (Fisher and Bhattacharya, 2011). The findings of these studies further indicate that the accuracy of PCE deteriorates over long propagation intervals relative to MC simulations, although the method still provides considerable improvements compared with a purely deterministic optimisation approach (Fisher and Bhattacharya, 2011; Whittle and Sagliano, 2018).

Jiang and Li (2019) applied this methodology to the optimisation of a Mars atmospheric-entry trajectory. Although a simplified exponential atmospheric model was employed, several sources of uncertainty were considered, including the initial position vector, initial velocity vector, lift-to-drag ratio, atmospheric density, and ballistic coefficient. The results demonstrated that the method can remain computationally efficient even when a relatively large number of uncertain variables is included. Nevertheless, PCE was selected primarily because the optimisation-integrated framework had already been established, and no comparison with alternative uncertainty-propagation techniques was performed.

Park and Scheeres (2006) and Yang et al. (2019) employed STTs to design orbital manoeuvres, including rendezvous operations, under uncertainty. Park and Scheeres (2006) introduced the STT methodology at a time when MC simulations and linear techniques represented the predominant approaches to uncertainty propagation. Consequently, the use of STTs already constituted a substantial improvement in computational efficiency. Yang et al. (2019) selected STTs because of their ability to propagate arbitrary distributions accurately when the expansion is retained up to a sufficiently high order, while the tensor expansion needs to be computed only once for the complete trajectory. However, these advantages are also provided by DA, with the additional benefit that high-order derivatives can be computed automatically and efficiently.

The use of an optimisation algorithm already entails a substantial computational burden. Consequently, when uncertainty-propagation techniques are integrated into optimisation algorithms, computational efficiency is generally prioritised over very high propagation accuracy. One example is the use of the UT by Ricciardi et al. (2021) to propagate uncertainty in atmospheric temperature along a re-entry trajectory. This study employed the US76 atmospheric model, representing an improvement over the use of a simplified exponential atmosphere.

Selim and Ozkol (2023) implemented a more sophisticated extension of the UT, conceptually similar to non-intrusive PCE, in an attempt to maximise computational efficiency while also capturing higher-order statistical moments. However, parallel-processing capabilities were required to obtain sufficient computational performance. The uncertainty-propagation algorithm was employed to robustly optimise a re-entry trajectory, although only an exponential atmospheric model was considered. A non-intrusive technique was selected for two principal reasons. First, it enables complex nonlinear dynamical models to be treated as black boxes. Second, optimisation procedures based on intrusive methods do not necessarily converge in the presence of strong nonlinearities and may therefore require additional convexification techniques (Wang et al., 2019).

In the context of robust low-thrust trajectory optimisation, several uncertainty-propagation techniques

have been employed. Linear covariance propagation was used by Benedikter et al. (2022) and Oguri and McMahon (2021), whereas the UT was employed by Marmo et al. (2023). These approaches necessarily rely on restrictive assumptions, including the local linearity of the dynamics and/or the Gaussianity of the uncertainty distributions throughout the entire time of flight. For this reason, linear methods have already been excluded from further consideration, as discussed previously.

MC sampling (Robert and Casella, 2004) was employed by Blackmore et al. (2010). However, this approach can be highly computationally intensive. To address this limitation in the context of robust space trajectory design, Greco et al. (2022) employed a non-intrusive technique in which the nonlinear dynamical map is approximated through a truncated polynomial expansion. The polynomial map is constructed using stochastic collocation on a sparse Smolyak grid, with Chebyshev polynomials employed for their convergence properties over compact domains. The resulting surrogate map is then used to propagate uncertainty efficiently, avoiding repeated integration of the full nonlinear dynamics for every uncertainty realisation.

By contrast, Greco et al. (2020) employed an intrusive formulation. In this approach, the uncertain state set was represented using polynomial expansions, specifically fourth-degree Chebyshev polynomials in the considered test case, and propagated through the dynamics using polynomial algebra rather than through the direct propagation of individual samples. The resulting polynomial map was then sampled to compute statistical expectations and probabilities.

Although the methods presented by Greco et al. (2020, 2022) have the advantage of accounting for epistemic uncertainties, they still rely on sampling possible realisations of the observations or uncertainties. Consequently, their computational cost may increase rapidly for high-dimensional uncertainty spaces, long trajectories, or problems involving numerous observation arcs. Furthermore, although polynomial approximations accelerate the nonlinear propagation and navigation analysis, they remain approximate representations whose accuracy may deteriorate in the presence of strongly nonlinear dynamics or large uncertainty domains.

Finally, the Low-Order Automatic Domain Splitting (LOADS) method developed by Losacco et al. (2024), and subsequently extended by Caleb (2025), combines Gaussian Mixture Model decomposition with Differential Algebra. When the dynamics are locally close to linear, the method applies linear covariance propagation. When the nonlinearity index exceeds a prescribed threshold, the Gaussian distribution is recursively divided into Gaussian mixture components. Each component is then propagated using a local Taylor map generated through DA. While the framework scales well to moderately complex systems, its runtime for long-duration or high-fidelity scenarios remains a bottleneck. One response could be to parallelise the optimisation over the Gaussian components (mixands), instead of processing them sequentially. Moreover, the dimensionality of the problems that can be addressed is limited due to the factorial complexity of differential algebra operations. Consequently, applying this method to high-dimensional problems appears unrealistic.

5.2.3. SELECTION OF METHODS

The underlying principles, applications, and selection rationale associated with each method have now been analysed. On this basis, a decision can be made regarding the uncertainty-propagation schemes considered most promising for the present work. In general, the following conclusions can be drawn:

- Intrusive methods remain computationally efficient even when a large number of uncertain variables is considered. However, the associated dynamical models are often considerably simplified in order to avoid cumbersome manipulation of the governing equations. For complex dynamics involving discontinuities or piecewise-defined differential equations, an approach such as that proposed by Bergsma and Mooij (2016) is required.
- Non-intrusive methods can be applied directly to complex and realistic dynamical models without requiring simplification. However, because they are affected by the curse of dimensionality, the number of uncertain variables that can be considered is typically limited. This limitation can be mitigated through the use of PDF approximation techniques, such as the Gaussian Mixture Model.

Based on this analysis, preference is given to non-intrusive methods, provided that they are combined with a PDF approximation technique to improve their applicability to realistic problems. In the present work, the selected strategy combines a Gaussian Mixture Model with the Unscented Transform.

The GMM enables non-Gaussian initial uncertainties to be represented as a weighted collection of Gaussian components, thereby rendering the problem compatible with covariance-steering techniques. For each mixture component, the UT propagates the corresponding mean and covariance using a small, deterministic set of sigma points. This avoids both the high sampling cost associated with Monte Carlo methods and

the construction and evaluation of high-dimensional polynomial surrogate models required by PCE-based approaches.

This formulation is also particularly well suited to GPU acceleration, since the propagation of the individual mixture components is independent, while the sigma points associated with each component can likewise be propagated in parallel. Consequently, the GMM–UT formulation naturally exposes two levels of parallelism while remaining compatible with a covariance-steering optimisation framework.

5.3. TRANSCRIPTION METHODS

The probabilistic constraints and costs introduced in Sections 5.1.2.1 and 5.1.2.2 are not directly tractable. Some approaches, such as reinforcement learning (Federici et al., 2021; Holt et al., 2021) and MC sampling (Blackmore et al., 2010; Geletu et al., 2013), act on the stochastic problem directly; however, most stochastic optimisation methods that can handle stochastic costs and constraints rely on a transcription method to transform the stochastic optimisation problem into a deterministic one. These transcription methods achieve this by converting stochastic constraints into tractable deterministic sufficient conditions and are adopted in this work.

Several assumptions are adopted in this thesis. First, each mixture component is assumed to remain approximately multivariate Gaussian throughout the propagation. This assumption will be verified by assessing whether the propagated components remain sufficiently close to Gaussianity. Indeed, the control action is expected to limit the dispersion associated with each component, thereby preventing the corresponding uncertainty distribution from spreading excessively. Second, all functions are assumed to exhibit sufficient regularity.

The transcription methods of robust solvers are designed to enforce hard constraints with a failure risk lower than a given β. Finding a transcription method for constraints of the type of Equations 5.13c and 5.13d, for any $n_y \in \mathbb{N}^*$ and $\boldsymbol{y} \sim \mathcal{N}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma})$, essentially consists of finding a tractable sufficient condition $\mathcal{C}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}, \beta)$:

$$\mathcal{C}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}, \beta) \Longrightarrow \mathbb{P}(\boldsymbol{y} \preceq \mathbf{0}) \geq 1 - \beta \tag{5.15}$$

5.3.1. NORM CHANCE CONSTRAINTS

Certain methods were designed to tackle specific types of chance constraints. One exists for norm constraints such as

$$\mathbb{P}\left(\|\boldsymbol{u}\| \leq u_{\max}\right) \geq 1 - \beta \tag{5.16}$$

Ridderhof et al. (2020) present a first sufficient condition for $\boldsymbol{u}$ to satisfy a control-norm constraint:

$$\|\bar{\boldsymbol{u}}\| + \left[\sqrt{2\ln\left(\frac{1}{\beta}\right)} + \sqrt{n_u}\right] \rho(\boldsymbol{\Sigma}_u) \leq u_{\max} \Longrightarrow \mathbb{P}\left(\|\boldsymbol{u}\| \leq u_{\max}\right) \geq 1 - \beta \tag{5.17}$$

where $\rho(\boldsymbol{\Sigma}_\mathbf{u})$ is the positive square root of the spectral radius of $\boldsymbol{\Sigma}_\mathbf{u}$, i.e., its largest eigenvalue.
They also provide a sharper result for the case $n_u \in \{1, 2\}$:

$$\|\bar{\boldsymbol{u}}\| + \sqrt{2\ln\left(\frac{1}{\beta}\right)} \rho(\boldsymbol{\Sigma}_{\boldsymbol{u}}) \leq u_{\max} \Longrightarrow \mathbb{P}\left(\|\boldsymbol{u}\| \leq u_{\max}\right) \geq 1 - \beta \tag{5.18}$$

To reduce conservatism, Oguri and Lantoine (2022) proposed the following refined result:

$$\|\bar{\boldsymbol{u}}\| + \Psi_{n_u}^{-1}(\beta) \rho(\boldsymbol{\Sigma}_\mathbf{u}) \leq u_{\max} \Longrightarrow \mathbb{P}\left(\|\boldsymbol{u}\| \leq u_{\max}\right) \geq 1 - \beta \tag{5.19}$$

All three methods involve computing the spectral radius, which has a computational complexity of $\mathcal{O}(n_u^3)$ using the Jacobi eigenvalue algorithm (Jacobi, 1846).

5.3.2. SCALAR LINEAR CHANCE CONSTRAINTS

Blackmore et al. (2011) address general scalar linear constraints involving a multivariate Gaussian random variable $\boldsymbol{z} \sim \mathcal{N}(\bar{\boldsymbol{z}}, \boldsymbol{\Sigma}_{\boldsymbol{z}})$ of dimension n_z. Let $\boldsymbol{h} \in \mathbb{R}^{n_z}$ and $a \in \mathbb{R}$. Then, a transcription is given for chance constraints of the form:

$$\mathbb{P}\left(\boldsymbol{h}^T \boldsymbol{z} \leq a\right) \geq 1 - \beta \tag{5.20}$$

with the following result:

$$\boldsymbol{h}^T \bar{\boldsymbol{z}} + \Phi_G^{-1}(1-\beta)\sqrt{\boldsymbol{h}^T \boldsymbol{\Sigma}_{\boldsymbol{z}} \boldsymbol{h}} \leq a \iff \mathbb{P}\left(\boldsymbol{h}^T \boldsymbol{z} \leq a\right) \geq 1-\beta \tag{5.21}$$

This method is widely used in the literature (Farina et al., 2016; Okamoto et al., 2018; Ridderhof et al., 2019; Ozaki et al., 2020; Lew et al., 2020; Oguri and McMahon, 2021; Boone and McMahon, 2022).

Nakka and Chung (2023) use a similar method, but with an even more conservative safety bound:

$$\bar{y} + \sqrt{\frac{1-\beta}{\beta}}\sqrt{\Sigma_y} \leq 0 \implies \mathbb{P}(y \leq 0) \geq 1-\beta \tag{5.22}$$

These always-positive safety bounds force the mean $\bar{y}$ to move further from 0, ensuring that a sufficient portion of the distribution remains negative. For the methods of Blackmore et al. (2011) and Nakka and Chung (2023), the computational complexity is dominated by matrix-vector products, resulting in an overall complexity of $\mathcal{O}(n_u^2)$.

5.3.3. MULTIDIMENSIONAL LINEAR CHANCE CONSTRAINTS

This section presents two first-order transcription methods for general n_y-dimensional linear chance constraints.

The goal is to transcribe a general multi-dimensional inequality constraint such as:

$$\mathbb{P}\left(\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) \preceq \mathbf{0}\right) \geq 1-\beta, \tag{5.23}$$

where $\boldsymbol{f}$ is linear in both the state and the control, with the latter expressed through the linear feedback control policy $\boldsymbol{u} = \bar{\boldsymbol{u}} + \mathbf{K}\Delta\boldsymbol{x}$, as described in detail in Section 5.5. Moreover, we define $\boldsymbol{y} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$. This random variable $\boldsymbol{y}$ is such that $\boldsymbol{y} \sim \mathcal{N}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_{\boldsymbol{y}})$ with:

$$\bar{\boldsymbol{y}} = \bar{\boldsymbol{f}}, \qquad \boldsymbol{\Sigma}_{\boldsymbol{y}} = \left(\left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}})} + \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}})} \mathbf{K}\right) \boldsymbol{\Sigma}_{\boldsymbol{x}} \left(\left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}})} + \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{(\bar{\boldsymbol{x}},\bar{\boldsymbol{u}})} \mathbf{K}\right)^T \tag{5.24}$$

Note that the chance constraint $\mathbb{P}(\mathbf{y} \preceq \mathbf{0}) \geq 1-\beta$ is covered by Eq. (5.21) if $n_y = 1$. The idea of the following results is to generalise this result to any dimension n_y. Let $\rho(\mathbf{A})$ be the positive square root of the spectral radius of $\mathbf{A}$, i.e., its largest eigenvalue. In this context, Caleb (2025) proposes the spectral transcription method and provides its proof:

$$\forall i \in \mathbb{N}_{n_y}^*, \quad \bar{y}_i + \Psi_{n_y}^{-1}(\beta)\rho(\boldsymbol{\Sigma}_y) \leq 0 \implies \mathbb{P}(\boldsymbol{y} \preceq \mathbf{0}) \geq 1-\beta \tag{5.25}$$

This method has cubic computational complexity in n_y, due to the eigenvalue decomposition. As a remark, under the small covariance assumption, the spectral radius transcription method of Equation 5.25 is less conservative than that of Equation 5.19 presented by Oguri and Lantoine (2022) when applied to Gaussian norm constraints ($y = \|\bar{\mathbf{u}}\| - u_{\max}$), and therefore also less conservative than those of Equations 5.17 and 5.18; the reader is referred to the work of Caleb (2025) for the proof.

This sufficient condition of the spectral transcription method is highly conservative; Caleb (2025) therefore presents and proves the first-order transcription method. Let $\boldsymbol{\sigma}_y$ be the vector of the positive square roots of the diagonal coefficients of $\boldsymbol{\Sigma}_y$, then:

$$\bar{\boldsymbol{y}} + \Psi_{n_y}^{-1}(\beta)\,\boldsymbol{\sigma}_y \preceq 0 \implies \mathbb{P}(\boldsymbol{y} \preceq \mathbf{0}) \geq 1-\beta \tag{5.26}$$

Compared to the spectral radius method, this method only requires computing the square roots of n_y terms of a known matrix, therefore, the complexity is $\mathcal{O}(n_y)$.

As demonstrated by Caleb (2025), the transcription of Equation 5.26 is less conservative than that of Equation 5.25, except for the case $n_y = 1$, where these transcriptions are equivalent.

From Equation 5.26, the following proposition can be obtained (Caleb, 2025):

$$\mathbb{P}\left(\boldsymbol{y} \preceq \bar{\boldsymbol{y}} + \Psi_{n_y}^{-1}(\beta)\boldsymbol{\sigma}_y\right) \geq 1-\beta \tag{5.27}$$

This result can be applied to the case where y is the cost function ($n_y = 1$): optimising

$$\bar{\boldsymbol{y}} + \Psi_{n_y}^{-1}(\beta)\boldsymbol{\sigma}_y$$

then minimises a conservative upper bound on the cost that is exceeded with probability no greater than β.

5.3.4. RISK ESTIMATION METHODS

The real risk $\beta_R = 1 - \mathbb{P}(y \preceq 0)$ is useful to assess mission safety, though not always tractable. Indeed, one needs to compute the integral:

$$\mathbb{P}(\boldsymbol{y} \preceq \mathbf{0}) = \int_{(\mathbb{R}^-)^{n_y}} p_G(\boldsymbol{y}; \bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_y)\, \mathrm{d}\boldsymbol{y} \tag{5.28}$$

There is no general closed form for this integral; therefore, numerical methods must be used. One of them is Monte Carlo (MC) estimation. To formulate the MC estimator, the indicator function of the negative orthant is defined as

$$\boldsymbol{I}_{(\mathbb{R}^-)^{n_y}}(\boldsymbol{y}) = \begin{cases} 1, & \text{if } \boldsymbol{y} \in (\mathbb{R}^-)^{n_y} \\ 0, & \text{if } \boldsymbol{y} \notin (\mathbb{R}^-)^{n_y} \end{cases} \tag{5.29}$$

The probability of satisfying the constraint can then be expressed as the expected value of this indicator function:

$$\mathbb{P}\big(\boldsymbol{y} \in (\mathbb{R}^-)^{n_y}\big) = \mathbb{E}\big[\boldsymbol{I}_{(\mathbb{R}^-)^{n_y}}(\boldsymbol{y})\big]\,. \tag{5.30}$$

Given N independent realisations $\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(N)}$, this expectation is approximated by the sample mean:

$$P(\boldsymbol{y} \in (\mathbb{R}^-)^{n_y}) \approx \frac{1}{N} \sum_{j=1}^{N} \boldsymbol{I}_{(\mathbb{R}^-)^{n_y}}\left(\boldsymbol{y}^{(j)}\right) \tag{5.31}$$

Its convergence rate is independent of the dimension n_y and is $\mathcal{O}\big(N^{-1/2}\big)$, where N is the number of samples. However, if the function to integrate is computationally expensive or if the required precision is high, the number of samples to draw can be intractable. To address this, a methodology developed by Caleb (2025) exists to conservatively estimate the failure risk, i.e., to provide an upper bound of β_R. This makes it possible to associate a risk estimate with each transcription method by identifying (when possible) the smallest β_T such that the sufficient condition $\mathcal{C}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_y, \beta_T)$ holds. Since transcriptions correspond to sufficient conditions, this leads to $\beta_T \geq \beta_R$, ensuring that the true risk is not underestimated.

Since the transcription methods developed are of the general form:

$$\mathcal{T}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_y, \beta) \preceq 0 \quad \Longrightarrow \quad \mathbb{P}(\boldsymbol{y} \preceq 0) \geq 1 - \beta, \tag{5.32}$$

the goal is then to find β_T such that

$$\mathcal{T}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_y, \beta_T) = 0. \tag{5.33}$$

In the case of scalar linear chance constraints, as in Equation 5.21 of Blackmore et al. (2011), the failure risk β_R can be computed exactly (Caleb, 2025):

$$\beta_{T,B} = \beta_R = 1 - \Phi_G\left(\frac{a - \boldsymbol{h}^T \bar{\boldsymbol{z}}}{\sqrt{\boldsymbol{h}^T \boldsymbol{\Sigma}_z \boldsymbol{h}}}\right). \tag{5.34}$$

Similarly, for the result of Equation 5.22 derived by Nakka and Chung (2023), if $\bar{y} \leq 0$, then

$$\beta_{T,NC} = \frac{\boldsymbol{h}^T \boldsymbol{\Sigma}_z \boldsymbol{h}}{\boldsymbol{h}^T \boldsymbol{\Sigma}_z \boldsymbol{h} + (a - \boldsymbol{h}^T \bar{\boldsymbol{z}})^2} \geq \beta_R. \tag{5.35}$$

In the same spirit, the failure-risk estimate $\beta_{T,R}$ associated with the methods of Equations 5.17 and 5.18 of Ridderhof et al. (2020) is as follows. If $n_u \in \{1, 2\}$, then

$$\beta_{T,R} = \exp\left[-\frac{1}{2}\left(\frac{\|\bar{\boldsymbol{u}}\| - u_{\max}}{\rho(\boldsymbol{\Sigma}_u)}\right)^2\right] \geq \beta_R. \tag{5.36}$$

Otherwise, if $n_u > 2$ and

$$\left(\frac{\|\bar{\boldsymbol{u}}\| - u_{\max}}{\rho(\boldsymbol{\Sigma}_u)} + \sqrt{n_u}\right) \leq 0,$$

then

$$\beta_{T,R} = \exp\left[-\frac{1}{2}\left(\frac{\|\bar{\boldsymbol{u}}\| - u_{\max}}{\rho(\boldsymbol{\Sigma}_u)} + \sqrt{n_u}\right)^2\right] \geq \beta_R. \tag{5.37}$$

No failure risk estimation is available when $n_u > 2$ and

$$\left(\frac{\|\bar{\boldsymbol{u}}\| - u_{\max}}{\rho(\boldsymbol{\Sigma}_u)} + \sqrt{n_u}\right) > 0,$$

since in that case there is no real solution to the equation

$$-\left(\frac{\|\bar{\boldsymbol{u}}\| - u_{\max}}{\rho(\boldsymbol{\Sigma}_u)} + \sqrt{n_u}\right) = \sqrt{2\log(1/\beta_T)}$$

with respect to the unknown β_T. For the less conservative method of Oguri and Lantoine (2022), described in Equation 5.19, a failure-risk estimate is also obtained. If $\|\bar{\boldsymbol{u}}\| \leq u_{\max}$, then

$$\beta_{T,OL} = \Psi_{n_u}\left(\frac{u_{\max} - \|\bar{\boldsymbol{u}}\|}{\rho(\boldsymbol{\Sigma}_u)}\right) \geq \beta_R. \tag{5.38}$$

Failure-risk estimates are also available for multidimensional chance constraints. First, for the spectral-radius transcription method of Equation 5.25, it can be derived that, if $\bar{\boldsymbol{y}} \preceq 0$, then

$$\beta_{T,\rho} = \Psi_{n_y}\left[\frac{\min(-\bar{\boldsymbol{y}})}{\rho(\boldsymbol{\Sigma}_y)}\right] \geq \beta_R. \tag{5.39}$$

Secondly, for the first-order transcription method introduced in Equation 5.26, the corresponding failure-risk estimate can be obtained as follows. If $\bar{\boldsymbol{y}} \preceq \mathbf{0}$, then

$$\beta_{T,1} = \Psi_{n_y}\left(\min_{i=1,\ldots,n_y} r_i\right) \geq \beta_R. \tag{5.40}$$

The vector $\boldsymbol{r}$ is defined as follows. Let $\boldsymbol{y} \sim \mathcal{N}(\bar{\boldsymbol{y}}, \boldsymbol{\Sigma}_y)$. The vector of signed Mahalanobis distances (Mahalanobis, 1936) from the mean $\bar{\boldsymbol{y}}$ to the n_y constraint hyperplanes $\mathcal{H}_i$ is given by

$$\boldsymbol{r} = \begin{bmatrix} r_1 & r_2 & \cdots & r_{n_y} \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} -\dfrac{\bar{y}_1}{\sigma_{y,1}} & -\dfrac{\bar{y}_2}{\sigma_{y,2}} & \cdots & -\dfrac{\bar{y}_{n_y}}{\sigma_{y,n_y}} \end{bmatrix}^{\mathrm{T}}. \tag{5.41}$$

Caleb (2025) proposes a higher-order method for estimating the failure risk, referred to as the n_y-th-order failure-risk estimation method. If $\bar{\boldsymbol{y}} \preceq \mathbf{0}$, the corresponding risk estimate is given by

$$\beta_{T,n_y} = 1 - \sum_{i=1}^{n_y} \Delta\Psi_{n_y}\left(\tilde{r}_{i-1}, \tilde{r}_i\right) \max\left\{0, 1 - \frac{1}{2}\sum_{j=1}^{i-1} \mathcal{A}\left(\frac{\tilde{r}_j}{\tilde{r}_i}\right)\right\} \geq \beta_R. \tag{5.42}$$

Here,

$$\tilde{\boldsymbol{r}} = \begin{bmatrix} \tilde{r}_1 & \tilde{r}_2 & \cdots & \tilde{r}_{n_y} \end{bmatrix}^{\mathrm{T}} \tag{5.43}$$

denotes the vector containing the Mahalanobis distances $\boldsymbol{r}$ arranged in increasing order. Furthermore,

$$\Delta\Psi_{n_y}\left(\tilde{r}_0, \tilde{r}_1\right) = \Psi_{n_y}(\tilde{r}_0) - \Psi_{n_y}(\tilde{r}_1). \tag{5.44}$$

The auxiliary function $\mathcal{A}(x)$ is defined as

$$\mathcal{A}(x) = A_{\sqrt{1-x}}\left(\frac{n_y - 1}{2}, \frac{1}{2}\right), \tag{5.45}$$

where $A_x(a, b)$ denotes the regularised incomplete Euler β-function:

$$A_x(a, b) = \frac{1}{B(a, b)} \int_0^x t^{a-1}(1-t)^{b-1}\,\mathrm{d}t, \tag{5.46}$$

and $B(a, b)$ is the Euler β-function.

For $n_y = 1$, the spectral-radius, first-order, and n_y-th-order methods are equivalent. This failure-risk estimate directly yields an additional chance-constraint transcription. If there exists an index $j \in \mathbb{N}^*_{n_y}$ such that $\bar{y}_j \neq 0$, then

$$\begin{cases} \beta_{T,n_y} |\bar{y}_j| \leq -\beta \bar{y}_j, \\ \bar{y}_i \leq 0, \qquad \forall i \in \mathbb{N}^*_{n_y} \setminus \{j\}, \end{cases} \implies \quad \mathbb{P}(\boldsymbol{y} \preceq \mathbf{0}) \geq 1 - \beta. \tag{5.47}$$

Computing β_{T,n_y} requires at most $\mathcal{O}(n_y^2)$ operations. Consequently, the resulting transcription method has a computational complexity of $\mathcal{O}(n_y^2)$. Moreover, evaluating β_{T,n_y} requires the computation of relatively complex special functions. Therefore, the numerical conditioning of this transcription is less favourable than that of the spectral-radius and first-order transcriptions.

5.3.5. MEASURE OF CONSERVATISM

The aim of this section is to provide methods to compare the various transcription methods; the focus is therefore on $n_y > 1$, since the spectral-radius, first-order, and n_y-th-order transcriptions are the same when $n_y = 1$. As shown in Section 5.3.4, each method comes with a failure risk estimation. Comparing this estimation to the real risk $\beta_R = 1 - \mathbb{P}(\boldsymbol{y} \preceq \mathbf{0})$ computed using an MC method (Robert and Casella, 2004) gives a measure of the conservatism of a risk estimation associated with a transcription. The conservatism $\gamma(\beta_T)$ of a transcription method compares a given estimation of the failure risk β_T with the real risk β_R (Caleb, 2025). It is defined as:

$$\gamma(\beta_T) = \frac{\beta_T}{\beta_R} \sqrt{\frac{1 - \beta_R^2}{1 - \beta_T^2}}.$$

The function γ is monotonically increasing. Therefore, since by design of a transcription method $\beta_T \geq \beta_R$, it follows that $\gamma(\beta_T) \geq 1$. Transcription methods can be compared using this value: a transcription is less conservative than another if its conservatism is closer to 1 than the other. When $1 - \beta_T \ll 1$ but $\beta_R \ll 1$, the estimator does not bring any information and is penalised by a high conservatism. Moreover, we have $\beta_R \ll 1 \wedge \beta_T \ll 1 \implies \gamma(\beta_T) \sim \frac{\beta_T}{\beta_R}$. Thus, it is a good measure of the overestimation of the risk and will be a tool to compare transcription methods. The quantities γ_ρ, γ_1, and γ_{n_y} are defined as the conservatism values of the spectral-radius, first-order, and n_y-th-order methods, respectively. If $\bar{\boldsymbol{y}} \preceq \mathbf{0}$, then

$$1 \leq \gamma_{n_y} \leq \gamma_1 \leq \gamma_\rho, \tag{5.48}$$

as demonstrated by Caleb (2025).

With regard to norm chance constraints, the transcription method introduced in Equation 5.19 is selected. Unlike the formulations presented in Equations 5.20, 5.22, and 5.26, it does not rely on a small-covariance assumption or on a linearisation of the norm. The latter formulations are therefore less general, since the validity of the linear approximation cannot be guaranteed for all cases considered in the present study. Furthermore, for $n_y = 1$, the first-order, d-th-order, and spectral-radius transcription methods are equivalent. Moreover, the method of Equation 5.19 is also less conservative than those of Equations 5.17 and 5.18.

A different selection rationale applies to multidimensional linear chance constraints. For multivariate Gaussian distributions, the numerical investigations conducted by Caleb (2025) indicate that the n_y-th-order transcription outperforms the spectral-radius and first-order methods, particularly in high-dimensional settings, where the latter may fail to provide meaningful results. Furthermore, the study demonstrates that spectral-radius-based transcription methods are overly conservative and computationally demanding, primarily because they require eigenvalue computations, resulting in a computational complexity of $\mathcal{O}(n_y^3)$. By contrast, the first-order and n_y-th-order methods avoid these limitations by relying solely on scalar-function evaluations and square-root computations, yielding computational complexities of $\mathcal{O}(n_y)$ and $\mathcal{O}(n_y^2)$, respectively. These lower computational requirements render both methods more practical and efficient.

In the present thesis, since the state-space dimension is $n_y = 6$, the first-order method will be adopted as the primary approach for transcribing multidimensional linear inequality constraints. Even if, under Gaussian assumptions, it is more conservative than the general n_y-th-order method, as demonstrated by Caleb (2025), this additional conservatism is expected to be beneficial in the present setting. Indeed, the dynamics considered in this work are highly nonlinear, and initially Gaussian mixture components may therefore deviate slightly from Gaussianity during propagation. A moderately more conservative transcription may consequently provide a safer and more robust approximation than the tighter n_y-th-order method. Nevertheless, both methods will be implemented, tested, and compared.

5.3.6. FAILURE RISK ALLOCATION METHODS

Theoretical results that justify the use of risk estimation on GMMs are presented in this section. The remainder of this section details how the nonlinear problem is decomposed into several subproblems.

In this section, we consider a random variable $\boldsymbol{y} \sim \mathrm{GMM}(\boldsymbol{\alpha}, \bar{\boldsymbol{\mu}}, \boldsymbol{S})$ of M mixands. The objective is to satisfy the non-Gaussian chance constraint: $\mathbb{P}(\boldsymbol{y} \preceq 0) \geq 1 - \beta$.

For each Gaussian mixand $\boldsymbol{y}_j \sim \mathcal{N}(\bar{\boldsymbol{\mu}}_j, \boldsymbol{S}_j)$, we define $\beta_{T,j}$ a conservative (tractable) estimation of the failure risk, $\beta_{R,j}$ the true (intractable) failure risk, and β_R the global failure risk of the GMM. It can easily be derived that (Caleb, 2025):

$$\beta_R = \sum_{j=1}^{M} \alpha_j \beta_{R,j}. \tag{5.49}$$

From this result, we observe that if each mixand is such that $\beta_{T,j} \leq \beta$ and if we define $\beta_T = \sum_{j=1}^{M} \alpha_j \beta_{T,j}$, then $\beta_T \leq \sum_{j=1}^{M} \alpha_j \beta = \beta$. Moreover, since by design $\beta_{R,j} \leq \beta_{T,j}$, we obtain $\beta_R \leq \beta$. Therefore, ensuring that each mixand individually satisfies its chance constraint guarantees that the entire GMM satisfies the global chance constraint.

In sequential optimisation, the target failure risk β can be dynamically relaxed to reduce conservatism. For instance, if all $\alpha_j = 1/M$ and the first mixand satisfies $\beta_{T,1} = 0$, then the constraint on the second mixand can be relaxed to $\mathbb{P}(y_2 \preceq 0) \geq 1 - 2\beta$, while still ensuring $\beta_R \leq \beta$. In other words, unused failure risk margins from previous mixands can be reallocated to subsequent ones, facilitating a more efficient optimisation process.

This idea generalises to alternative risk allocation strategies. The naive approach of risk estimate β_T^n enforces $\forall j \in \mathbb{N}_M^*$, $\beta_{T,j} \leq \beta$. However, it is possible to design methods where $\forall j \in \mathbb{N}_M^*$, $\beta_{T,j} \leq \beta_j^*$ with $\beta_j^* \geq \beta$, while still guaranteeing $\beta_R \leq \beta$. In such cases, the risk estimate satisfies $\beta_T \geq \beta_T^n$, allowing for higher flexibility and less conservative solutions. Nonetheless, these techniques will not be explored in this work, which focuses solely on multi-core CPU and GPU computing and is therefore not concerned with sequential optimisation.

5.4. DETERMINISTIC TRAJECTORY OPTIMISATION SOLVERS

A comprehensive and systematic classification of deterministic trajectory optimisation methods is provided by Morante et al. (2021).

In general, these approaches can be classified into two broad categories: analytical and numerical methods. Analytical approaches yield closed-form solutions for the optimal trajectory. However, because such solutions can only be derived for specific cases, analytical methods are rarely applicable to most spacecraft trajectory optimisation problems. Consequently, the majority of research efforts have focused on numerical methods, which enable the solution of more realistic and practically relevant problems.

Numerical approaches can be divided into three well-established categories: indirect methods, direct methods, and dynamic programming. Indirect methods are based on Pontryagin's minimum principle (PMP), dynamic programming relies on Hamilton–Jacobi–Bellman theory, and direct methods are associated with the Karush–Kuhn–Tucker (KKT) optimality conditions. Furthermore, each approach results in a distinct mathematical problem, which can be solved using gradient-based, heuristic, or hybrid optimisation techniques. Each combination presents specific advantages and limitations. In the following, a brief overview of these approaches and their associated solution techniques is provided.

5.4.1. INDIRECT, DIRECT, AND DYNAMIC PROGRAMMING APPROACHES

The three principal categories of methods employed in low-thrust trajectory optimisation are summarised below:

- **Indirect approach:** The objective of an indirect approach is to solve the multipoint boundary-value problem (MPBVP) resulting from the application of Pontryagin's minimum principle (PMP) Pontryagin (1962). The PMP characterises the first-order necessary conditions that an optimal solution must satisfy. Its derivation involves determining the state and costate variables, which must satisfy the Euler–Lagrange equations. In particular, the minimum principle makes it possible to determine the continuous control law as a function of the state and costate variables at each instant, either explicitly or numerically. Furthermore, a set of additional conditions, including transversality and complementarity conditions, must be satisfied (Bryson et al., 1979).

- **Direct approach:** The fundamental principle underlying direct methods is the transcription of the continuous optimal control problem (COCP) into a nonlinear programming problem (NLP), in which the objective function is optimised directly. The transcription process requires the control variables to be discretised over a temporal grid. The objective of the resulting NLP is to determine a vector of unknown decision variables that satisfies a set of nonlinear equality and inequality constraints. An optimal solution to the NLP must satisfy the corresponding first-order necessary optimality conditions, known as the Karush–Kuhn–Tucker (KKT) conditions (Karush, 1939; Kuhn and Tucker, 1951). The NLP is subsequently solved numerically using established optimisation techniques (Rao, 2009).
- **Dynamic programming approach:** Dynamic programming is based on Bellman's principle of optimality (Bellman and Kalaba, 1965): "An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision." Although dynamic programming was originally developed for discrete-time systems, it was subsequently extended to continuous-time problems. The continuous-time counterpart of Bellman's principle leads to the Hamilton–Jacobi–Bellman (HJB) equation (Caputo, 2005). In this case, the corresponding partial differential equation, or system of partial differential equations, must first be solved.

Indirect and direct methods typically employ one of the following techniques to impose the governing dynamical equations:

- **Single shooting:** The trajectory is propagated from t_0 to the final time t_f using a time-marching integration method. In this formulation, the required initial values of the state and, where applicable, the costate variables are treated as unknowns to be determined, while the boundary constraints are imposed at the end of the integration interval.
- **Multiple shooting:** The time interval $[t_0, t_f]$ is divided into $N + 1$ subintervals. The trajectory is propagated independently over each subinterval $[t_i, t_{i+1}]$, with the initial values of the state and, where applicable, the adjoint variables at each subinterval treated as unknown decision variables. Continuity conditions must additionally be imposed at the interfaces between consecutive subintervals.
- **Collocation:** The state and, where applicable, costate variables are discretised over a predefined temporal grid and are therefore represented only at a finite number of points. The governing differential equations are transcribed into discrete defect constraints, which relate the variable values at the beginning and end of each subinterval. Different collocation methods are characterised by the quadrature rules used to approximate the differential equations between consecutive grid points and can generally be classified as local or global collocation methods.

Indirect methods may also be solved using gradient-based techniques involving forward integration of the dynamical system and backward integration of the adjoint equations. To perform the forward integration and initialise the adjoint variables, an initial time-dependent control function must be prescribed. These unknown quantities constitute the decision variables and are iteratively varied until the necessary optimality conditions are satisfied. Depending on the procedure employed to update the control variables, first-order (Gottlieb, 1967) and second-order (Longmuir and Bohn, 1969) gradient methods can be distinguished.

A further technique belonging to the direct-method category is differential inclusion. In this formulation, only the state variables are discretised over a predefined temporal grid. The equations of motion are imposed at each discrete time point through inequality constraints on the state rates of change. These inequalities are obtained by substituting the upper and lower bounds of the control vector into the equations of motion.

For dynamic programming, one of the most successful techniques is Differential Dynamic Programming (DDP) (Mayne, 1973). DDP is a second-order, gradient-based technique founded on the HJB equation and the successive minimisation of quadratic approximations of the original problem. The algorithm proceeds iteratively by performing a backward pass along the nominal trajectory to generate an updated control sequence, followed by a forward pass to propagate the resulting trajectory and evaluate its associated cost.

5.4.2. GRADIENT-BASED, HEURISTIC, AND HYBRID SOLUTION METHODS

Most of the approaches introduced previously transform the continuous optimal control problem (COCP) into the problem of determining an unknown vector of decision variables. In direct methods, this decision vector must satisfy a set of nonlinear constraints while minimising an objective function, thereby defining a nonlinear programming problem (NLP). In indirect methods, by contrast, the unknown parameters must satisfy a system of nonlinear equations and boundary conditions, resulting in a multipoint boundary-value problem (MPBVP).

The numerical algorithms used to solve NLPs and MPBVPs can generally be classified as gradient-based, heuristic, or hybrid methods. All three categories employ iterative procedures, but differ in the rules used to update the candidate solutions. Their principal characteristics are summarised below:

- **Gradient-based methods:** In a gradient-based method, an initial estimate of the unknown decision vector $\boldsymbol{z}$ is first prescribed. At the k^{th} iteration, a search direction $\boldsymbol{p}_k$ and a step length α_k are determined. The search direction defines the direction in $\mathbb{R}^{n_z}$ along which the current estimate $\boldsymbol{z}_k$ is modified, whereas the step length determines the magnitude of this modification. The decision vector is updated according to

$$\boldsymbol{z}_{k+1} = \boldsymbol{z}_k + \alpha_k \boldsymbol{p}_k. \tag{5.50}$$

  The iterative procedure continues until the relevant convergence and Karush–Kuhn–Tucker conditions are satisfied to within a prescribed tolerance. To determine the search direction, these methods require derivative information concerning the objective function and the constraints, where applicable. The most widely used gradient-based algorithms include sequential quadratic programming methods, implemented in solvers such as SNOPT and NPSOL, and interior-point methods, implemented in solvers such as IPOPT and KNITRO. Detailed descriptions of their respective implementations are provided in (Gill, 2008; Wächter and Biegler, 2006).

- **Heuristic methods:** In heuristic methods, the search is conducted using stochastic or metaheuristic procedures and does not require gradient information. Evolutionary algorithms constitute one of the most widely used classes of heuristic techniques. These algorithms begin by generating a set of candidate solutions, or individuals,

$$z_{i,0}, \qquad i = 1, \ldots, n,$$

  collectively referred to as the population. The population is then iteratively modified using a set of stochastic rules

$$\Pi : \mathcal{Z} \to \mathcal{Z},$$

  which may incorporate random processes. Accordingly, the population at the $(k+1)^{\text{th}}$ iteration is generated according to

$$z_{i,k+1} = \Pi(z_{i,k}).$$

  The iterations continue until a prescribed stopping criterion is satisfied, such as a maximum number of iterations. The feasible candidate solution associated with the lowest objective-function value is then selected as the solution to the problem.

- **Hybrid methods:** Hybrid approaches combine gradient-based update rules with heuristic search mechanisms to operate iteratively on either a single solution or a population of candidate solutions. Gradient-information is exploited to drive the constraints to zero, while heuristic rules are applied to efficiently explore large design domains or to manage integer variables. These methods are commonly implemented using a nested two-level structure. A heuristic solver operates on a subset of the decision variables in the outer loop, while the remaining design variables are optimised using a gradient-based method in the inner loop.

5.4.3. SOLVER SELECTION

The principal advantage of indirect methods is that they directly enforce the first-order necessary conditions for optimality. Indirect formulations may also provide valuable theoretical insight into the physical and mathematical characteristics of the problem. However, they offer limited flexibility because explicit derivations of the costate equations, optimal control law, and associated boundary conditions are required. Depending on the optimal control problem under consideration, these derivations may be particularly difficult.

The numerical techniques used to solve the resulting MPBVP generally require an appropriate initial estimate of the costate variables. Such an estimate is often difficult to construct because costates do not usually possess an intuitive physical interpretation. Moreover, indirect formulations may exhibit limited numerical robustness, since the resulting trajectory can be highly sensitive to variations in the initial costates. The indirect approach is further complicated by the need to reformulate the MPBVP whenever different state variables, constraints, or dynamical models are introduced. Owing to these practical difficulties, indirect methods are generally unsuitable for highly constrained problems or for applications in which robustness, flexibility, or automation is required.

Direct methods, by contrast, do not require the user to derive the first-order necessary conditions explicitly. They are also generally easier to initialise because they tend to possess a larger domain of convergence and employ optimisation variables with more intuitive physical interpretations. Although direct methods still require an initial estimate and may fail to converge to the true optimum, they can often return feasible suboptimal solutions in cases where an indirect method fails to converge. This property can be particularly valuable in concurrent engineering environments.

A further advantage of direct methods is their ability to accommodate complex state and control constraints. In the presence of path inequality constraints, the sequence of unconstrained and constrained trajectory arcs does not need to be specified a priori. Their principal limitation is that the solution of the resulting NLP is not necessarily an exact optimal solution of the original COCP. Consequently, it may remain uncertain whether the computed trajectory represents a true local optimum of the continuous problem or only a suboptimal solution of its discretised approximation.

Dynamic programming offers two principal advantages over the other approaches considered. First, an exact dynamic programming formulation searches the complete state space and can therefore recover a globally optimal solution. Second, once the problem has been solved, the optimal control is available as a function of the state. Dynamic programming can consequently produce a closed-loop control policy rather than a single open-loop control history. This characteristic also enables the method to be extended naturally to uncertain and stochastic problems.

The principal limitation of dynamic programming is the curse of dimensionality. The memory requirements and computational cost of conventional dynamic-programming methods increase rapidly with the number of state variables and become impractical for high-dimensional state spaces. Their direct application is therefore generally restricted to problems with relatively low state-space dimensionality. The curse of dimensionality can be mitigated through approximate techniques based on local representations of the value function, such as Differential Dynamic Programming. However, the resulting solution is no longer guaranteed to be globally optimal, and the associated closed-loop control policy is valid only locally.

Regarding the numerical solution algorithms, gradient-based methods provide deterministic convergence properties under suitable regularity assumptions. They can efficiently handle large numbers of decision variables and constraints. However, they generally require the objective function and constraints to be sufficiently smooth and, depending on the algorithm, continuously differentiable or twice differentiable. Consequently, these methods are less suitable for problems involving discontinuities, non-smooth functions, or tabulated data.

Gradient-based methods also require an initial estimate located sufficiently close to the region of attraction of a solution. They may encounter difficulties when searching for a global optimum over a broad design domain and are generally unable to explore an entire multi-objective design space within a single optimisation run.

Heuristic methods are particularly suitable for problems involving a relatively limited number of decision variables distributed over a broad or highly multimodal design domain. They do not necessarily require an initial estimate, which facilitates automation of the optimisation process. Whereas a gradient-based method generally performs a local search, a heuristic algorithm is intended to conduct a more global exploration of the design domain. Heuristic methods are also more flexible because they do not require the objective and constraint functions to be differentiable.

However, heuristic algorithms do not provide deterministic guarantees of optimality. Different runs may produce different solutions because of their stochastic nature. Furthermore, constraints may be difficult to satisfy accurately because gradient information is not used explicitly to reduce constraint violations.

Hybrid approaches provide intermediate performance in terms of flexibility, robustness, and optimality when compared with purely deterministic and purely heuristic methods. They exploit several advantages of heuristic algorithms, including their potential for automation, their ability to handle integer variables, and their capacity to search efficiently over large and multi-objective design domains. At the same time, they compensate for the limited constraint-handling capability of heuristic methods by incorporating a gradient-based local optimisation procedure.

Within this methodological landscape, the selection of an optimisation approach must be consistent with the objectives and requirements of this work. Table 5.1 provides a qualitative comparison of indirect methods, direct methods, and dynamic programming according to four weighted criteria: flexibility, robustness, optimality, and parallelisability. Each criterion is assigned a grade ranging from 1 to 5, where 1 indicates poor performance and 5 indicates excellent performance. The overall score is calculated using weights of 0.2 for flexibility, robustness, and optimality, and 0.4 for parallelisability, reflecting the scope and requirements of

Table 5.1: Comparison of optimisation methods using weighted qualitative grades ranging from 1 (poor) to 5 (excellent).

Approach	Flexibility	Robustness	Optimality	Parallelisability	Score
Indirect	1	1	4	2	2.00
Direct	5	4	3	4	4.00
Dynamic Programming	2	4	5	4	3.80

the present study.

Based on these scores, a direct approach is selected. More specifically, the Trajectory Bundle Method proposed by Tracy et al. (2025), which is introduced in Chapter TBD, will be employed to solve a fuel-optimal control problem. The resulting solution will subsequently be used as a warm start for the robust optimisation problem, which will be addressed using a direct hybrid multiple-shooting approach formulated by Marmo and Zavoli (2024) and presented in Section 5.5.

5.5. PROBLEM FORMULATION

In this section, a comprehensive overview of the envisioned robust trajectory-optimisation problem is presented. This entails a detailed elucidation of its key components: the constraints on the initial and final state distributions, the applied control, and the criteria guiding the minimisation of the cost function.

The spacecraft is modelled as a point-mass object, and its state $\boldsymbol{x}'(t) \in \mathbb{R}^6$ is defined in terms of the Cartesian position and velocity components; the mass $m(t) \in \mathbb{R}_{\geq 0}$ is also included as a state variable. Thus, hereafter the augmented state $\boldsymbol{x}(t) = [\boldsymbol{x}'(t)\ m(t)]^T \in \mathbb{R}^n$, with $n_x = 7$, composed of the Cartesian state and mass, is adopted.

The initial and final transfer epochs are denoted as t_i and t_f, respectively. The trajectory is assumed to be composed of N arcs of equal duration, where the thrust remains constant in both magnitude and direction throughout each arc. A uniform time grid is thus defined as

$$\tilde{t}_i = t_0 \leq t_1 \leq \cdots \leq t_N = \tilde{t}_f \tag{5.51}$$

having $\Delta t_k = t_k - t_{k-1} = (\tilde{t}_f - \tilde{t}_i)/N = \Delta T/N$, $\forall k = 1, \ldots, N$, where the tilde denotes constant, user-defined quantities. The imparted acceleration $\boldsymbol{u}_k \in \mathbb{R}^3$ is considered as the specific thrust having

$$\boldsymbol{u}_k = \frac{\boldsymbol{T}_k}{m(t)} \qquad \forall t \in [t_{k-1}, t_k] \qquad \forall k = 1, \ldots, N \tag{5.52}$$

where $\boldsymbol{T}_k \in \mathbb{R}^3$ is the imparted thrust from the engines, thus a zero-order-hold (ZOH) discretisation is employed.

The dynamics of a system perturbed by a stochastic disturbance term are described by the system of stochastic differential equations (SDEs) defined in Equation 3.58.

In the context of the problem under consideration, the diffusion matrix will be modelled as

$$\boldsymbol{g}\big(\boldsymbol{x}(t), \boldsymbol{u}(t), t\big) = \begin{bmatrix} (\sigma_{r,w})^2 \mathbb{I}_3 & \mathbf{0}_{3\times 3} & \mathbf{0}_{3\times 1} \\ \mathbf{0}_{3\times 3} & (\sigma_{v,w})^2 \mathbb{I}_3 & \mathbf{0}_{3\times 1} \\ \mathbf{0}_{1\times 3} & \mathbf{0}_{1\times 3} & (\sigma_{m,w})^2 \end{bmatrix} \quad \forall t \in [t_{k-1}, t_k] \quad \forall k = 1, \ldots, N \tag{5.53}$$

where $\sigma_{r,w}$, $\sigma_{v,w}$ and $\sigma_{m,w}$ model the variance of the position, velocity, and mass random disturbance, respectively.

The spacecraft state at the initial time is assumed to follow a multivariate Gaussian distribution:

$$\boldsymbol{x}_0 \sim \mathcal{N}\left(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0\right), \tag{5.54}$$

where $\boldsymbol{\mu}_0 \in \mathbb{R}^7$ and $\boldsymbol{\Sigma}_0 \in \mathbb{R}^{7\times 7}$ denote the initial mean vector and covariance matrix, respectively. The problem formulation will be extended, at the end of this section, to remove the Gaussianity assumption.

At the arrival epoch t_f, the mean position and velocity are constrained to a target value; a bound on the state dispersion is also enforced as a constraint on the final position and velocity distribution. Thus,

$$\mathbb{E}[\boldsymbol{x}'_N] = \boldsymbol{\mu}'_N = \tilde{\boldsymbol{\mu}}'_N = [\tilde{\boldsymbol{\mu}}^r_N, \tilde{\boldsymbol{\mu}}^v_N] \tag{5.55}$$

$$\boldsymbol{\Sigma}'_N \preceq \tilde{\boldsymbol{\Sigma}}'_N \tag{5.56}$$

Here, the notation $\boldsymbol{A} \succeq \mathbf{0}$ indicates that $\boldsymbol{A}$ is a positive semi-definite (PSD) matrix.

A closed-loop (CL) feedback control $\boldsymbol{u}_k = \boldsymbol{u}(\boldsymbol{x}_k)$, which depends on the spacecraft state $\boldsymbol{x}_k$, is considered. The thrust imparted by the engine is bounded in magnitude as $\|\boldsymbol{T}_k\| \leq T_{\max}$, with $T_{\max}$ as a mission parameter of user choice. However, from the definition of the feedback control law, it is apparent that $\boldsymbol{T}$ is a random variable, as it depends on a state (or possibly a measurement), which is also random (Marmo et al., 2023). Therefore, the bounded-magnitude impulse constraint is supposed to hold in a probabilistic sense, as

$$\Pr\left\{\|\boldsymbol{T}_k\| \leq T_{\max}\right\} \geq 1 - \beta \qquad \forall k = 0, \ldots, N \tag{5.57}$$

where the problem parameter $\beta \in [0, 1]$ is the probability that the constraint is violated.

The aim of the optimisation is to minimise the propellant consumption, which, under the assumption of a uniform time grid and constant I_{sp}, is proportional to the cumulative thrust magnitude:

$$\min_{\boldsymbol{T}_k} \; J = \sum_{k=0}^{N} \|\boldsymbol{T}_k\| \tag{5.58}$$

As in the case of the control magnitude constraint, the expression of Equation (5.58) must be reformulated, as the CL control is a random variable. Aiming hence at optimising for the worst case, a suitable metric is the minimisation of the p-th percentile $Q_\Phi(p)$ of the distribution of the random variable $\Phi = \sum_{k=0}^{N} \|\boldsymbol{T}_k\|$, where $p = 100(1 - \alpha)$, having $\alpha \in [0, 1]$ as the violation parameter.

The goal of the problem is to find a CL control law that makes it possible to reach the target in a given time t_f within a certain level of accuracy, while minimising the control effort, that is, the cumulative control magnitude, and ensuring compliance with the thrust-magnitude constraint. The general ROCP is therefore formulated as

$$\min_{\boldsymbol{T}_k} \quad Q_\Phi(p) \tag{5.59a}$$

$$\text{s.t.:} \quad (3.58),\ (5.54) - (5.57) \tag{5.59b}$$

with user-defined parameters α and β.

As for the control, the choice of the feedback controller is restricted to the class of linear Markov control policies. Moreover, the state vector cannot be perfectly known; in reality, the controller depends on navigation measurements $\boldsymbol{y}_k$ of the spacecraft state $\boldsymbol{x}'_k$. For the sake of simplicity, the case is considered where the navigation measurement $\boldsymbol{y}_k = \boldsymbol{y}(t_k)$ is affine in the state and corrupted by some zero-mean additive Gaussian noise $\boldsymbol{\eta}_k \sim \mathcal{N}(\mathbf{0}, \tilde{\mathbf{R}})$, so that

$$\mathbf{y}_k = \mathbf{H}\mathbf{x}'_k + \boldsymbol{\eta}_k \qquad \forall k = 0, \ldots, N \tag{5.60}$$

While quite simple, this model finds applications in a number of cases of practical interest, where $\mathbf{y}$ represents the state estimate returned by a (possibly autonomous) navigation filter and $\tilde{\mathbf{R}}$ is the covariance of the estimation error. In this study, $\boldsymbol{H}$ is the identity matrix. Alternatively, the estimated state $\boldsymbol{y}_k$ could be substituted by a vector of measurements of the state (e.g., range and range-rate measurements), resulting in a general nonlinear form $\boldsymbol{y}_k = \boldsymbol{h}(\boldsymbol{x}'_k)$. However, this general formulation is beyond the scope of this work, and thus $\boldsymbol{y}_k$ denotes the filtered state. Hereafter, with a slight abuse of notation, $\boldsymbol{y}$ is simply referred to as the measurement and to $\boldsymbol{\eta}$ as the measurement error.

The control $\boldsymbol{u}_k$ is defined through a CL control law within the class of linear Markov feedback control policies as

$$\boldsymbol{T}_k = \boldsymbol{S}_k + K_k\left(\boldsymbol{y}_k - \mathbb{E}\left[\boldsymbol{y}_k\right]\right) \qquad \forall k = 0, \ldots, N \tag{5.61}$$

where $\boldsymbol{S}_k \in \mathbb{R}^3$ represents the feedforward, i.e. open-loop (OL), control signal, while the feedback term $\boldsymbol{K}_k\left(\boldsymbol{y}_k - \mathbb{E}[\boldsymbol{y}_k]\right)$ accounts for the difference between the expected and measured states. Finally, by recalling Equation 5.60, the control is rewritten as

$$\boldsymbol{T}_k = \boldsymbol{S}_k + \boldsymbol{K}_k\left(\boldsymbol{y}_k - \mathbf{H}\boldsymbol{\mu}'_k\right) \qquad \forall k = 0, \ldots, N \tag{5.62}$$

having the control law parameterised by the OL control terms $\boldsymbol{S}_k$ and the feedback control gains $\boldsymbol{K}_k \in \mathbb{R}^{3\times 6}$.

Note that the trajectory is guided towards the reference mean by leveraging only position and velocity as driving factors, rather than the full augmented state $\boldsymbol{x}(t)$, which would also comprise the mass $m(t)$.

Given the closed-loop (CL) control expression for the thrust vector in Equation 5.62, a further limitation of linear uncertainty-propagation methods arises when estimating the terminal state. This limitation results from

combining the equations of motion in Equation 3.58 with the control law specified in Equation 5.62, which introduces a term that depends on the realised initial state and therefore renders the equations of motion non-unique. This characteristic increases the complexity of propagating the stochastic moments compared with the more straightforward case of a purely ballistic arc. The dependence on the initial-state realisation makes many conventional techniques either infeasible or exceedingly difficult to apply in this scenario. Therefore, to propagate effectively the first two stochastic moments of both the state and the CL control, as required to solve the problem formulated in Equation 5.59a, this work employs a modified version of the Unscented Transform (UT), as described in Section 5.5.2

The propagation of the mean state and of the covariance matrices of the state $\boldsymbol{\Sigma}_k$ and thrust vector $\boldsymbol{\Sigma}_k^T$, explained in detail in Section 5.5.2, is summarised by the expression

$$\left\{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \boldsymbol{\Sigma}_k^T\right\} = \mathcal{F}\left(\boldsymbol{\mu}_{k-1}, \boldsymbol{\Sigma}_{k-1}, \boldsymbol{S}_k, \boldsymbol{K}_k, \Delta t_k; \boldsymbol{Q}_k\right) \qquad \forall k = 1, \ldots, N \tag{5.63}$$

Here, $\mathcal{F}(\cdot)$ represents any uncertainty-propagation scheme capable of delivering the control covariance, as well as the propagated mean and covariance of the state, based on the control parameters, time interval, and perturbation parameters. Moreover, $\boldsymbol{Q}_k$ represents the cumulative effect of the stochastic perturbation term $\boldsymbol{g}(\boldsymbol{x}, t)\, d\boldsymbol{w}$ over a finite-time arc, as explained in detail in Section 5.5.2.

To render the problem numerically tractable, it is essential to reformulate the cost function of Equation 5.58 and the chance constraint on the control magnitude of Equation 5.57 into a deterministic form.

Assuming the state $\boldsymbol{x}'_{k-1}$ follows a multivariate Gaussian distribution, the thrust vector $\boldsymbol{T}_k$, governed by the feedback control law, can, in turn, be considered a random Gaussian vector, i.e., $\boldsymbol{T}_k \sim \mathcal{N}\left(\boldsymbol{S}_k, \boldsymbol{\Sigma}_k^T\right)$. Consequently, the squared norm of the control, $\|\boldsymbol{T}_k\|^2$, follows a generalised noncentral χ^2 distribution (Mathai and Provost, 1992). This assumption holds true for degrees of position and velocity dispersion that align with the typical navigation requirements of space missions. The justification for this assumption hinges on the efficacy of the proposed method, as the central objective of the designed robust control is to effectively constrain this dispersion below the allowable thresholds. By applying Equation 5.19 with $p = 1 - \beta$, the chance constraint on the control-magnitude bound of Equation 5.57 can be rewritten as

$$\|\boldsymbol{S_k}\| + \Psi_{n_T}^{-1}(\beta)\rho(\boldsymbol{\Sigma}_k^T) \leq T_{\max} \qquad \forall k = 1, \ldots, N \tag{5.64}$$

The term $\|\boldsymbol{S}_k\| = T_{d,k}$ is the norm of the deterministic component of thrust vector control $\boldsymbol{T}_k$ and $\Psi_{n_T}^{-1}(\beta)\rho(\boldsymbol{\Sigma}_k^T) = T_{s,k}$ is a measure (up to a confidence level p) of the control effort due to the CL term.

This allows to express the merit index in Equation 5.59a in a straightforward form:

$$J = \sum_{k=1}^{N} \left(\|\boldsymbol{S}_k\| + \Psi_{n_T}^{-1}(\beta)\, \rho\left(\boldsymbol{\Sigma}_k^T\right)\right) \tag{5.65}$$

where the first term of Equation 5.65 accounts for the cumulative OL cost, while the second term measures the CL control effort, that depends on the dispersion of the control vector through the matrix $\boldsymbol{\Sigma}_k^T$.

It is noteworthy that this formulation allows for the augmentation of the overall OL thrust expenditure, denoted as $T_d = \sum_{k=1}^{N} T_{d,k}$, when such augmentation proves advantageous for the reduction of the CL control term $T_s = \sum_{k=1}^{N} T_{s,k}$. This strategic flexibility aims to diminish the overall thrust magnitude, expressed as $T_{\text{tot}} = T_d + T_s = J$. This distinctive feature stands as the primary advantage of the proposed approach over traditional methods that involve deriving the nominal trajectory first and subsequently searching for a stabilising control.

The low-thrust ROCP is thus formulated as

$$\min_{\boldsymbol{\Theta}} \quad (5.65) \tag{5.66a}$$

$$\text{s.t.:} \quad (5.63), (5.55), (5.56), (5.64) \tag{5.66b}$$

where $\boldsymbol{\Theta}$ is the vector of problem unknowns. This vector is composed of $N\left(n_x + (n_x - 1)/2 + (n_x - 1)^2/2\right)$ optimisation variables, which encompass the n_x elements of $\boldsymbol{\mu}_k$, the $(n_x - 1)/2$ components of the OL control $\boldsymbol{S}_k$, and the $(n_x - 1)^2/2$ elements of the gain matrix $\boldsymbol{K}_k$. The vector is thus written as

$$\boldsymbol{\Theta} = \{\boldsymbol{\mu}_k, \boldsymbol{S}_k, \boldsymbol{K}_k,\ \forall k = 1, \ldots, N\} \tag{5.67}$$

having that $\boldsymbol{\mu}_k$ for $k = N$ is fixed to the assigned final mean state as in Equation 5.55. The problem of Equation 5.66 is solved by a dedicated *hybrid multiple-shooting* scheme. The mean trajectory is optimised

according to a multiple-shooting approach; that is, both the control $\boldsymbol{T}_k$ and the values of the states are considered as optimisation variables, enforcing matching conditions. Specifically, as indicated in Equation 5.63, the mean state and the corresponding covariance at the conclusion of each k-th arc are employed as the initial stochastic moments for the subsequent arc. This sequential approach ensures the continuity of the solution across different arcs. A single-shooting scheme is instead adopted for the covariance. This hybrid scheme makes it possible to retain the improved convergence capability (e.g., robustness to the initial guess) for the mean trajectory, while guaranteeing that the propagated covariance matrices are PSD at any time. Indeed, the latter circumstance might not be guaranteed if matching conditions or finite differences were to be performed directly on the elements of $\boldsymbol{\Sigma}$. Moreover, this approach is independent of the method used to propagate the state and the covariance over time.

All numerical computations will be made non-dimensional by scaling lengths, velocities, and times with respect to $r^{\text{conv}} = \|\boldsymbol{r}_{\oplus}\|$, $v^{\text{conv}} = \|\boldsymbol{v}_{\oplus}\|$, and $t^{\text{conv}} = r^{\text{conv}}/v^{\text{conv}}$, where $\boldsymbol{r}_{\oplus}$ and $\boldsymbol{v}_{\oplus}$ are the initial position and velocity vectors of the Earth.

When the initial-state distribution is non-Gaussian, or when it is initially Gaussian but the nonlinearity of the dynamics causes the propagated state distribution to deviate significantly from Gaussianity, the initial-state distribution is represented using a Gaussian Mixture Model (GMM), which will be explained in detail in Section 5.5.3. A reference trajectory and an associated feedback-control policy are then determined for each mixture component. In this manner, the complete stochastic problem is automatically decomposed into several locally weakly nonlinear subproblems, which can be solved in parallel, as previously anticipated.

Once the optimisation process has yielded the set of optimal trajectories, the resulting control policies can be used to steer a spacecraft whose initial condition $\hat{\boldsymbol{x}}_0$ is drawn from the initial-state distribution. To this end, the trajectory associated with the closest Gaussian mixture component, indexed by j, is selected according to the squared Mahalanobis distance:

$$j = \arg\min_i \left(\hat{\boldsymbol{x}}_0 - \bar{\boldsymbol{x}}_{0,i}\right)^{\mathrm{T}} \boldsymbol{\Sigma}_{x,0,i}^{-1} \left(\hat{\boldsymbol{x}}_0 - \bar{\boldsymbol{x}}_{0,i}\right) \tag{5.68}$$

The corresponding feedback-control law is subsequently applied to propagate the sample trajectory:

$$\hat{\boldsymbol{u}}_k = \bar{\boldsymbol{u}}_{k,j} + \boldsymbol{K}_{k,j} \left(\bar{\boldsymbol{x}}_{k,j} - \hat{\boldsymbol{x}}_k\right) \tag{5.69}$$

5.5.1. MULTIPLE-LEG TRAJECTORIES

Extension to the case of a multiple-leg mission is straightforward. A multiple-leg trajectory is defined as a sequence of single legs in which the assigned final state of a leg coincides with the assigned initial state of the next leg. Hence, to define a multiple-leg trajectory, it is now necessary to provide the number L of legs, the number N_i of arcs of each i-th leg, and the sequence of initial and final states of each i-th leg, $\tilde{\boldsymbol{\mu}}_{i,0}$ and $\tilde{\boldsymbol{\mu}}_{i,N_i}$, thus requiring the following linkage conditions to ensure the continuity of the trajectory:

$$\tilde{\boldsymbol{\mu}}_{i,N_i} = \tilde{\boldsymbol{\mu}}_{i+1,0}, \qquad \forall i = 1, \ldots, L-1 \tag{5.70}$$

$$\boldsymbol{\Sigma}_{i,N_i} = \boldsymbol{\Sigma}_{i+1,0}, \qquad \forall i = 1, \ldots, L-1 \tag{5.71}$$

$$\boldsymbol{\Sigma}_{i,N_i}^{T} = \boldsymbol{\Sigma}_{i+1,0}^{T}, \qquad \forall i = 1, \ldots, L-1 \tag{5.72}$$

Adding the notation for the leg number, the equation related to the path constraint on the control magnitude becomes

$$\|\boldsymbol{S}_{i,k}\| + \Psi_{n_T}^{-1}(\beta)\rho(\boldsymbol{\Sigma}_{i,k}^{T}) \leq T_{\max} \qquad \forall k = 0, \ldots, N_i; \qquad \forall i = 1, \ldots, L \tag{5.73}$$

while the terminal constraints are instead written as

$$\boldsymbol{\Sigma}'_{i,N_i} \preceq \tilde{\boldsymbol{\Sigma}}'_{i,N_i}, \qquad \forall i = 1, \ldots, L, \tag{5.74}$$

$$\boldsymbol{\mu}'_{i,N_i} = \tilde{\boldsymbol{\mu}}'_{i,N_i} = \left[\tilde{\boldsymbol{\mu}}^{r}_{i,N_i}, \tilde{\boldsymbol{\mu}}^{v}_{i,N_i}\right], \qquad \forall i = 1, \ldots, L. \tag{5.75}$$

and the cost function reads as

$$J = \sum_{i=1}^{L} \sum_{k=0}^{N_i} \left(\|\boldsymbol{S}_{i,k}\| + \Psi_{n_T}^{-1}(\beta)\, \rho\big(\boldsymbol{\Sigma}_{i,k}^{T}\big)\right) \tag{5.76}$$

5.5.2. UNSCENTED TRANSFORM

The Unscented Transform (UT) approximates the propagation of a probability distribution through nonlinear dynamics by replacing the original distribution with a finite set of deterministically selected sigma points. These points are chosen to reproduce the mean and covariance of the initial state and are then propagated independently through the nonlinear equations of motion. The propagated mean and covariance are subsequently reconstructed from the weighted sigma-point ensemble. In the present formulation, separate sigma-point sets are introduced for the spacecraft state $\mathcal{X}_|$ and the navigation measurement $\mathcal{Y}_|$.

The two sets of $2n+1$ sigma points (SPs) $\mathcal{X}_j$ and $\mathcal{Y}_j$, together with the related weights c_j, is evaluated for $\boldsymbol{x}_{k-1}$ and $\boldsymbol{y}_{k-1}$, with $j = 0, \ldots, 2n$, where the zeroth SP is termed the central, or pivotal, SP:

$$\mathcal{Y}_0 = \mathcal{X}_0 = \boldsymbol{\mu}_{k-1}, \tag{5.77}$$

$$\mathcal{Y}_j = \boldsymbol{\mu}_{k-1} + \left(\sqrt{(n_x + \kappa)\left(\boldsymbol{\Sigma}_{k-1} + \tilde{\boldsymbol{R}}_{\text{aug}}\right)} \right)_j, \qquad \forall j = 1, \ldots, n_x, \tag{5.78}$$

$$\mathcal{Y}_{n_x+j} = \boldsymbol{\mu}_{k-1} - \left(\sqrt{(n_x + \kappa)\left(\boldsymbol{\Sigma}_{k-1} + \tilde{\boldsymbol{R}}_{\text{aug}}\right)} \right)_j, \qquad \forall j = 1, \ldots, n_x, \tag{5.79}$$

$$\mathcal{X}_j = \boldsymbol{\mu}_{k-1} + \left(\sqrt{(n_x + \kappa)\boldsymbol{\Sigma}_{k-1}} \right)_j, \qquad \forall j = 1, \ldots, n_x, \tag{5.80}$$

$$\mathcal{X}_{n_x+j} = \boldsymbol{\mu}_{k-1} - \left(\sqrt{(n_x + \kappa)\boldsymbol{\Sigma}_{k-1}} \right)_j, \qquad \forall j = 1, \ldots, n_x, \tag{5.81}$$

$$c_0 = \frac{\kappa}{n_x + \kappa}, \tag{5.82}$$

$$c_j = \frac{1}{2(n_x + \kappa)}, \qquad \forall j = 1, \ldots, 2n_x. \tag{5.83}$$

Here $\kappa \in [0, \infty) \subset \mathbb{R}$ is a scaling parameter, that in the present work is set as $\kappa = 0$, following the general guidelines in the seminal work on UT (Julier et al., 1995), and the operator $(\cdot)_j$ represents the j-th column vector of the underlying matrix. Where $\tilde{\boldsymbol{R}}_{\text{aug}}$ refers to $\tilde{\boldsymbol{R}}$ augmented with a zero row and a zero column associated with the spacecraft mass:

$$\tilde{\boldsymbol{R}}_{\text{aug}} = \begin{bmatrix} \tilde{\boldsymbol{R}} & \mathbf{0}_{(n_x-1)\times 1} \\ \mathbf{0}_{1\times(n_x-1)} & 0 \end{bmatrix} \in \mathbb{R}^{n_x \times n_x}. \tag{5.84}$$

Following Equation 5.62, and recalling that $\mathbf{H} = \mathbb{I}_{n_x}$, the thrust vector related to each SP is computed as

$$\boldsymbol{T}_0 = \boldsymbol{S}_k, \tag{5.85}$$

$$\boldsymbol{T}_j = \boldsymbol{S}_k + \boldsymbol{K}_k \left(\mathcal{Y}'_j - \boldsymbol{\mu}'_{k-1} \right) \qquad \forall j = 1, \ldots, 2n_x. \tag{5.86}$$

where $\mathcal{Y}'_j \in \mathbb{R}^{n_x - 1}$ corresponds to the SP $\mathcal{Y}_j$ excluding the mass term, mirroring the structure of $\boldsymbol{x}'_{k-1}$ in relation to $\boldsymbol{x}_{k-1}$. Note that $\mathcal{Y}'_{2n_x-1} = \mathcal{Y}'_{2n_x} = \boldsymbol{\mu}'_{k-1}$, implying that $\boldsymbol{T}_{2n_x-1} = \boldsymbol{T}_{2n_x} = \boldsymbol{T}_0$. This equivalence arises because, for these two specific values of j, the corresponding SPs exhibit variations solely in mass. Consequently, their forms without the mass term are precisely identical to the mean position and velocity $\boldsymbol{\mu}'_{k-1}$.

With the thrust vectors computed via Equation 5.86, the covariance $\boldsymbol{\Sigma}_k^T$ of the thrust vector $\boldsymbol{T}_k$ is evaluated as

$$\boldsymbol{\Sigma}_k^T = \sum_{j=1}^{2(n_x-1)} \frac{1}{2(n_x - 1)} \left(\boldsymbol{T}_j - \boldsymbol{S}_k \right) \left(\boldsymbol{T}_j - \boldsymbol{S}_k \right)^T, \tag{5.87}$$

since $\mathbb{E}[\boldsymbol{T}_j] = \boldsymbol{S}_k$. The pivotal thrust vector $\boldsymbol{T}_0$ is intentionally excluded from the computation of $\boldsymbol{\Sigma}_k^T$ to align with the choice of $\kappa = 0$ in the state UT propagation. The thrust vectors $\boldsymbol{T}_{2n_x-1}$ and $\boldsymbol{T}_{2n_x}$ are likewise omitted from the computation, given that both are equal to $\boldsymbol{T}_0$.

To obtain the state covariance matrix instead, the SPs $\mathcal{X}_j$ are propagated to obtain the new set of samples $\mathcal{Z}_j$, with $j = 0, \ldots, 2n_x$. The propagation is performed using the deterministic component of the equations of motion in Equation 3.58.

The propagated SPs are thus

$$\mathcal{Z}_j = \boldsymbol{f}(\mathcal{X}_j) \qquad \forall j = 0, \ldots, 2n. \tag{5.88}$$

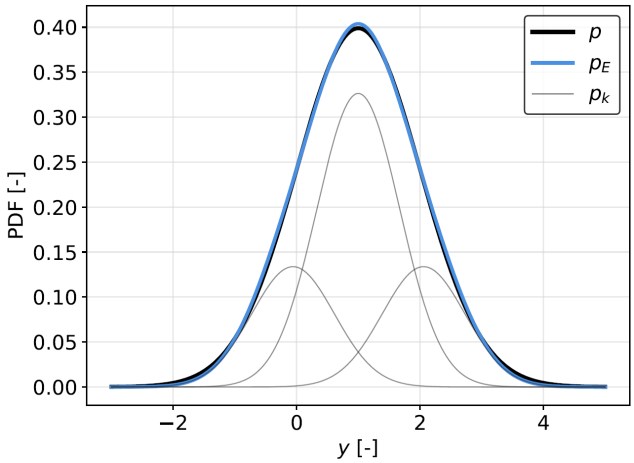


Figure 5.2: GMM decomposition of a univariate standard normal distribution using three mixands (DeMars et al., 2013; Caleb, 2025).

The final mean state $\boldsymbol{\mu}_k$ and its covariance matrix $\boldsymbol{\Sigma}_k$ are thus computed as

$$\boldsymbol{\mu}_k = \sum_{j=0}^{2n_x} c_j \mathcal{Z}_j, \tag{5.89}$$

$$\boldsymbol{\Sigma}_k = \sum_{j=0}^{2n_x} c_j \left(\mathcal{Z}_j - \boldsymbol{\mu}_k\right)\left(\mathcal{Z}_j - \boldsymbol{\mu}_k\right)^T. \tag{5.90}$$

Since the time-continuous stochastic perturbation, represented by the term $\boldsymbol{g}(\boldsymbol{x}, t)\, d\boldsymbol{w}$, is not accounted for while propagating the set of sampled states with the equations of motion, a mechanism is required to account for this random disturbance. This stochastic contribution is modelled as an instantaneous, additive, Gaussian perturbation with zero mean and covariance $\boldsymbol{Q}_k$, applied at the end of each arc. It signifies the cumulative effects of stochastic perturbations over a finite-time arc. The matrix $\boldsymbol{Q}_k$ is estimated for each k-th arc along a reference trajectory obtained from the optimal non-robust solution through the Trajectory Bundle Method (Tracy et al., 2025), following the methodology outlined in Appendix A2.

It is also noted that the value of $\boldsymbol{Q}_k$ is held constant throughout the first iterations of the solution process, i.e., it is not updated at each NLP iteration. This choice improves convergence stability and is justified by the fact that the reference trajectory obtained through the Trajectory Bundle Method and used as the reference for computing $\boldsymbol{Q}_k$ remains sufficiently close to the robust solution in the scenarios considered. In the cases where larger discrepancies arise, more frequent updates of $\boldsymbol{Q}_k$ based on the converged robust trajectory may be performed.

5.5.3. GAUSSIAN MIXTURE MODEL

A GMM approximates the PDF p describing a multivariate random variable $\boldsymbol{y}$ using a weighted sum of multivariate Gaussian distributions p_G:

$$p(\boldsymbol{y}) \approx p_E(\boldsymbol{y}; \theta) = \sum_{i=1}^{M} \alpha_i p_G\left(\boldsymbol{y}; \tilde{\boldsymbol{\mu}}_i, \tilde{\boldsymbol{\Sigma}}_i\right), \tag{5.91}$$

where

$$\theta = \left(\alpha_1, \alpha_2, \ldots, \alpha_M, \tilde{\boldsymbol{\mu}}_1, \tilde{\boldsymbol{\mu}}_2, \ldots, \tilde{\boldsymbol{\mu}}_M, \tilde{\boldsymbol{\Sigma}}_1, \tilde{\boldsymbol{\Sigma}}_2, \ldots, \tilde{\boldsymbol{\Sigma}}_M\right).$$

To provide more insight into the formulation, an illustrative example of a GMM approximation of a univariate standard normal distribution obtained by Caleb (2025) following the methodology of DeMars et al. (2013) and Fossà (2024) is provided in Figure 5.2.

The generated GMM can be validated against an MC, whose samples are drawn from the original distribution of PDF p. To do so, the likelihood agreement measure (LAM), used by DeMars et al. (2013), is adopted:

$$\mathcal{L}(p_1, p_2) = \int_{\mathbb{R}^{n_y}} p_1(\boldsymbol{y}) p_2(\boldsymbol{y}) \mathrm{d}\boldsymbol{y}. \tag{5.92}$$

The value of $\mathcal{L}$ measures the similarity between two distributions in terms of their PDFs. If $p_1 = p_E$ and a sample of size N, $\boldsymbol{Y} = (\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_N)$ is drawn from the original distribution of PDF p, then p can be approximated by a sum of equally weighted Dirac functions: $p \approx p_{MC}(\boldsymbol{y}) = \frac{1}{N} \sum_{k=1}^{N} \delta_{y_k}(\boldsymbol{y})$.

The raw LAM defined in Equation (5.92) is an overlap measure between two probability density functions. However, its numerical value depends on the self-overlap of the reference distribution and therefore is difficult to interpret directly. To obtain a dimensionless metric, a normalised LAM can be introduced as

$$\mathcal{L}_{\text{norm}}(p_E, p) = \frac{\mathcal{L}(p_E, p)}{\mathcal{L}(p, p)}, \tag{5.93}$$

where

$$\mathcal{L}(p, p) = \int_{\mathbb{R}^{n_y}} p(\boldsymbol{y})^2 \, \mathrm{d}\boldsymbol{y} \tag{5.94}$$

is the self-overlap of the reference distribution. By construction,

$$\mathcal{L}_{\text{norm}}(p, p) = 1.$$

When the reference PDF is represented by a Monte Carlo sample $\boldsymbol{Y} = (\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_N)$, Equation (5.93) can be estimated as

$$\mathcal{L}_{\text{norm}} \approx \frac{\sum_{k=1}^{N} p_E(\boldsymbol{y}_k)}{\sum_{k=1}^{N} p(\boldsymbol{y}_k)}. \tag{5.95}$$

Each Gaussian composing the GMM can be propagated independently using the method defined by Equation 5.63. Indeed, the norm of $\tilde{\Sigma}$ is smaller than the norm of Σ; it is therefore expected that the mixands will be more finely controlled when a dedicated control policy is designed for each of them, reducing the spread of the individual dispersions through the nonlinear dynamics and thus keeping the individual mixands closer to Gaussianity.

REFERENCES

Acciarini, G., Greco, C., and Vasile, M. Uncertainty propagation in orbital dynamics via galerkin projection of the fokker–planck equation. *Advances in Space Research*, 73:53–63, 2024. doi: 10.1016/j.asr.2023.11.042.

Acton, C. H. Ancillary data services of NASA's navigation and ancillary information facility. *Planetary and Space Science*, 44(1):65–70, 1996. doi: 10.1016/0032-0633(95)00107-7.

Akman, T., Ben-Asher, J. Z., and Holzapfel, F. Approximation of closed-loop sensitivities in robust trajectory optimization under parametric uncertainty. *Aerospace*, 11(8):640, 2024. doi: 10.3390/aerospace11080640.

Alfriend, K. T. and Park, I. When does the uncertainty become non-gaussian. In *Proceedings of the 2016 Advanced Maui Optical and Space Surveillance Technologies Conference*, pages 5005–514, 2016.

Andersen, E. D. On primal and dual infeasibility certificates in a homogeneous model for convex optimization. *SIAM Journal on Optimization*, 11(2):380–388, January 2000. ISSN 1095-7189. doi: 10.1137/S1052623499353145.

Archinal, B. A. et al. Report of the IAU working group on cartographic coordinates and rotational elements: 2009. *Celestial Mechanics and Dynamical Astronomy*, 109(2):101–135, 2011. doi: 10.1007/s10569-010-9320-4.

Archinal, B. A. et al. Report of the IAU working group on cartographic coordinates and rotational elements: 2015. *Celestial Mechanics and Dynamical Astronomy*, 130(3):22, 2018. doi: 10.1007/s10569-017-9805-5.

Arora, N., Vittaldev, V., and Russell, R. P. Parallel computation of trajectories using graphics processing units and interpolated gravity models. *Journal of Guidance, Control, and Dynamics*, 38(8):1345–1355, 2015.

Ayala, A. and Rigger, R. SMART-1 moon impact operations. In *Proceedings of the 20th International Symposium on Space Flight Dynamics*, September 2007. URL `https://ntrs.nasa.gov/citations/20080012693`.

Badolati, E. Gasparis, annibale de. In Hockey, T. et al., editors, *The Biographical Encyclopedia of Astronomers*, page 408. Springer, New York, 2007. doi: 10.1007/978-0-387-30400-7_502.

Battin, R. H. *An introduction to the mathematics and methods of astrodynamics*. AIAA, 1999.

Bellman, R. and Kalaba, R. E. *Dynamic Programming and Modern Control Theory*, volume 81. Academic Press, Cambridge, MA, USA, 1965.

Benedikter, B., Zavoli, A., Colasurdo, G., Pizzurro, S., and Cavallini, E. Autonomous upper stage guidance with robust splash-down constraint. In *72nd International Astronautical Congress*, IAC-21,D2,3,8,x66016, Dubai, UAE, 2021. International Astronautical Federation.

Benedikter, B., Zavoli, A., Wang, Z., Pizzurro, S., and Cavallini, E. Convex approach to covariance control with application to stochastic low-thrust trajectory optimization. *Journal of Guidance, Control, and Dynamics*, 45(11):2061–2074, 2022. doi: 10.2514/1.G006806.

Benkhoff, J. et al. BepiColombo—mission overview and science goals. *Space Science Reviews*, 217:90, 2021. doi: 10.1007/s11214-021-00861-4.

Bergsma, M. C. and Mooij, E. Application of taylor-series integration to reentry problems with wind. *Journal of Guidance, Control, and Dynamics*, 39(10):2324–2335, 2016.

Berz, M. Differential algebraic description of beam dynamics to very high orders. Technical Report SSC-152, SSC Central Design Group, 1988.

Berz, M. Chapter 2 - differential algebraic techniques. In *Modern map methods in particle beam physics*, volume 108, pages 81–117. Elsevier, 1999.

Blackmore, L., Ono, M., Bektassov, A., and Williams, B. C. A probabilistic particle-control approximation of chance-constrained stochastic predictive control. *IEEE Transactions on Robotics*, 26(3):502–517, June 2010. ISSN 1941-0468. doi: 10.1109/TRO.2010.2044948.

Blackmore, L., Ono, M., and Williams, B. C. Chance-constrained optimal path planning with obstacles. *IEEE Transactions on Robotics*, 27(6):1080–1094, December 2011. ISSN 1941-0468. doi: 10.1109/TRO.2011.2161160.

Boone, S. and McMahon, J. Non-gaussian chance-constrained trajectory control using gaussian mixtures and risk allocation. In *IEEE 61st Conference on Decision and Control (CDC)*, pages 3592–3597, 2022.

Bradley, N., Snyder, J. S., Lubey, D., Manka, A., Jones, D. R., Velez, D., and Koh, D. Pre-launch electric propulsion error models for psyche navigation. In *AAS/AIAA Astrodynamics Specialist Conference*, number AAS 23-103, 2023.

Bradley, N., Greaves, J., Han, D., Hollenberg, C., Jones, D., Koh, D., Lubey, D., McElrath, T., Nicholas, A., and Snyder, J. S. Navigating psyche: Launch, initial checkout, and start of cruise. *The Journal of the Astronautical Sciences*, 73(4), 2026. doi: 10.1007/s40295-025-00548-w. URL `https://doi.org/10.1007/s40295-025-00548-w`.

Brisebarre, N. and Joldecs, M. Chebyshev interpolation polynomial-based tools for rigorous computing. In *Proceedings of the 2010 International Symposium on Symbolic and Algebraic Computation*, pages 147–154, 2010.

Broughton, P. The first predicted return of comet halley. *Journal for the History of Astronomy*, 16(2):123–133, 1985. doi: 10.1177/002182868501600203.

Bryson, A. E., Ho, Y.-C., and Siouris, G. M. Applied optimal control: Optimization, estimation, and control. *IEEE Transactions on Systems, Man, and Cybernetics*, 9:366–367, 1979.

Büskens, C. and Wassel, D. The esa nlp solver worhp. pages 85–110. Springer New York, 2012. ISBN 9781461444695. doi: 10.1007/978-1-4614-4469-5_4.

Caleb, T. *Optimisation stochastique pour l'analyse mission*. Doctoral thesis, ISAE-SUPAERO, Université de Toulouse, Toulouse, France, December 2025.

Caputo, M. R. Dynamic programming and the hamilton–jacobi–bellman equation. In *Foundations of Dynamic Economic Analysis: Optimal Control Theory and Applications*, pages 511–536. Cambridge University Press, Cambridge, UK, 2005.

Chai, R., Savvaris, A., Tsourdos, A., Chai, S., and Xia, Y. A review of optimization techniques in spacecraft flight trajectory design. *Progress in Aerospace Sciences*, 109:100543, August 2019. doi: 10.1016/j.paerosci.2019.05.003.

Cheng, M., Ries, J. C., and Tapley, B. Assessment of the solar radiation model for GRACE orbit determination. *Advances in the Astronautical Sciences*, 129:501–510, March 2008.

Cheng, J., Grossman, M., and McKercher, T. *Professional CUDA C Programming*. Wrox, 1st edition, 2014.

Cheng, L., Wang, Z., Song, Y., and Jiang, F. Real-time optimal control for irregular asteroid landings using deep neural networks. *Acta Astronautica*, 170:66–79, 2020. ISSN 0094-5765. doi: 10.1016/j.actaastro.2019.11.039.

Conway, B., Prussing, J. J., and Mondelo, J. M. Primer vector theory and applications. chapter 2, pages 16–36. Cambridge University Press, 2010. doi: 10.1017/CBO9780511778025.

Cui, P. and Qiao, D. The present status and prospects in the research of orbital dynamics and control near small celestial bodies. *Theoretical and Applied Mechanics Letters*, 4(1):013013, 2014. doi: 10.1063/2.1401313.

Curtis, H. D. *Orbital Mechanics for Engineering Students*. Elsevier Butterworth-Heinemann, Oxford, 2005. ISBN 978-0-7506-6169-0.

da Graça Marto, S. and Vasile, M. Multi-objective robust trajectory optimization of multi asteroid fly-by under epistemic uncertainty. In Vasile, M. and Quagliarella, D., editors, *Advances in Uncertainty Quantification and Optimization Under Uncertainty with Aerospace Applications*, pages 209–230. Springer International Publishing, 2021. doi: 10.1007/978-3-030-80542-5_13.

da Graça Marto, S. and Vasile, M. Many-objective robust trajectory optimisation under epistemic uncertainty and imprecision. *Acta Astronautica*, 191:99–124, 2022. doi: 10.1016/j.actaastro.2021.10.022.

Darby, C. L., Hager, W. W., and Rao, A. V. An hp-adaptive pseudospectral method for solving optimal control problems. *Optimal Control Applications and Methods*, 32(4):476–502, 2011.

de Gasparis, A. Discovery of a new planet. *Monthly Notices of the Royal Astronomical Society*, 12:125, 1852.

DeMars, K. J., Bishop, R. H., and Jah, M. K. Entropy-based approach for uncertainty propagation of nonlinear dynamical systems. *Journal of Guidance, Control, and Dynamics*, 36(4):1047–1057, July 2013. ISSN 1533-3884. doi: 10.2514/1.58987.

Di Lizia, P., Armellin, R., Ercoli-Finzi, A., and Berz, M. High-order robust guidance of interplanetary trajectories based on differential algebra. *Journal of Aerospace Engineering, Sciences and Applications*, 1(1): 43–57, 2008.

Dibb, S. D. et al. A post-launch summary of the science of NASA's psyche mission. *AGU Advances*, 5(2): e2023AV001077, 2024. doi: 10.1029/2023AV001077.

Elkins-Tanton, L. T. et al. Observations, meteorites, and models: A preflight assessment of the composition and formation of (16) Psyche. *Journal of Geophysical Research: Planets*, 125(3):e2019JE006296, 2020. doi: 10.1029/2019JE006296.

Epstein, C., Miranker, W., and Rivlin, T. Ultra-arithmetic i: function data types. *Mathematics and Computers in Simulation*, 24(1):1–18, 1982.

Farina, M., Giulioni, L., and Scattolini, R. Stochastic linear model predictive control with chance constraints – a review. *Journal of Process Control*, 44:53–67, 2016. doi: 10.1016/j.jprocont.2016.03.005.

Farnocchia, D., Fuentes-Muñoz, O., Park, R. S., Baer, J., and Chesley, S. R. Mass, density, and radius of asteroid (16) Psyche from high-precision astrometry. *The Astronomical Journal*, 168(1):21, 2024. doi: 10.3847/1538-3881/ad50ca.

Federici, L., Benedikter, B., and Zavoli, A. Deep learning techniques for autonomous spacecraft guidance during proximity operations. *Journal of Spacecraft and Rockets*, 58(6):1774–1785, 2021. doi: 10.2514/1.A35076.

Ferrari, F., Lavagna, M., and Blazquez, E. A parallel-gpu code for asteroid aggregation problems with angular particles. *Monthly Notices of the Royal Astronomical Society*, 492(1):749–761, 2020.

Fisher, J. and Bhattacharya, R. Optimal trajectory generation with probabilistic system uncertainty using polynomial chaos. *Journal of Dynamic Systems, Measurement and Control*, 133(1), 2011.

Fossà, A. Multifidelity orbit uncertainty propagation in the presence of stochastic accelerations, 2024.

Fujimoto, K. and Scheeres, D. Tractable expressions for nonlinearly propagated uncertainties. *Journal of Guidance, Control, and Dynamics*, 38(6):1146–1151, 2015.

Fuller, A. T. Analysis of nonlinear stochastic systems by means of the fokker–planck equation. *International Journal of Control*, 9(6):603–655, 1969.

Gambier, A. and Badreddin, E. Multi-objective optimal control: An overview. In *2007 IEEE International Conference on Control Applications*, pages 170–175. IEEE, October 2007.

Geda, M. Massive parallelization of trajectory propagations using GPUs. MSc thesis, Delft University of Technology, Delft, The Netherlands, January 2019. Available at the TU Delft Repository.

Gelb, A. and Warren, R. Direct statistical analysis of nonlinear systems: Cadet. *AIAA Journal*, 11(5):689–694, 1973.

Geletu, A., Klöppel, M., Zhang, H., and Li, P. Advances and applications of chance-constrained approaches to systems optimisation under uncertainty. *International Journal of Systems Science*, 44(7):1209–1232, 2013. doi: 10.1080/00207721.2012.670310.

Geller, D. K. and Christensen, D. P. Linear covariance analysis for powered lunar descent and landing. *Journal of Spacecraft and Rockets*, 46(6):1231–1248, 2009.

Geller, D. K., Rose, M. B., and Woffinden, D. C. Event triggers in linear covariance analysis with applications to orbital rendezvous. *Journal of guidance, control, and dynamics*, 32(1):102–111, 2009.

Gill, P. R. User's guide for snopt version 7: Software for large-scale nonlinear programming. Technical report, Department of Mathematics, University of California, San Diego, CA, USA, 2008.

Giordano, C. and Topputo, F. Analysis, design, and optimization of robust trajectories in cislunar environment for limited-capability spacecraft. *The Journal of the Astronautical Sciences*, 70:53, 2023. doi: 10.1007/s40295-023-00413-8.

Gottlieb, D. and Shu, C.-W. On the gibbs phenomenon and its resolution. *SIAM Review*, 39(4):644–668, 1997.

Gottlieb, R. G. Rapid convergence to optimum solutions using a min-h strategy. *AIAA Journal*, 5:322–329, 1967.

Greco, C. and Vasile, M. Closing the loop between mission design and navigation analysis. In *Proceedings of the 71st International Astronautical Congress (IAC), The CyberSpace Edition*, Virtual Conference, October 2020.

Greco, C., Di Carlo, M., Vasile, M., and Epenoy, R. Direct multiple shooting transcription with polynomial algebra for optimal control problems under uncertainty. *Acta Astronautica*, 170:224–234, 2020. doi: 10.1016/j.actaastro.2019.12.010.

Greco, C., Campagnola, S., and Vasile, M. Robust space trajectory design using belief optimal control. *Journal of Guidance, Control, and Dynamics*, 45(6):1060–1077, 2022. doi: 10.2514/1.G005704.

Gunantara, N. A review of multi-objective optimization: Methods and its applications. *Cogent Engineering*, 5(1):1502242, January 2018.

Gurfil, P. Robust zero miss distance guidance for missiles with parametric uncertainties. In *Proceedings of the 2001 American Control Conference*, volume 5, pages 3364–3369, 2001.

Gustafson, E. D. *Stochastic Optimal Control of Spacecraft*. Phd thesis, The University of Michigan, 2010. URL `https://hdl.handle.net/2027.42/77707`.

Hirabayashi, M. et al. Hayabusa2 extended mission: New voyage to rendezvous with a small asteroid rotating with a short period. *Advances in Space Research*, 68(3):1533–1555, 2021. doi: 10.1016/j.asr.2021.03.030.

Holt, H., Armellin, R., Baresi, N., Hashida, Y., Turconi, A., Scorsoglio, A., and Furfaro, R. Optimal q-laws via reinforcement learning with guaranteed stability. *Acta Astronautica*, 187:511–528, 2021. doi: 10.1016/j.actaastro.2021.07.010.

Huang, Y., Li, H., Du, X., and He, X. Mars entry trajectory robust optimization based on evidence under epistemic uncertainty. *Acta Astronautica*, 163:225–237, 2019.

Izzo, D., Öztürk, E., and Märtens, M. Interplanetary transfers via deep representations of the optimal policy and/or of the value function. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '19 Companion)*, pages 1971–1979, Prague, Czech Republic, 2019. Association for Computing Machinery. ISBN 978-1-4503-6748-6. doi: 10.1145/3319619.3326834.

Izzo, D. PyGMO and PyKEP: Open source tools for massively parallel optimization in astrodynamics - the case of interplanetary trajectory optimization. In *5th International Conference on Astrodynamics Tools and Techniques (ICATT)*, Noordwijk, the Netherlands, 2012.

Jacobi, C. G. J. Über ein leichtes verfahren die in der theorie der säcularstörungen vorkommenden gleichungen numerisch aufzulösen. *Journal für die reine und angewandte Mathematik (Crelles Journal)*, 1846(30):51–94, 1846. doi: 10.1515/crll.1846.30.51.

Jiang, X. and Li, S. Mars entry trajectory planning using robust optimization and uncertainty quantification. *Acta Astronautica*, 161:249–261, 2019.

Julier, S., Uhlmann, J., and Durrant-Whyte, H. F. A new approach for filtering nonlinear systems. In *Proceedings of 1995 American Control Conference – ACC'95*, volume 3, pages 1628–1632, 1995. doi: 10.1109/ACC.1995.529783.

Kaleem, M. K. and Leitner, J. Cuda massively parallel trajectory evolution. In *GPUs for Genetic and Evolutionary Computation at the Genetic and Evolutionary Computation Conference (GECCO)*, Dublin, Ireland, 2011.

Karush, W. Minima of functions of several variables with inequalities as side conditions. Master's thesis, Department of Mathematics, University of Chicago, Chicago, IL, USA, 1939.

Kaula, W. M. *Theory of Satellite Geodesy*. Blaisdell Publishing Company, Waltham, 1966.

Kuhn, H. W. and Tucker, A. W. Nonlinear programming. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 481–492, Berkeley, CA, USA, 1951. University of California Press.

Kuninaka, H. and Kawaguchi, J. Lessons learned from round trip of Hayabusa asteroid explorer in deep space. In *2011 IEEE Aerospace Conference*, IEEE Aerospace Conference Proceedings, pages 1–8, 2011. ISBN 9781424473502. doi: 10.1109/AERO.2011.5747599.

LaFarge, N. B., Miller, D., Howell, K. C., and Linares, R. Autonomous closed-loop guidance using reinforcement learning in a low-thrust, multi-body dynamical environment. *Acta Astronautica*, 186:1–23, 2021. doi: 10.1016/j.actaastro.2021.05.014.

Laipert, F. E. and Longuski, J. M. Automated missed-thrust propellant margin analysis for low-thrust trajectories. *Journal of Spacecraft and Rockets*, 52(4):1135–1143, 2015.

Lalande, J.-J. L. d. *Tables astronomiques de M. Halley, pour les planètes et les comètes, réduites au nouveau stile et méridien de Paris, augmentées de plusieurs tables nouvelles de différens auteurs, pour les satellites de Jupiter et les étoiles fixes, avec des explications détaillées. Et l'histoire de la comète de 1759*. Durand, Paris, 1759.

Leon Dasi, M. Collision probability through orbital uncertainty propagation: the hybrid differential algebra and gaussian mixture model approach. Master's thesis, TU Delft, 2021.

Lew, T., Bonalli, R., and Pavone, M. Chance-constrained sequential convex programming for robust trajectory optimization. In *2020 European Control Conference (ECC)*, pages 1871–1878, 2020. doi: 10.23919/ECC51009.2020.9143595.

Lin, M., Xu, M., and Fu, X. A parallel algorithm for the initial screening of space debris collisions prediction using the sgp4/sdp4 models and gpu acceleration. *Advances in Space Research*, 59(9):2398–2406, 2017.

Liu, P., Yuan, T., Ma, Y., Wang, L., Liu, D., Yue, S., and Kołodziej, J. Parallel processing of massive remote sensing images in a gpu architecture. *Computing and Informatics*, 33(1):197–217, 2014.

Liu, K., Jia, B., Chen, G., Pham, K., and Blasch, E. A real-time orbit satellites uncertainty propagation and visualization system using graphics computing unit and multi-threading processing. In *Proceedings of the 2015 IEEE/AIAA 34th Digital Avionics Systems Conference (DASC)*, pages 8A2–1–8A2–10, 2015.

Longmuir, A. G. and Bohn, E. V. Second-variation methods in dynamic optimization. *Journal of Optimization Theory and Applications*, 3:164–173, 1969.

Longuski, J. M., Guzman, J. J., and Prussing, J. E. *Optimal Control with Aerospace Applications*. Springer New York, 2014.

Losacco, M., Fossà, A., and Armellin, R. Low-order automatic domain splitting approach for nonlinear uncertainty mapping. *Journal of Guidance, Control, and Dynamics*, 47(2):291–310, February 2024. ISSN 1533-3884. doi: 10.2514/1.G007271.

Lunghi, P., Armellin, R., Di Lizia, P., and Lavagna, M. Semi-analytical adaptive guidance computation based on differential algebra for autonomous planetary landing. In *Advances in the Astronautical Sciences*, volume 158, pages 2003–2022, 2016.

Lunghi, P., Lavagna, M., Di Lizia, P., Armellin, R., and Mease, K. D. Atmospheric entry guidance based on differential algebra for high elevation mars landing. In *Proceedings of the 2018 Space Flight Mechanics Meeting*, number AIAA-18-1458, 2018.

Luo, Y.-z. and Yang, Z. A review of uncertainty propagation in orbital mechanics. *Progress in Aerospace Sciences*, 89:23–39, 2017. doi: 10.1016/j.paerosci.2016.12.002.

Macklin, M. Warp: Differentiable spatial computing for python. In *SIGGRAPH Courses '24*, 2024. doi: 10.1145/3664475.3664543. URL `https://doi.org/10.1145/3664475.3664543`.

Mahalanobis, P. C. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

Marchisio, D. L. and Fox, R. O. Solution of population balance equations using the direct quadrature method of moments. *Journal of Aerosol Science*, 36(1):43–73, 2005.

Marmo, N. and Zavoli, A. Chance-constraint optimization of interplanetary trajectories with a hybrid multiple-shooting approach. In *AAS/AIAA Astrodynamics Specialist Conference*, 2022.

Marmo, N. and Zavoli, A. Chance-constraint method for covariance control of low-thrust interplanetary missions. In *AIAA SCITECH 2024 Forum*, pages 1–18. American Institute of Aeronautics and Astronautics, 2024. doi: 10.2514/6.2024-0630.

Marmo, N., Zavoli, A., Ozaki, N., and Kawakatsu, Y. A hybrid multiple-shooting approach for covariance control of interplanetary missions with navigation errors. In *AAS/AIAA Astrodynamics Specialist Conference*, 2023.

Marmo, N., Zavoli, A., and Ozaki, N. Chance-constraint robust trajectory optimization with a hybrid multiple-shooting approach. *Journal of Guidance, Control, and Dynamics*, 48(11):2495–2511, November 2025. doi: 10.2514/1.G008275.

Massari, M., Di Lizia, P., and Rasotto, M. Nonlinear uncertainty propagation in astrodynamics using differential algebra and graphics processing units. *Journal of Aerospace Information Systems*, 14(9):493–503, 2017.

Mathai, A. M. and Provost, S. B. *Quadratic Forms in Random Variables: Theory and Applications*, volume 87. Marcel Dekker, 1992.

Mayne, D. Q. Differential dynamic programming—a unified approach to the optimization of dynamic systems. In *Control and Dynamic Systems*, volume 10, pages 179–254. Academic Press, Cambridge, MA, USA, 1973.

Melman, J., Mooij, E., and Noomen, R. Autonomous navigation at asteroids. In *Proceedings of the 60th International Astronautical Congress*, volume 2, pages 1519–1530, 2009.

Montenbruck, O. and Gill, E. *Satellite Orbits: Models, Methods and Applications*. Springer, Berlin, Germany, 2001. doi: 10.1007/978-3-642-58351-3.

Mooij, E. *Re-entry Systems*. Springer Aerospace Technology. Springer, Cham, 1 edition, 2024. ISBN 978-3-031-62174-1. doi: 10.1007/978-3-031-62174-1.

Moore, R. E. *Interval analysis*. Prentice-Hall, New Jersey, 1966.

Morante, D., Sanjurjo Rivo, M., and Soler, M. A survey on low-thrust trajectory optimization approaches. *Aerospace*, 8(3):88, 2021. doi: 10.3390/aerospace8030088. URL `https://doi.org/10.3390/aerospace8030088`.

Nakka, Y. K. and Chung, S.-J. Trajectory optimization of chance-constrained nonlinear stochastic systems for motion planning under uncertainty. *IEEE Transactions on Robotics*, 39(1):203–222, 2023. doi: 10.1109/tro.2022.3197072.

Nascimento, N. Gpu-accelerated 3d-aware model predictive path integral control for high-speed autonomous lunar navigation. Master's thesis, Delft University of Technology, Delft, The Netherlands, 2025. To be defended publicly on July 2, 2025.

NVIDIA Corporation. *CUDA Programming Guide*, May 2026. URL `https://docs.nvidia.com/cuda/cuda-programming-guide/pdf/cuda-programming-guide.pdf`. Release 13.3, May 27, 2026. Accessed June 20, 2026.

Oguri, K. and Lantoine, G. Stochastic sequential convex programming for robust low-thrust trajectory design under uncertainty. In *2022 AAS/AIAA Astrodynamics Specialist Conference*, AAS Paper 22-708, Springfield, VA, 2022.

Oguri, K. and McMahon, J. W. Robust spacecraft guidance around small bodies under uncertainty: Stochastic optimal control approach. *Journal of Guidance, Control, and Dynamics*, 44(7):1295–1313, 2021. doi: 10.2514/1.G005426. URL `https://doi.org/10.2514/1.G005426`.

Oguri, K. and McMahon, J. W. Stochastic primer vector for robust low-thrust trajectory design under uncertainty. *Journal of Guidance, Control, and Dynamics*, 45(1):84–102, 2022. doi: 10.2514/1.G005970.

Oguri, K., Lantoine, G., and Sweetser, T. H. Robust solar sail trajectory design under uncertainty with application to nea scout mission, 2022. Government sponsorship acknowledged.

Oh, D., Landau, D., Randolph, T., Timmerman, P., Chase, J., Sims, J., and Kowalkowski, T. Analysis of system margins on deep space missions using solar electric propulsion. In *44th AIAA/ASME/SAE/ASEE Joint Propulsion Conference and Exhibit*, pages 1–30, Reston, Virginia, July 2008. American Institute of Aeronautics and Astronautics. doi: 10.2514/6.2008-5286.

Oh, D. Y. et al. Design and development of the psyche spacecraft for NASA's discovery program. *Space Science Reviews*, 221:96, 2025. doi: 10.1007/s11214-025-01222-1.

Okamoto, K., Goldshtein, M., and Tsiotras, P. Optimal covariance control for stochastic systems under chance constraints. *IEEE Control Systems Letters*, 2(2):266–271, 2018. doi: 10.1109/lcsys.2018.2826038.

Øksendal, B. *Stochastic Differential Equations: An Introduction with Applications*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2003. doi: 10.1007/978-3-642-14394-6.

Omar, K. H. Simultaneous orbital and attitude propagation of satellites in low-earth orbit using cuda for aerodynamics simulation. In *55th AIAA Aerospace Sciences Meeting*, page 2005, 2017.

Ortega, C., Riccardi, A., Vasile, M., and Tardioli, C. Smart-uq: Uncertainty quantification toolbox for generalised intrusive and non intrusive polynomial algebra. In *6th International Conference on Astrodynamics Tools and Techniques*, 2016.

Ostro, S. J., Campbell, D. B., and Shapiro, I. I. Mainbelt asteroids: Dual-polarization radar observations. *Science*, 229(4712):442–446, 1985. doi: 10.1126/science.229.4712.442.

Ozaki, N., Campagnola, S., Funase, R., and Yam, C. H. Stochastic differential dynamic programming with unscented transform for low-thrust trajectory design. *Journal of Guidance, Control, and Dynamics*, 41(2):377–387, 2018. doi: 10.2514/1.G002367.

Ozaki, N., Campagnola, S., and Funase, R. Tube stochastic optimal control for nonlinear constrained trajectory optimization problems. *Journal of Guidance, Control, and Dynamics*, 43(4):645–655, 2020. doi: 10.2514/1.G004363.

Park, R. S. and Scheeres, D. J. Nonlinear mapping of gaussian statistics: theory and applications to spacecraft trajectory design. *Journal of Guidance, Control, and Dynamics*, 29(6):1367–1375, 2006.

Park, R. S. and Scheeres, D. J. Nonlinear semi-analytic methods for trajectory estimation. *Journal of Guidance, Control, and Dynamics*, 30(6):1668–1676, 2007.

Park, S. H. *Nonlinear trajectory navigation*. PhD thesis, The University of Michigan, 2007.

Petit, G. and Luzum, B. IERS conventions (2010). IERS Technical Note 36, International Earth Rotation and Reference Systems Service, Frankfurt am Main, 2010.

Polanskey, C. A. et al. Psyche mission description and design rationale. *Space Science Reviews*, 221:95, 2025. doi: 10.1007/s11214-025-01218-x.

Pontryagin, L. S. *Mathematical Theory of Optimization Processes*. CRC Press, Boca Raton, FL, USA, 1962.

Rao, A. V. A survey of numerical methods for optimal control. *Advances in the Astronautical Sciences*, 135: 497–528, 2009.

Rayman, M. D. and Varghese, P. The deep space 1 extended mission. *Acta Astronautica*, 48(5–12):693–705, 2001. doi: 10.1016/S0094-5765(01)00044-3.

Rayman, M. D. and Williams, S. N. Design of the first interplanetary solar electric propulsion mission. *Journal of Spacecraft and Rockets*, 39(4):589–595, 2002. doi: 10.2514/2.3848.

Rayman, M. D., Fraschetti, T. C., Raymond, C. A., and Russell, C. T. Coupling of system resource margins through the use of electric propulsion: Implications in preparing for the dawn mission to ceres and vesta. *Acta Astronautica*, 60(10–11):930–938, 2007.

Rayman, M. D. Lessons from the Dawn mission to Ceres and Vesta. *Acta Astronautica*, 176:233–237, 2020. doi: 10.1016/j.actaastro.2020.06.023.

Ricciardi, L. A., Maddock, C. A., and Vasile, M. Multi-objective optimisation under uncertainty with unscented temporal finite elements. *Mathematics*, 9(23), 2021.

Richards, A. G. and How, J. P. Robust stable model predictive control with constraint tightening. In *Proceedings of the 2006 American Control Conference*, pages 1557–1562. IEEE, 2006. doi: 10.1109/ACC.2006.1656440. URL `https://doi.org/10.1109/ACC.2006.1656440`.

Ridderhof, J. and Tsiotras, P. Minimum-fuel powered descent in the presence of random disturbances. In *AIAA Guidance, Navigation, and Control Conference*, number AIAA 2019-0646, San Diego, CA, January 2019. doi: 10.2514/6.2019-0646.

Ridderhof, J. and Tsiotras, P. Chance-constrained covariance steering in a gaussian random field via successive convex programming. *Journal of Guidance, Control, and Dynamics*, 45(4):599–610, 2022. doi: 10.2514/1.G005941.

Ridderhof, J., Okamoto, K., and Tsiotras, P. Nonlinear uncertainty control with iterative covariance steering. In *2019 IEEE 58th Conference on Decision and Control (CDC)*, pages 3484–3490. IEEE, 2019. doi: 10.1109/cdc40024.2019.9029993.

Ridderhof, J., Pilipovsky, J., and Tsiotras, P. Chance-constrained covariance control for low-thrust minimum-fuel trajectory optimization. In *AAS/AIAA Astrodynamics Specialist Conference*, AAS 20-618, 2020.

Ridderhof, J., Tsiotras, P., and Johnson, B. J. Stochastic entry guidance. *Journal of Guidance, Control, and Dynamics*, 45(2):320–334, 2022. doi: 10.2514/1.G005964.

Ridolfi, G., Mooij, E., Dirkx, D., and Corpino, S. Robust multi-disciplinary optimization of unmanned entry capsules. In *Proceedings of the 2012 AIAA Modeling and Simulation Technologies Conference*, number AIAA-12-5006, 2012.

Ritt, J. F. *Differential algebra*, volume 33. American Mathematical Society, 1950.

Robert, C. and Casella, G. Monte carlo statistical methods. chapter 3, pages 79–122. Springer New York, 2004. doi: 10.1007/978-1-4757-4145-2.

Römgens, B., Mooij, E., and Naeije, M. Satellite collision avoidance prediction using verified interval orbit propagation. *Journal of Guidance, Control, and Dynamics*, 36(3):821–832, 2013.

Rubinsztejn, A., Sood, R., and Laipert, F. E. Neural network optimal control in astrodynamics: Application to the missed thrust problem. *Acta Astronautica*, 176:192–203, 2020. doi: 10.1016/j.actaastro.2020.05.027.

Sanchez-Sanchez, C., Izzo, D., and Hennes, D. Learning the optimal state-feedback using deep networks. In *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 1–8. IEEE, 2016. doi: 10.110 9/SSCI.2016.7850215.

Sanders, J. and Kandrot, E. *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley, 1 edition, 2010.

Selim, A. and Ozkol, I. Robust trajectory optimization of re-entry flight with prescribed endpoint region via sparse grid ensemble pseudospectral optimal control. *Advances in Space Research*, 72(11):5102–5128, 2023.

Shepard, M. K. et al. Radar observations and shape model of asteroid 16 psyche. *Icarus*, 281:388–403, 2017. doi: 10.1016/j.icarus.2016.08.011.

Shepard, M. K. et al. Asteroid 16 psyche: Shape, features, and global map. *The Planetary Science Journal*, 2(4):125, 2021. doi: 10.3847/PSJ/abfdba.

Silva, J. G. Robust atmospheric entry trajectory optimisation through uncertainty propagation. MSc thesis, Delft University of Technology, Delft, The Netherlands, February 2025. Faculty of Aerospace Engineering.

Smith, J., Taber, W., Drain, T., Evans, S., Evans, J., Guevara, M., Schulze, W., Sunseri, R., and Wu, H.-C. MONTE python for deep space navigation. In *Proceedings of the 15th Python in Science Conference*, Austin, Texas, July 2016. doi: 10.25080/Majora-629e541a-009.

Snyder, J. S., Lenguito, G., Frieman, J. D., Haag, T. W., and Mackey, J. A. Effects of background pressure on SPT-140 hall thruster performance. *Journal of Propulsion and Power*, 36(5):668–676, 2020. doi: 10.251 4/1.B37702.

Soyata, T. *GPU Parallel Program Development Using CUDA*. Chapman & Hall/CRC Computational Science. CRC Press, Boca Raton, FL, 1 edition, 2018. ISBN 978-1-4987-5075-2.

Sun, Y. and Kumar, M. Uncertainty propagation in orbital mechanics via tensor decomposition. *Celestial Mechanics and Dynamical Astronomy*, 124:269–294, 2016.

Sun, Z. J., Luo, Y. Z., di Lizia, P., and Zazzera, F. B. Nonlinear orbital uncertainty propagation with differential algebra and gaussian mixture model. *Science China: Physics, Mechanics and Astronomy*, 62(3), 2019.

Tardioli, C., Kubicek, M., Vasile, M., Minisci, E., and Riccardi, A. Comparison of non-intrusive approaches to uncertainty propagation in orbital mechanics. In *Proceedings of the 2015 AAS/AIAA Astrodynamics Specialist Conference*, pages 3979–3992, 2015.

Taylor, J. Handbook for the direct statistical analysis of missile guidance systems via cadet trademark. Technical Report TR-385-2, The Analytic Sciences Corp. Report, 1975.

Tracy, K., Zhang, J. Z., Arrizabalaga, J., Schaal, S., Tassa, Y., Erez, T., and Manchester, Z. The trajectory bundle method: Unifying sequential-convex programming and sampling-based trajectory optimization, 2025.

Tsuda, Y., Mori, O., Funase, R., Sawada, H., Yamamoto, T., Saiki, T., Endo, T., Yonekura, K., Hoshino, H., and Kawaguchi, J. Achievement of IKAROS—japanese deep space solar sail demonstration mission. *Acta Astronautica*, 82(2):183–188, 2013. doi: 10.1016/j.actaastro.2012.03.032.

Vittaldev, V., Mooij, E., and Naeije, M. C. Unified state model theory and application in astrodynamics. *Celestial Mechanics and Dynamical Astronomy*, 112(3):253–282, March 2012. doi: 10.1007/s10569-011-9396-5.

Vittaldev, V., Russell, R. P., and Linares, R. Spacecraft uncertainty propagation using gaussian mixture models and polynomial chaos expansions. *Journal of Guidance, Control, and Dynamics*, 39(12):2615–2626, 2016.

Wächter, A. and Biegler, L. T. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106:25–57, 2006.

Wagner, S., Kaplinger, B., and Wie, B. GPU accelerated genetic algorithm for multiple gravity-assist and impulsive δv maneuvers. In *AIAA/AAS Astrodynamics Specialist Conference*, Minneapolis, MN, USA, August 2012. American Institute of Aeronautics and Astronautics.

Wakker, K. F. *Fundamentals of Astrodynamics*. Delft University of Technology, Delft, The Netherlands, 2015. URL `https://resolver.tudelft.nl/uuid:3fc91471-8e47-4215-af43-718740e6694e`.

Wang, F., Yang, S., Xiong, F., Lin, Q., and Song, J. Robust trajectory optimization using polynomial chaos and convex optimization. *Aerospace Science and Technology*, 92:314–325, 2019.

Wang, J., Abdul, A. T., and Theodorou, E. A. cuNRTO: GPU-accelerated nonlinear robust trajectory optimization, 2026. arXiv preprint, version 2.

Watanabe, S.-i., Tsuda, Y., Yoshikawa, M., Tanaka, S., Saiki, T., and Nakazawa, S. Hayabusa2 mission overview. *Space Science Reviews*, 208(1–4):3–16, 2017. doi: 10.1007/s11214-017-0377-1.

Weisman, R., Majji, M., and Alfriend, K. T. Solution of liouville's equation for uncertainty characterization of the main problem in satellite theory. *Tech Science Press CMES*, 111(3):269–304, 2016.

Whittle, L. and Sagliano, M. Stochastic optimal trajectory generation via multivariate polynomial chaos. In *Proceedings of the 2018 AIAA Guidance, Navigation, and Control Conference*, number AIAA-18-0849, 2018.

Wiener, N. The homogeneous chaos. *American Journal of Mathematics*, 60(4):897–936, 1938.

Wittig, A., Vasile, M., and Izzo, D. On the use of GPUs for massively parallel optimization of low-thrust trajectories. In *6th International Conference on Astrodynamics Tools and Techniques (ICATT)*, Darmstadt, Germany, 2016.

Xiong, F., Xiong, Y., and Xue, B. Trajectory optimization under uncertainty based on polynomial chaos expansion. In *AIAA Guidance, Navigation, and Control Conference*, AIAA SciTech Forum, 2015. doi: 10.2514/6.2015-1761.

Xu, Y. and Xin, M. Nonlinear stochastic control for space launch vehicles. *IEEE Transactions on Aerospace and Electronic Systems*, 47(1):98–108, 2011.

Xu, Y. Nonlinear robust stochastic control for unmanned aerial vehicles. *Journal of guidance, control, and dynamics*, 32(4):1308–1319, 2009.

Yang, X., Luo, Y., and Zhang, J. Nonlinear uncertainty propagation based on state transition tensors. *Science China: Physics, Mechanics and Astronomy*, 62(3), 2019. Check this entry against the original source: the thesis text cites Yang et al. (2016), but the bibliography extraction around this item was ambiguous.

Yi, Z., Cao, Z., Theodorou, E., and Chen, Y. Nonlinear covariance control via differential dynamic programming, 2019.

Zarchan, P. Complete statistical analysis of nonlinear missile guidance systems-slam. *Journal of guidance and control*, 2(1):71–78, 1979.

Zavoli, A. and Federici, L. Reinforcement learning for robust trajectory design of interplanetary missions. *Journal of Guidance, Control, and Dynamics*, 44(8):1440–1453, 2021. doi: 10.2514/1.G005794.

Ziebart, M. Generalized analytical solar radiation pressure modeling algorithm for spacecraft of complex shape. *Journal of Spacecraft and Rockets*, 41(5):840–848, September 2004. doi: 10.2514/1.13097.

Zimmer, S., Ocampo, C., and Bishop, R. Reducing orbit covariance for continuous thrust spacecraft transfers. *IEEE Transactions on Aerospace and Electronic Systems*, 46(2):771–791, 2010. doi: 10.1109/TAES.2010.5461656.

Zou, Y. and Jiang, F. GPU-accelerated pseudospectral methods for optimal control problems. *Mathematics*, 13(20):3252, 2025. doi: 10.3390/math13203252.

APPENDIX

A1. RESEARCH PLAN

This section outlines the structured research plan established to guide and schedule the progression of this research study. Section A1.1 provides an in-depth overview of each work package (WP), detailing their objectives, required inputs, expected outputs, and associated tasks. Following this, Section A1.2 offers a personal evaluation of the thesis progress, reflecting on accomplishments and areas for improvement. Lastly, Section **??** showcases the Gantt chart illustrating the timeline and interdependencies of the WPs.

A1.1. WORK BREAKDOWN STRUCTURE

This master's thesis focuses on the development and assessment of a GPU-accelerated stochastic trajectory optimisation method for low-thrust interplanetary missions. The method is designed to generate optimal solutions that are robust to uncertainties, errors, and stochastic disturbances. The Psyche mission is adopted as the reference case for evaluating the validity and performance of the proposed methodology. The research activities are organised according to a Work Breakdown Structure (WBS) comprising seven main work packages.

WP1: Initial Simulator Set-up and Verification

The initial phase of this research focuses on establishing the baseline software and dynamical model required for the subsequent deterministic and stochastic optimisation activities.

Input: Equations of motion, reference-frame definitions, physical parameters of the Psyche mission obtained from the literature review, SPICE kernels, Tudat, and the source code and technical documentation of the Trajectory Bundle Method (TBM).

Tasks:

1. Implement the utilities required to nondimensionalise and redimensionalise the variables used in all numerical computations. *[1 working day]*
2. Implement the low-fidelity propagator in both the Solar System barycentre and Mars-centred ICRF frames. The dynamical model shall include the commanded thrust acceleration and the point-mass gravitational accelerations of the Sun, Earth, and Mars.
   [1 working day]
3. Implement and execute unit tests for the nondimensionalisation routines and the individual force model components. *[1 working day]*
4. Read the technical documentation of the Trajectory Bundle Method and inspect the code architecture. *[3 working days]*
5. Perform an acceptance test of the Trajectory Bundle Method code. *[1 working day]*
6. Retrieve the precise epoch and spacecraft state vector associated with the Psyche mission's Mars gravity assist using the SPICE toolkit. *[1 working day]*
7. Execute a subsystem test of the augmented-state propagation under the low-fidelity dynamical model, including consistency checks against Tudat. *[1 working day]*
8. Document results in the thesis report. *[1 working day]*
9. Buffer. *[1 working day]*

Expected outcome: Verified nondimensionalisation utilities, a verified low-fidelity propagator, a tested installation of the TBM source code, and a mission-consistent Mars gravity-assist epoch and state vector.
Pauses: None.
Duration: 11 working days, from 6 July to 20 July 2026.

WP2: Deterministic Optimisation and Numerical Integration

This work package develops the deterministic optimal control problem that provides the nominal reference trajectory for the subsequent stochastic formulations. It also establishes and verifies the ordinary and stochastic differential equation integration routines required by the complete optimisation framework.

Input: The mission boundary and path constraints, the spacecraft control limits and the deterministic optimal control formulation derived from the literature review. The verified low-fidelity propagator developed in the previous work package and the TBM source code.

Tasks:

1. Investigate and implement a systematic method for generating an initial guess for the deterministic optimal-control problem. *[4 working days]*
2. Define and implement the mathematical formulation of the deterministic optimal control problem. *[1 working day]*
3. Execute the optimisation algorithm to obtain a converged nominal reference trajectory. *[1 working day]*
4. Perform a trade-off analysis of candidate ordinary differential equation integrators. *[2 working days]*
5. Perform an acceptance test of the selected ordinary differential equation integration routine against an independent reference solution. *[1 working day]*
6. Perform a comparative analysis of candidate stochastic differential equation integration methods. *[2 working days]*
7. Perform an acceptance test of the selected stochastic differential equation integration routine using a problem with known and reproducible stochastic moments. *[1 working day]*
8. Document the results in the report. *[1 working day]*
9. Buffer. *[1 working day]*

Expected outcome: A converged deterministic reference trajectory, verified ordinary and stochastic differential equation integration routines.
Pauses: None.
Duration: 14 working days, from 21 July to 7 August 2026.

WP3: Simplified Stochastic Optimisation and Uncertainty Propagation

This work package extends the deterministic problem to a stochastic optimal control problem in which the initial spacecraft state is represented by a multivariate Gaussian distribution. A parallel implementation of the uncertainty propagation through Unscented Transform is developed, while accumulated process noise is incorporated at the end of each time discretisation arc through additive covariance matrices reconstructed along the reference trajectory.

Input: The deterministic reference trajectory obtained in WP2, the verified ordinary differential equation and stochastic differential equation integration routines, the multicore CPU and GPU parallel-programming environments, and the theoretical foundations of Unscented Transform uncertainty propagation and dynamical disturbance modelling derived from the literature review.

Tasks:

1. Implement the stochastic optimal control problem formulation for a multivariate Gaussian initial state distribution. *[1 working day]*
2. Design and implement a synchronisation strategy for parallel trajectory propagations at the Mars SOI entry and exit events. *[4 working days]*
3. Implement parallel Unscented Transform uncertainty propagation for the state and control distributions. *[3 working days]*
4. Perform a subsystem test of the parallel Unscented Transform uncertainty propagation implementation. *[1 working day]*
5. Implement the reconstruction of the covariance matrices related to the process noise accumulation along the deterministic reference trajectory. *[2 working days]*

6. Perform a subsystem test of the accumulated process noise covariance reconstruction routine. *[1 working day]*
7. Document the results in the report. *[1 working day]*
8. Buffer. *[2 working days]*

Expected outcome: An implemented stochastic optimal control problem formulation, a verified parallel Unscented Transform implementation, event-synchronised propagation through the Mars gravity-assist phase, and a method for the reconstruction of the process-noise additive covariance matrices.
Pauses: Holiday from 17 August to 21 August 2026.
Duration: 15 working days, from 10 August to 4 September 2026, excluding the holiday period.

WP4: Optimisation Algorithm Selection and Non-Gaussian Initial Distribution Extension

This work package selects the optimisation algorithm used in the final robust trajectory optimisation framework and extends the Gaussian formulation to non-Gaussian initial state uncertainties. The initial probability density is approximated automatically using a Gaussian Mixture Model, after which the individual mixture components are propagated in parallel and incorporated into the ROCP formulation.

Input: The Gaussian stochastic optimal control problem and the parallel Unscented Transform uncertainty propagation implementation developed in WP3. Candidate nonlinear-programming algorithms. Theory of Gaussian Mixture Model PDF approximation and risk-allocation methods derived from the literature review.

Tasks:

1. Perform a comparative analysis of candidate optimisation algorithms. *[3 working days]*
2. Perform an acceptance test of the selected optimisation algorithm. *[1 working day]*
3. Implement an automatic Gaussian Mixture Model approximation routine for initial-state probability distributions. *[2 working days]*
4. Implement and execute unit tests of the Gaussian Mixture Model PDF approximation routine. *[1 working day]*
5. Parallelise the propagation of the Gaussian Mixture Model components and their associated sigma points. *[4 working days]*
6. Perform a subsystem test of the parallel GMM component propagation. *[1 working day]*
7. Extend the robust optimal control problem formulation to non-Gaussian initial state uncertainties by combining the Gaussian Mixture Model representation with a naive risk-allocation strategy. *[2 working days]*
8. Perform a low-fidelity system test of the SOCP solution using an independent Monte Carlo campaign. *[2 working days]*
9. Document the results in the report. *[1 working day]*
10. Buffer. *[2 working days]*
11. **Mid-Term Review** (2 October 2026).

Expected outcome: A selected and verified optimisation algorithm, an automatic Gaussian Mixture Model PDF approximation routine, parallel propagation of the mixture components and sigma points, a robust formulation applicable to non-Gaussian initial state uncertainties, and preliminary Monte Carlo validation results.
Pauses: None.
Duration: 19 working days, from 7 September to 2 October 2026.

WP5: GPU Performance Analysis and High-Fidelity Dynamics Definition

This work package evaluates whether the developed formulation meets the computational performance and robustness objectives of the research. The multicore CPU and GPU implementations are compared using equivalent numerical settings, after which the physical fidelity of the propagation model is increased. A benchmark is defined to quantify the accuracy of the low-fidelity model, and the force, environment, and spacecraft models are subsequently refined.

Input: The tested low-fidelity robust optimisation framework developed in WP4, the multicore CPU and GPU implementations and profiling tools.

Tasks:

1. Compare the performance of the multicore CPU and GPU implementations while tuning and profiling the parallel implementations. *[5 working days]*
2. Define a benchmark for assessing the physical accuracy of the propagator along the reference trajectory. *[1 working day]*
3. Compare the low-fidelity propagation against the benchmark and determine the required extensions to the force, spacecraft, and environment models. Implement the selected high-fidelity configuration. *[2 working days]*
4. Document the results in the report. *[1 working day]*
5. Buffer. *[1 working day]*

Expected outcome: Quantified GPU speed-up and scalability relative to the multicore CPU implementation, an assessment of the numerical consistency between the two implementations, quantified physical model accuracy and a selected high-fidelity dynamical model.
Pauses: None.
Duration: 10 working days, from 5 to 16 October 2026.

WP6: High-Fidelity Dynamics Refinement and System Verification

In this work package, the deterministic and stochastic optimisation problems are solved again using the selected high-fidelity configuration. Subsequently, the multicore CPU and GPU implementations are compared using the updated dynamical settings. Finally, sensitivity studies are conducted to assess the uncertainty and error sources affecting the mission.

Input: The high-fidelity dynamical model obtained in WP5, the multicore CPU and GPU implementations, profiling tools, the reference mission uncertainty and error model defined through the literature review.

Tasks:

1. Solve the deterministic optimal control problem using the high-fidelity dynamical model. *[2 working days]*
2. Solve the stochastic optimal control problem using the high-fidelity model while tuning and profiling the parallel implementations. *[4 working days]*
3. Perform a high-fidelity system test of the SOCP solution using a Monte Carlo campaign. *[2 working days]*
4. Compare the performance of the multicore CPU and GPU implementations incorporating the high-fidelity dynamical model. *[2 working days]*
5. Perform sensitivity analyses with respect to the initial state uncertainty, diffusion matrix, navigation error and manoeuvre-execution error parameters. *[3 working days]*
6. Buffer. *[2 working days]*

Expected outcome: Updated deterministic and robust trajectories, with the SOCP solution validated using a high-fidelity Monte Carlo campaign; revised quantification of the GPU speed-up and scalability relative to the multicore CPU implementation; an assessment of the numerical consistency between the two implementations; and the results of the sensitivity analysis.
Pauses: Holiday from 9 November to 13 November 2026.
Duration: 15 working days, from 19 October to 13 November 2026.

WP7: Finalisation

This work package concludes the research project. Its main purpose is to finalise the structure and content of the thesis report, incorporate the feedback received during the Green Light Review, and prepare for the public thesis defence.

Input: An advanced thesis draft and the complete set of results generated during the preceding work packages.

Tasks:

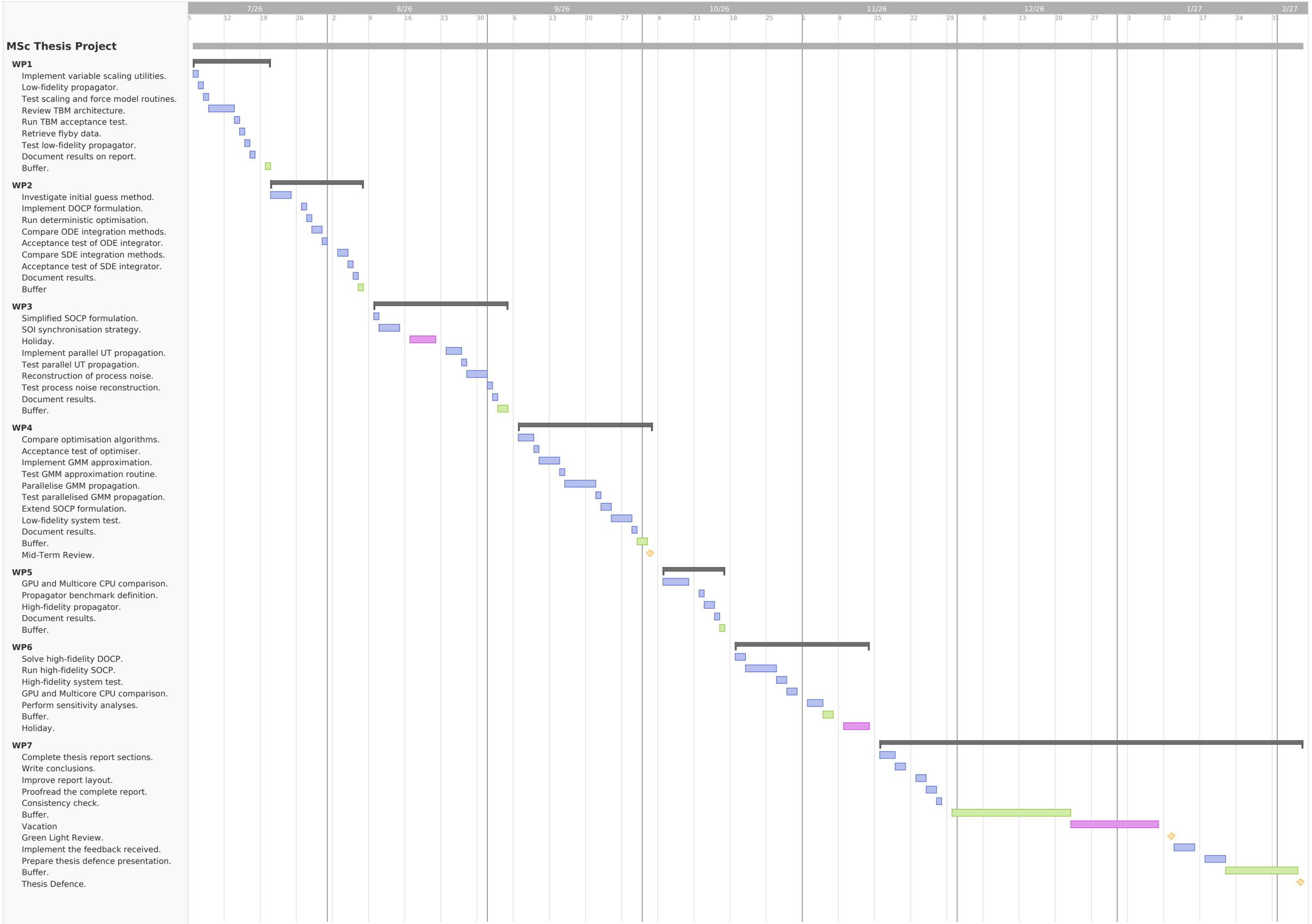
1. Complete and consolidate the sections of the thesis report. *[3 working days]*

2. Write the conclusions and recommendations based on the research results. *[2 working days]*
3. Improve the report layout. *[2 working days]*
4. Proofread the complete report. *[2 working days]*
5. Perform a consistency check of the nomenclature and bibliography. *[1 working day]*
6. Buffer. *[17 working days]*
7. **Green Light Review** (11 January 2027).
8. Implement the feedback received. *[4 working days]*
9. Prepare the thesis-defence presentation. *[4 working days]*
10. Buffer. *[10 working days]*
11. **Thesis Defence** (5 February 2027).

Pauses: Holiday from 23 December 2026 to 8 January 2027.
Duration: 51 working days, from 16 November 2026 to 11 February 2027.

A1.2. REFLECTION

At the end of this research work, it is valuable to reflect on the adequacy of the original research plan ...


7/26
8/26
9/26
10/26
11/26
12/26
1/27
2/27
5 12 19 26 2 9 16 23 30 6 13 20 27 4 11 18 25 1 8 15 22 29 6 13 20 27 3 10 17 24 31
MSc Thesis Project
WP1
Implement variable scaling utilities.
Low-fidelity propagator.
Test scaling and force model routines.
Review TBM architecture.
Run TBM acceptance test.
Retrieve flyby data.
Test low-fidelity propagator.
Document results on report.
Buffer.
WP2
Investigate initial guess method.
Implement DOCP formulation.
Run deterministic optimisation.
Compare ODE integration methods.
Acceptance test of ODE integrator.
Compare SDE integration methods.
Acceptance test of SDE integrator.
Document results.
Buffer
WP3
Simplified SOCP formulation.
SOI synchronisation strategy.
Holiday.
Implement parallel UT propagation.
Test parallel UT propagation.
Reconstruction of process noise.
Test process noise reconstruction.
Document results.
Buffer.
WP4
Compare optimisation algorithms.
Acceptance test of optimiser.
Implement GMM approximation.
Test GMM approximation routine.
Parallelise GMM propagation.
Test parallelised GMM propagation.
Extend SOCP formulation.
Low-fidelity system test.
Document results.
Buffer.
Mid-Term Review.
WP5
GPU and Multicore CPU comparison.
Propagator benchmark definition.
High-fidelity propagator.
Document results.
Buffer.
WP6
Solve high-fidelity DOCP.
Run high-fidelity SOCP.
High-fidelity system test.
GPU and Multicore CPU comparison.
Perform sensitivity analyses.
Buffer.
Holiday.
WP7
Complete thesis report sections.
Write conclusions.
Improve report layout.
Proofread the complete report.
Consistency check.
Buffer.
Vacation
Green Light Review.
Implement the feedback received.
Prepare thesis defence presentation.
Buffer.
Thesis Defence.

A2. DYNAMICAL DISTURBANCE MODEL

Let us consider the linear stochastic system obtained by linearizing Equation 3.58 along a reference trajectory $(\hat{\boldsymbol{x}}(t), \hat{\boldsymbol{u}}(t))$

$$d\boldsymbol{x} = (\mathbf{A}(t)\boldsymbol{x} + \mathbf{B}(t)\boldsymbol{u} + \mathbf{c}(t))\, dt + \mathbf{G}(t)d\boldsymbol{w} \tag{A2.1}$$

where

$$\boldsymbol{A}(t) = \frac{\partial \boldsymbol{f}(\hat{\boldsymbol{x}}(t), \hat{\boldsymbol{u}}(t), t)}{\partial \boldsymbol{x}}, \qquad \boldsymbol{B}(t) = \frac{\partial \boldsymbol{f}(\hat{\boldsymbol{x}}(t), \hat{\boldsymbol{u}}(t), t)}{\partial \boldsymbol{u}}, \qquad \boldsymbol{G}(t) = \boldsymbol{g}(\hat{\boldsymbol{x}}(t), \hat{\boldsymbol{u}}(t), t), \tag{A2.2}$$

$$\boldsymbol{c}(t) = \mathbf{f}(\hat{\boldsymbol{x}}(t), \hat{\boldsymbol{u}}(t)) - \boldsymbol{A}(t)\hat{\boldsymbol{x}}(t) - \boldsymbol{B}(t)\hat{\boldsymbol{u}}(t) \tag{A2.3}$$

Assume a time grid discretization $t_0 \leq t_1 \leq \ldots \leq t_N = t_f$, and a piecewise constant control law in each sub-interval, that is

$$\boldsymbol{u}(t) = \boldsymbol{u}(t_k) = \boldsymbol{u}_k, \qquad t \in [t_k, t_{k+1}), \qquad \forall k = 0, \ldots, N-1 \tag{A2.4}$$

Equation (A2.1) can be written in the discrete form (Ridderhof et al., 2020)

$$\boldsymbol{x}_{k+1} = \boldsymbol{A}_k \boldsymbol{x}_k + \boldsymbol{B}_k \boldsymbol{u}_k + \boldsymbol{c}_k + \boldsymbol{G}_k \boldsymbol{w}_k \tag{A2.5}$$

with

$$\boldsymbol{A}_k = \boldsymbol{\Phi}(t_{k+1}, t_k), \qquad \boldsymbol{B}_k = \int_{t_k}^{t_{k+1}} \boldsymbol{\Phi}(t_{k+1}, s)\boldsymbol{B}(s)\, ds, \qquad \boldsymbol{c}_k = \int_{t_k}^{t_{k+1}} \boldsymbol{\Phi}(t_{k+1}, s)\boldsymbol{c}(s)\, ds \tag{A2.6}$$

where $\boldsymbol{\Phi}(t, s) = \boldsymbol{A}(t)$ is the state transition matrix and $\boldsymbol{G}_k$ is a matrix so that the stochastic process $\boldsymbol{G}_k \boldsymbol{w}_k$ has covariance $\boldsymbol{Q}_k$, evaluated as

$$\boldsymbol{Q}_k = \int_{t_k}^{t_{k+1}} \boldsymbol{\Phi}(t_{k+1}, s)\boldsymbol{G}(s)\boldsymbol{G}(s)^T \boldsymbol{\Phi}(t_{k+1}, s)^T\, ds \tag{A2.7}$$

Recalling that $\boldsymbol{\Phi}(s, t_{k+1}) = \boldsymbol{\Phi}(s, t_k)\boldsymbol{\Phi}(t_k, t_{k+1})$ and $\boldsymbol{\Phi}(t_k, s) = \boldsymbol{\Phi}(s, t_k)^{-1}$, Equation (A2.7) can be rewritten as

$$\boldsymbol{Q}_k = \boldsymbol{A}_k \left(\int_{t_k}^{t_{k+1}} \boldsymbol{\Phi}^{-1}(s, t_k)\boldsymbol{G}(s)\boldsymbol{G}^T(s) \left(\boldsymbol{\Phi}^{-1}(s, t_k)\right)^T ds \right) \boldsymbol{A}_k^T \tag{A2.8}$$